

SPARTADOS



- _____
- _____
- _____
- _____



Programming
Guide
DLT

SpartaDOS X

Programming Guide

V. 4.50

English version:

Konrad Kokoszkiewicz

© 2024 DLT

Note - throughout this manual:

SpartaDOS, SpartaDOS X, SpartaDOS Toolkit, MAC/65, and DOS XL are trademarks of FTe - all Abandonware.

All 8-bit hardware and software products are trademarks of Atari Corp. - Abandonware.

Axlon RAMPOWER is a trademark of Axlon, Inc - Abandonware.

MyDOS is Copyright © 1988 by WORDMARK Systems, Public Domain Software.

SpartaDOS X User Guide is Copyright © 2015 - 2024 by DLT

Contents

Cover

Preface

1 Symbols

1.1	What a symbol is	3
1.2	SYMBOL structure	3

2 SpartaDOS X Binary File Format

2.1	Binary block types	5
2.2	RAM allocation block (BSS)	5
2.3	Position-independent binary block	5
2.4	Fix-ups	6
2.5	Required symbols (XREFs)	7
2.5.1	Weak symbols	7
2.6	Symbol definitions (XDEFs)	7

3 Memory Management

3.1	SpartaDOS X memory usage	9
3.2	Conventional RAM (MALLOC)	10
3.3	Allocation of system extended area (MALLOC)	11
3.4	Allocation of program extended area (MALLOC)	11
3.5	Memory allocation in TSR programs (INSTALL)	12
3.6	Memory indices (MALLOC)	12
3.7	Extended bank access	13
3.7.1	Direct data transfers to and from extended RAM	14
3.7.2	XRWMEM	14
3.7.3	System extended area access (MALLOC, EXTENDED)	14
3.7.4	Program extended area access (MALLOC, EXTENDED)	15
3.8	Memory index to register value conversion (T_)	16
3.8.1	Indices \$00 and \$02	16
3.8.2	Indices \$01 and \$03	16
3.8.3	All other indices	16
3.9	Technical information on extended RAM	17

4 Error Handling

4.1	Standard error handler (U_FAIL)	19
4.2	Setting a trap (U_SFAIL)	19
4.3	Removing a trap (U_XFAIL)	19
4.4	Error message (U_ERROR/U_PERROR)	20
4.5	Error trapping example	20

5 Command Line Processing

5.1	LBUF buffer, BUFOFF index	21
5.2	COMFNAM buffer	21
5.3	Parsing command line parameters	21
5.4	Text parameters (U_GETPAR)	22
5.5	Numeric parameters (U_GETNUM)	22
5.6	Addresses (U_GETADR)	22
5.7	ON/OFF switch (U_GONOFF)	23

5.8	Options (U_SLASH)	23
5.9	Keyword (U_GETPAR/U_TOKEN)	24
5.10	Device identifier (U_GETPAR/U_GEFINA)	25
5.11	File specification (U_GETPAR/U_GEFINA)	25
5.12	Directory specification (U_GETPAR/U_GEPATH)	25
5.13	File specification and attributes (U_GETATTR)	26
5.14	File specification with a default mask (U_FSPEC)	26
5.15	Various parameter types combined	27
5.16	Getting loader flags (U_GETFLG)	27
5.17	Other functions (U_PARAM, _CRUNCH)	28
6	Processing File and Path names	
6.1	8+3 to internal format conversion (U_GEFINA)	29
6.2	Helper function PRO_NAME	29
6.3	Internal format to 8+3 conversion (U_EXPAND)	29
6.4	Internal format to path string conversion (U_PATH)	29
7	Environment Variables	
7.1	What is an environment variable	31
7.2	Getting a variable by its name (GETENV)	31
7.3	Getting a variable by its number (NUMENV)	31
7.4	Putting and deleting variables (PUTENV)	31
8	Accessing Files	
8.1	Opening a file (FOPEN)	33
8.1.1	Access mode	33
8.1.2	File search attributes	34
8.1.3	Attributes assigned	34
8.2	Closing files (FCLOSE/FCLOSEAL)	34
8.3	Reading and writing single bytes (FGETC/FPUTC)	35
8.3.1	Micro-buffers	35
8.4	Reading and writing text records (FGETS/FPUTS)	35
8.5	Reading and writing binary blocks (FREAD/FWRITE)	36
8.6	Obtaining file's length (FILELENG)	36
8.7	Obtaining and changing current file's position (FTELL/FSEEK)	36
8.8	Writing formatted text to a file (FPRINTF)	37
9	Console I/O	
9.1	Writing single characters to the console (PUTC)	39
9.2	Writing a text record to the console (PUTS)	39
9.3	Printing formatted text (PRINTF)	39
9.4	Reading single characters from the console (GETC)	42
9.5	Reading a text record from the console (GETS)	43
9.6	I/O redirection (DIVIO/XDIVIO)	43
9.7	Vectored output (PUT_V/VPRINTF)	44
9.8	Reading a byte from keyboard (U_GETKEY)	44
10	Directories	
10.1	Searching for files (FFIRST/FNEXT)	47
10.2	Reading formatted directory (FDOPEN/FDGETC/FDCLOSE)	47

11 Loading Binaries	
11.1 Loading a program to memory (U_LOAD)	49
11.2 Unloading a program (U_UNLOAD)	50
12 File Management Functions	
12.1 Rename file (RENAME)	51
12.2 Rename directory (RENDIR)	51
12.3 Delete file (REMOVE)	51
12.4 Delete directory (RMDIR)	51
12.5 Making directory (MKDIR)	52
12.6 Change current directory (CHDIR)	52
12.7 Get current directory (GETCWD)	52
12.8 Changing attributes (CHMOD)	52
12.9 Set BOOT file (SETBOOT)	52
13 Other Disk Related Functions	
13.1 Format disk (FORMAT)	53
13.2 Soft formatting (BUILDDIR)	53
13.3 Get disk parameters (GETDFREE)	53
13.4 Change volume name (CHVOL)	54
13.5 Get and change current drive (CURDEV)	54
14 Auxiliary Functions	
14.1 32-bit multiplication and division (MUL_32/DIV_32)	55
14.2 Converting characters to upper-case (TOUPPER)	55
14.3 Check for path separator (CKSPEC)	55
15 Initializations	
15.1 Initializing TSR programs after a RESET (S_ADDIZ)	57
15.2 Unconditional exit to DOS (_DOS)	57
15.3 SpartaDOS X warm restart (_INITZ)	57
16 Manipulating The Symbol List	
16.1 Looking up the symbols (S_LOOKUP)	59
16.2 Walking through symbol list (S_NEXT)	59
16.3 Adding a symbol (S_ADD)	60
16.4 Removing symbols (S_CLEAR)	60
17 Remaining Symbols	
18 Some Programming Techniques	
18.1 Example application program	63
18.1.1 FA's syntax	63
18.1.2 ELSA's syntax	63
18.1.3 MADS's syntax	64
18.1.4 Commentary	65
18.2 Example program 2: Reading data from a file	65
18.3 Example 3: A resident program	66
18.3.1 With the use of system extended RAM	66
18.3.2 With the use of program extended RAM	67
18.3.3 Common optimizations	68
18.4 Converting ASCII digits to a value	68

18.5	SPRINTF	70
18.6	Kernel device types	71
18.6.1	Checking for a special device	71
18.6.2	Checking for a file device	72
18.6.3	Files vs pseudo-files	72
18.7	Decoding device identifier	73
18.8	Checking for existence of a file	74
18.9	Starting programs with parameters	74
18.9.1	Starting a program directly with U_LOAD	75
18.9.2	Starting a program through COMMAND.COM	76
18.9.3	Returning status to the parent program	77
18.9.4	Loading overlays	77
18.9.5	Redirecting output of a child program	78
18.9.5.1	Redirecting to a file	78
18.9.5.2	Redirecting to memory	78

19 65C816 Support

19.1	The 65816.SYS driver	81
19.1.1	Calling legacy functions	81
19.1.1.1	The Rapidus OS style interface	82
19.1.1.2	Stack-indirect calls	83
19.1.1.3	Custom jump tables	85
19.1.2	Symbol remarks (S_LOOKUP/S_ADD)	85
19.1.2.1	Symbol manipulation (H_SFIRST/H_SNEXT/H_SADD)	85
19.1.3	Binary file format remarks (U_LOAD)	86
19.1.3.1	Headers	86
19.1.3.2	The relocation scheme	86
19.1.3.3	Extended relocatable binary block	88
19.1.4	Error handling	89
19.1.4.1	Setting a trap in 24-bit address space (H_SFAYL)	89
19.1.4.2	Removing a trap (H_XFAIL)	90
19.1.4.3	Standard error handler (H_FAIL)	90
19.1.5	Memory management	90
19.1.5.1	65C816 High RAM	90
19.1.5.2	High RAM allocation: memory index \$03 (MALLOC)	91
19.1.5.3	Checking how much free RAM is available (MALLOC)	91
19.1.5.4	Freeing the allocated blocks explicitly (MALLOC)	92
19.1.5.5	Wrappers for MALLOC (H_MALLOC, H_MFREE)	92
19.1.5.6	High RAM access	93
19.1.5.7	XRWMEM	93
19.1.5.8	Copying RAM blocks (H_MEMCPY)	94
19.1.5.9	Filling RAM block with a constant (H_MEMSET)	95
19.1.5.10	Initializing TSR programs after a RESET (H_SADDIZ)	95
19.2	The ENV816.SYS module (H_GETENV/H_PUTENV/H_NUMENV)	96
19.2.1.1	H_GETENV	96
19.2.1.2	H_PUTENV	97
19.2.1.3	H_NUMENV	97
19.3	The SIO816.SYS module (H_SIOADD)	98
19.4	The EXT816.SYS module	99

19.4.1	Binary loader (H_LOAD)	99
19.4.2	Opening files (H_FOPEN, H_FILE_P)	100
19.4.3	Accessing files	100
19.4.3.1	Using FREAD/FWRITE on the High RAM	100
19.4.3.2	Large data transfers (H_FREAD/H_FWRITE)	101
19.4.3.3	Text records (H_FGETS/H_FPUTS)	102
19.4.4	Formatted text output	102
19.4.4.1	Output to the console (H_PRINTF)	103
19.4.4.2	Output to a file (H_FPRINTF)	103
19.4.4.3	Vectored output (H_VPRINTF, H_PUT_V)	103
19.4.4.4	Output to a buffer (H_SPRNPF)	104
19.4.5	Integer multiplication and division	105
19.4.5.1	16-bit unsigned integer division (H_DIVU16)	105
19.4.5.2	16-bit signed integer division (H_DIVS16)	105
19.4.5.3	16-bit unsigned integer multiplication (H_MULU16)	106
19.4.5.4	16-bit signed integer multiplication (H_MULS16)	106
19.4.5.5	32-bit unsigned integer division (H_DIVU32)	107
19.4.5.6	32-bit signed integer division (H_DIVS32)	107
19.4.5.7	32-bit unsigned integer multiplication (H_MULU32)	108
19.4.5.8	32-bit signed integer multiplication (H_MULS32)	108
19.4.6	Floating point support	109
19.4.6.1	32-bit integer to FP conversion (H_IFP32/H_IFP32S)	109
19.4.6.2	FP to 32-bit integer conversion (H_FPI32/H_FPI32S)	109
19.4.6.3	FP addition and subtraction (H_FADD/H_FSUB)	110
19.4.6.4	FP multiplication and division (H_FMUL/H_FDIV)	110
19.4.7	Other support	110
19.4.7.1	Stopwatch (H_TIME)	110
19.4.7.2	Pseudo-random number generator (H_RANDOM)	111
19.4.7.3	TOUPPER/TOLOWER	111
19.5	Replacement calls for the cartridge functions	112
20	Using SORTDIR.OVL	
20.1	Function	115
20.2	Initializing	115
20.3	Memory usage	116
20.4	Preparing directory data	116
20.5	Invoking the sorting procedure	117
20.6	Accessing buffered directory	118
20.7	65C816 remarks	119
21	Using SCANDIR.OVL	
21.1	Function	120
21.2	Principles of operation	120
21.3	Usage	120
21.3.1	Example loader code: FIND.COM	120
21.3.2	Initialization (SD_INIT)	121
21.3.3	The main loop (SD_FIRST/SD_NEXT)	122
21.3.4	Internal variables (SD_VARS)	122
21.3.5	Paths (SD_DPATH/SD_FPATH)	123
21.3.6	Call-backs (SD_INCB/SD_EXCB)	123

22 SpartaDOS X SIO Interface

22.1	Function	124
22.2	Calling convention	124
22.2.1	XDCB structure	124
22.3	Adding your device to the system	125
22.3.1	Control tables and variables	125
22.3.2	The entry point	125
22.3.3	Installing your driver	126
22.4	Basic operation	127
22.4.1	The function of sio_index	128
22.4.2	Global RAM locations your driver may use internally	129
22.4.3	Memory management within the driver	129
22.4.4	(X)DCB interpretation steps	131
22.5	65C816 support	132

23 Block Data Management Interface (Block I/O)

23.1	Function	133
23.2	Calling convention	133
23.3	Function codes	134

24 SpartaDOS X Kernel Interface

24.1	Function	135
24.2	Calling convention	135
24.3	Page 7 kernel parameter table	136
24.4	General distribution of kernel function codes	137
24.5	Character devices	138
24.5.1	Character device definition	138
24.6	File devices	139
24.6.1	File device definition	139
24.7	Special devices	141
24.8	Adding your device to the system	142
24.8.1	The entry point	142
24.8.2	Setting up driver's memory	142
24.8.3	Registering the driver	143
24.8.4	Enabling micro-buffers	144
24.8.5	Configuring for the OSS CLI	145
24.8.6	Initialization	145
24.8.7	Completion	145
24.9	Operation	146
24.9.1	Assigning file handles	146
24.9.2	Global RAM locations your driver may use internally	146
24.9.3	Memory management within the driver	147

25 Misc_ Interface

25.1	Function	148
25.2	Calling convention	148
25.3	List of function codes	148

26 Hwctrl Interface

26.1	Function	150
26.2	Calling convention	150

26.3 List of function codes 150

27 SpartaDOS X Block Diagram

Index

Backcover

Spine

Preface

The „SpartaDOS X Programming Guide“ is aimed at coders who are willing to write applications and system programs for SpartaDOS X (SDX). Authors presume that the reader has a knowledge on Atari programming and is an advanced user of SpartaDOS X, therefore information covered in „SpartaDOS X User Guide“ is skipped here as being obvious. It is also assumed that terms such as „PORTB“ do not need to be explained.

The assembly listings are written in pseudo-code modelled after the MAE assembler syntax. It is close to the MAC/65 syntax, and differs in details from the syntax used by the cross-assembler MADS. The programmer would be willing to adjust it to the preferred assembler.

This edition is expanded, mainly with additional chapters on the SpartaDOS X kernel and other subsystems. The remaining text has been revised, which caused a number of inaccuracies to get corrected. Also, many memory locations have been relabelled to follow more closely the customs of the original authors of the SpartaDOS X code.

Whatever you are doing, please keep in mind that SpartaDOS X is not Windows nor Linux – it is more like MS-DOS: a traditional operating system written with the assumption that a software developer would not do openly stupid or malicious things to the system environment. Intentionally or not, it is rather easy to crash the system or impair its stability, and its means to prevent that are rather limited. Therefore it is advisable to write programs, and especially the system drivers, for SpartaDOS X very carefully – just as it generally is with Atari 8-bit programming.

We wish you pleasant reading

DLT

1 Symbols

1.1 What a symbol is

While writing a SpartaDOS X programming guide, it is a good idea to begin with an explanation of the term *symbol* which the reader will often meet in this book.

A symbol in SpartaDOS X is a kind of extended pointer. It contains information on where in the memory an object (a variable, a table or a subroutine) is located. One symbol usually corresponds to one such object. There are about 100 symbols in total, and they form a linked list. This list provides access to the most important – from the programmer's point of view – of SpartaDOS X's subroutines and internal structures, as well as to TSR programs and drivers, because resident programs are (naturally) allowed to add their own symbols to the list.

This symbol list is also called the external symbol dictionary. The operating system exports them, and the programs being loaded import them. The entity responsible for resolving the references is the binary loader.

The greatest advantage of such an approach is that the system routines, being pointed to by symbols, are not associated with fixed addresses. Their addresses may vary between DOS versions, but programs written for an older DOS version will still run on a newer one, despite the fact that the system function in question is called with a direct call. Moreover, if a new symbol is defined with a name of an already existing symbol, this new symbol replaces that old one. This way a TSR program or a loader may intercept calls an application is making to system routines.

1.2 SYMBOL structure

A description of the SYMBOL structure does not seem necessary at all, because programs usually do not use it in a way, which would require the programmer's interaction. But this information may at least appease the reader's curiosity.

A symbol consists of 13 or 14 bytes, as follows:

- \$00-\$01: pointer to next symbol (2 bytes)
- \$02-\$09: symbol name (8 ASCII characters)
- \$0A: program level index
- \$0B-\$0D: target address (2 or 3 bytes)

Pointer to next symbol is a 16-bit address, the symbols form an unidirectional linked list, and this is the link. Note that the link is an information internal to the symbol database and therefore its exact meaning may depend on the actual implementation of that database. In any case, you should never use this link directly.

Symbol name meets criteria similar to a file name: it consists of eight ASCII characters, and most of the time these characters are upper case letters, the underscore character and @. If the name contains less than eight characters, it is space padded.

In **Program level index** two most significant bits contain the index of the memory which the symbol points to: it is a %00 for the main memory, %10 for the extended memory, oth-

er values reserved. Six lower bits form the so called PID (*Program identifier*): it is a number assigned to the program which owns the symbol. A zero here means the system library, which will be described later on.

Target address is the address of the object the symbol points to, in the memory space specified by the program level index byte (above).

The functions of the library which allow symbol list manipulations are described in one of the chapters below.

CAUTION: when a program is being executed with the X command, the symbol list becomes unavailable until its (the program's) termination.

2 SpartaDOS X Binary File Format

2.1 Binary block types

AtariDOS recognizes only one type of a binary block, it is a segment with a six-byte header, which begins with an optional signature \$FFFF. This format is widely documented, so the description will be skipped here.

SpartaDOS X knows seven types of binary blocks. One of these is, naturally, the AtariDOS-like segment mentioned above. A fixed-address („absolute”) SpartaDOS X segment is almost identical, the only difference is that the header's signature is \$FFFA instead of \$FFFF. This change prevents other DOS-es from loading such binaries, as the format's purpose is to contain SpartaDOS X system files (such as drivers etc.)

The remaining five types are:

- 1) RAM allocation block (BSS)
- 2) position-independent binary block
- 3) fix-ups
- 4) symbols required by program (XREFs)
- 5) symbols defined by program (XDEFs)

2.2 RAM allocation block (BSS)

A RAM allocation block, as the name indicates, tells the binary loader to allocate an amount of memory of an indicated type. Such a block consists solely of the following 8-byte header:

\$00-\$01:	signature \$FFFE
\$02:	block number
\$03:	control byte, \$80 + index of the memory to be allocated
\$04-\$05:	offset to the beginning of the program (or \$0000)
\$06-\$07:	amount of bytes to allocate

The memory will get allocated at an address which is pointed to by the corresponding free memory pointer (i.e. for example, „over *memlo*” in the conventional memory). The memory type is indicated by the memory index stored in the control byte. The possible values of that index here are: \$x0 conventional RAM, \$x2 system extended area, (as of SpartaDOS X 4.47) \$x4 program extended area, other values reserved. Memory indexes will be further explained in the chapter 3 „Memory management”.

As of SpartaDOS X 4.43, if the control byte has its 6th bit set (+\$40), the address of the block being allocated will be first rounded up to the closest free page boundary (which results in page-alignment of the block).

The offset field must be either \$0000 in all blocks of this type or carry cumulative size of all previous program blocks of this type. See section 2.6 below for further explanation.

2.3 Position-independent binary block

A position independent binary block has a header, which is almost identical to the one found in RAM allocation block:

\$00-\$01: signature \$FFFE
 \$02: block number
 \$03: control byte, \$00 + index of the memory to be allocated
 \$04-\$05: offset to the beginning of the program (or \$0000)
 \$06-\$07: amount of bytes to allocate/load

Two things make these blocks different: first, the control byte has its 7th bit cleared here; it is a signal for the loader that there is data to load behind the header. The other thing is just this fact. The binary data is loaded to the address indicated by the corresponding free memory pointer. The memory type is carried by the index stored in the control byte, as above.

As of SpartaDOS X 4.43, if the control byte has its 6th bit set (+\$40), the address of the block being allocated will be first rounded up to the closest free page boundary (which results in page-alignment of the block).

The offset field must be either \$0000 in all blocks of this type or carry cumulative size of all previous program blocks of this type. See section 2.6 below for further explanation.

2.4 Fix-ups

A block containing fix-up information for a position independent binary block has a five-byte header:

\$00-\$01: signature \$FFFD
 \$02: base block's number: its load address will be the base address of relocation
 \$03-\$04: size of the fix-up block in bytes (or 0)

The first byte after that is usually \$FE followed by another byte: the latter specifies the block number inside which the fix-ups will take place (target block).

The bytes which follow contain offsets inside the target block. The first byte is relative to the beginning of the target block, any next one is relative to the immediately preceding one. In other words, a fix-up byte means „advance the given number of bytes and fix-up”.

Fixing the address up is done so that the address of the base block is each time added to a two-byte word, which is found at the address indicated by the corresponding fix-up byte inside the target block (i.e. the one being relocated).

Caveat: A fix-up byte always points to a two-byte word. Therefore a position independent binary **may not** contain a value of its internal address split into lower and higher order byte stored separately. Such constructions as:

```

lda #<address
ldx #>address

```

or separate tables containing low or high bytes of pointers are not allowed.

The above meaning has any fix-up byte with value lower than \$FC. Values \$FC, \$FD and \$FF are additional control codes:

\$FF: advance 250 bytes (and do nothing)
 \$FE: change the target block number to the number specified by the next byte
 \$FD: change the position within the target block to the absolute 16-bit address specified by the following word, and do fix-up (this is always being done on the „main“ memory, i.e. *memreix*=0, and refers to the position-dependent, „absolute“ binary blocks of type \$FFFA)
 \$FC: end of the fix-up information

2.5 Required symbols (XREFs)

Blocks which begin with a signature \$FFFB form a list of symbols which are required by the program being loaded. In other words, these symbols must already be defined in the system, when the program is being loaded. A lack of one causes the loading to abort and error 154 (Symbol not defined) is returned.

\$00-\$01: signature \$FFFB
 \$02-\$09: symbol name (8 characters)
 \$0A-\$0B: fix-up block size in bytes (or 0)

The following data are fix-up and control bytes, in identical manner as in the fix-up block. The address pointed to by the symbol is added to a two-byte word found at the address indicated by the fix-up byte.

2.5.1 Weak symbols

As of SpartaDOS X 4.50 the system loader implements a concept of „weak“, or optional, symbols. These work just like the regular („strong“) symbols, with the exception that if a program marks one of its required symbols as a „weak“ one, and the definition of that symbol is not found in the symbol database during loading of the said program, the loading continues normally instead of being aborted with error 154. In this case the references to weak symbols are not fixed up, and it is up to the program to check later if the symbol definition was found during loading.

This is useful when a program relies on an optional device driver (such as *S_VBXE.SYS*), but wants to fall back to default system services or own routines, when the main device driver is not present for a reason: checking for its presence is then reduced to a check if a reference pointer to the symbol in question is zero or not, instead of going the hard way with calls to *S_LOOKUP* etc.

A weak symbol record is almost identical to the regular symbol record, with the sole exception that the eighth (or the last) character of the symbol name within the \$FFFB record is in the inverse video (= has its bit 7 set to 1).

2.6 Symbol definitions (XDEFs)

Symbol definition block makes the loader to add a new symbol to the global list of symbols. Structure:

\$00-\$01: signature \$FFFC
 \$02: number of the binary block which defines the symbol
 \$03-\$04: symbol offset

\$05-\$0C: symbol name

The address pointed to by the new symbol is the load address of the binary block indicated by the byte \$02, plus the „symbol offset”, **and** minus the offset field for the same block.

For example, if a program consists of two \$FFFE blocks, the first one carrying a value of 100 in its offset field, the other carrying 200 there, then, if the latter one defines a symbol which points to its very beginning, the offset field in the XDEF block **will be increased by 100**. That is, in this example it will be 100 instead of the expected 0. While installing the symbol and setting up its address in the memory, the loader will subtract this value from the effective address.

This explains why in the sections above the offset field in the relocatable block's header is described as „offset to the beginning of the program (or \$0000)” – this field has no other function besides obfuscating the information where the XDEFs point to, so if they are kept all \$0000, their cumulative value will be \$0000 as well, thus allowing the *symbol offset* to be a real offset from the beginning of its block (0 in the example above).

Additional information regarding 65C816 extensions to this format can be found in the section 19.1.3.

3 Memory Management

3.1 SpartaDOS X memory usage

The following memory areas are discerned by SpartaDOS X:

- 1) Main memory: the conventional 48 KiB RAM (from *memlo* to *memtop*).
- 2) System extended area: the extra 14 KiB RAM („under ROM”, areas \$C000-\$CFFF and \$D800-\$FFFF) in 800XL and 65XE computers; or a 16 KiB bank of extra RAM (area \$4000-\$7FFF) in computers with PORTB-type or Axlon-type RAM extensions. If there is not enough this memory, the main RAM is allocated instead.
- 3) Program extended area: up to 252 banks (4032 KiB) in area \$4000-\$7FFF in computers with PORTB-type or Axlon-type RAM extensions.¹⁾

The category of Program extended area exists as of SpartaDOS X 4.47.

- 4) ROM cartridge: 128 KiB ROM at \$A000-\$BFFF.
- 5) Optional 65C816 high RAM: up to 16256 KiB in area \$010000-\$FF0000 on computers equipped with the 65C816 processor. This memory is not normally used by the system as a whole, but individual programs (including resident drivers) may take advantage of its presence. Also note that the last 64k segment at \$FF0000-\$FFFFFF is reserved.

The system allocates memory for own purposes as follows:

- 1) The kernel in main RAM, from \$0700 to *memlo*.
- 2) The system drivers in system extended area.
- 3) The system library in ROM cartridge.
- 4) The CAR: device in ROM cartridge.

The *system library* which was already mentioned before, contains a collection of functions, which go between user programs and the DOS kernel, and one application program – namely the *SpartaDOS X Formatter*. The library occupies two banks (8 KiB each) of the ROM cartridge in SpartaDOS X 4.20, and three such banks as of SpartaDOS X 4.40. The operation and use of the system library is discussed throughout the main part of this guide.

The main library module (bank no. 1) is normally present in \$A000-\$BFFF, but it can be switched off and replaced with RAM, if the program is executed with the X.COM command. It is normally practised with Atari DOS binaries (with the \$FFFF header).

The RAM for all the system resources is allocated depending on whether the particular RAM area exists or not, and depending what the user has chosen in the CONFIG.SYS file. There are four combinations available:

¹ In fact, a PORTB-type extension may contain up to 64 banks (1024k).

- 1) Using the main memory: USE NONE in the CONFIG.SYS. All the DOS components are loaded to the main memory, starting from address \$0700. This is the default on an Atari 800 which is equipped with not more than 48 KiB RAM.
- 2) Using the RAM „under ROM”: USE OSRAM in the CONFIG.SYS. The kernel takes up the memory from \$0700 to *memlo*, the rest of the DOS components are loaded under the ROM at \$E400-\$FFBF (7104 bytes); 2 KiB at \$D800 are occupied by internal data structures (as of SpartaDOS X 4.40 – that area was left unused before), the \$C000-\$CFFF remains free, the rest occupies the main memory. This is the default on an Atari XL/XE with not more than 128 KiB RAM.
- 3) Using the RAM „under ROM”: USE OSRAM and DEVICE SPARTA OSRAM in the CONFIG.SYS. As above, but the area \$C000-\$CFFF is allocated for DOS buffers, thus diminishing the usage of the main memory (the *memlo* will be lower).
- 4) Using the extended RAM: USE BANKED in the CONFIG.SYS. The kernel takes up the memory from \$0700 to *memlo*, the rest of the DOS components, data and buffers are loaded occupy a „system extended area” 16 KiB bank at \$4000-\$7FFF. This is the default for an Atari equipped with the Axlon RAM extension, or if an XL/XE computer has more than 128 KiB RAM.

SpartaDOS X functions allow an easy access to extended banks of RAM, both system extended area and program extended area.

3.2 Conventional RAM (MALLOC)

The size of the free main memory is calculated the traditional way, subtracting the value of *memlo* (\$02E7/8) from the value of *memtop* (\$02E5/6). The *memlo* always points to the beginning of the free memory, i.e. it gets set so that it points to the memory available over the loaded programs. This however only applies to position independent modules (see chapter 2).

To allocate this RAM a function may be used called MALLOC.²⁾ It makes sure that allocated blocks fit in the memory and do not overlap.

MALLOC requires the following parameters:

- *bytbuf* (\$0785/6) block size;
- the Y register must be zeroed;
- the X register must contain the index of the desired memory: for the conventional RAM its value is \$00. If the block has to be page-aligned, add \$40 to the index (alignment works as of SpartaDOS X 4.43).

MALLOC returns with a positive value on success (N=0), the low word of *addpos* (\$0782/3) will then contain the address of the allocated block, and the X register – the index of the allocated memory. On an out of memory condition the result is negative (N=1).

²⁾ Note that the label „MALLOC” is not tied to a fixed address: it is a symbol which is dynamically resolved to an actual address when your program is being loaded. The symbol definition syntax may vary between assemblers, more on that in chapter 18.

Example of allocating 1025 bytes of conventional memory:

```
lda #<1025
sta bytbuf
lda #>1025
sta bytbuf+1
ldx #$00                      ;conventional RAM, unaligned
ldy #$00
jsr MALLOC
bmi out_of_memory_error
...
```

The area allocated so will be protected by raising *memlo* past its end. The memory will be released when the program, which called MALLOC, terminates.

3.3 Allocation of system extended area (MALLOC)

The MALLOC function may also be used to allocate blocks of RAM in system extended area. The parameters are as above, with the exception of the memory index: for the system extended area its value is \$02 (or \$42, when the block has to be page-aligned).

Let us try to allocate 1025 bytes from that area:

```
lda #<1025
sta bytbuf
lda #>1025
sta bytbuf+1
ldx #$02                      ;extended system RAM, unaligned
ldy #$00
jsr MALLOC
bmi out_of_memory_error
...
```

When an out of memory condition occurs while allocating the extended memory, MALLOC will fall back to the main memory instead. The returned X register value informs which memory type was really allocated: \$02 if the request was satisfied from the extended RAM, or \$00 if the request was satisfied from the conventional RAM. *It is important to save this value in a private variable, because it will be needed later to gain access to the allocated memory.*

A way to access the system extended area allocated so is described below (section „Extended bank access“).

As it is with the conventional memory, the area allocated so will be released when the program, which called MALLOC, terminates.

3.4 Allocation of program extended area (MALLOC)

As of SpartaDOS X 4.47 MALLOC can allocate program extended area. The procedure is similar as above, with exceptions as follows:

- 1) The memory index for program extended area is \$04 (or \$44 if the block is to be page-aligned);

- 2) a maximum size of a block to be allocated at a time is 16384 bytes. That is, the size of a single bank. An attempt to allocate a bigger block causes an error regardless of the total amount of RAM remaining free;
- 3) when there is no more free memory of this type, no attempt is made to fall back to another type, but an error condition is returned immediately;
- 4) on success (N=0) the X register contains the index of the allocated bank (every bank has its own, individual memory index).

Example:

```

lda #<1025
sta bytbuf
lda #>1025
sta bytbuf+1
ldx #$04                ;extended program RAM, unaligned
ldy #$00
jsr MALLOC
bmi out_of_memory_error
...

```

Again, it is crucial to save the returned X value, as it specifies which bank was allocated.

A way to access the program extended area allocated so is described on page 13.

As it is with the conventional memory, the area allocated so will be released when the program, which called MALLOC, terminates.

3.5 Memory allocation in TSR programs (INSTALL)

As it was said above, any memory allocated by a program is released when the program terminates. To prevent this – in resident programs – a program, before it exits to DOS, must write a value of \$FF to a variable pointed to by symbol INSTALL.

INSTALL is zeroed before program execution, so it is enough to decrement the variable:

```
dec INSTALL
```

before quitting to DOS with an RTS.

3.6 Memory indices (MALLOC)

Memory indices are of two different kinds:

- 1) Indices of requested memory. Such one is passed on to the MALLOC function as one of its parameters. There are only three indices of this type: \$00 for conventional memory, \$02 for system extended area and \$04 for program extended area. A \$40 may be added to every one of them, as it was said above. Any other value is reserved and an attempt to use it may cause undefined behaviour.

- 2) Indices of granted memory. These are returned by `MALLOC`, when it succeeds. There are 256 indices of this type. In case of the extended RAM, saving such a granted index in program's private variable is essential when it comes to accessing the allocated memory later.

Of that latter type, index \$00 means conventional RAM, \$02 system extended area (which always occupies at most one bank of memory), and indices \$01 and \$03 are reserved for future usage (these two are never returned by `MALLOC`). The remaining indices are assigned to the banks of program extended area, up to 252 – which means that up to 4032 KiB of memory may be addressed that way.

An actual number of valid indices of course depends on the size of the RAM extension. The banks are always assigned numbers from 4 upwards. It follows that on a stock 130XE (4 banks) the indices of banks are 4, 5, 6 and 7 respectively. On a 320 KiB Atari (16 banks) there will be 16 valid indices, from 4 to 19, and so on.

Up to (and including) SpartaDOS X 4.46, only system indices \$00 and \$02 could have been used that way. An access to remaining banks was troublesome, therefore as of SpartaDOS X 4.47 the memory handler has been expanded to assign individual memory indices to all the extension banks.

3.7 Extended bank access

Switching in an extended RAM bank is done by calling the `V_setme` (\$07F1) function with the appropriate memory index given in the accumulator. Switching out and restoring the previous memory setup – because the main memory was not necessarily active before a call to `V_setme` – is done by `V_popme` (\$07F4). That last function does not require any parameters.³⁾

Calling `V_setme` must later always have a counterpart call to `V_popme`, when the extended RAM access is not necessary anymore. The calls can be nested, up to 16 nested calls are possible, but in general all switches in must be paired by the same number of switches out (it works in a stack-like manner).

The argument for `V_setme` must be a memory index returned by `MALLOC`. It cannot be a constant, because, first, at the time of doing the call it is unknown, if the extended RAM exists at all, and second, even if it does, if any of its portion has been allocated for us by the DOS.

Attention should be paid to the fact, that the code calling these functions must be located outside the area \$4000-\$7FFF. In case of a position-independent binary file it may be difficult to accomplish, but, for instance, a relevant piece of code may be put on page 6.

`V_setme` preserves registers X, Y and the CPU flag register, and `V_popme` additionally the accumulator.

³ In previous versions of this documentation `V_setme` and `V_popme` were called `jext_on` and `jext_off` respectively. The labels have been changed to match the ones used by the original developers at ICD.

3.7.1 Direct data transfers to and from extended RAM

It is of course possible to make direct reads from a file to the extended banks, and a write in the other direction, see FREAD and FWRITE (chapter 8.5).

3.7.2 XRWMEM

In a rare case when other ways are inconvenient or impossible, a function called XRWMEM may be useful. It allows „safe“ reading and writing single bytes across the entire memory, and accepts parameters as follows:

- A 16-bit address in *addpos* (\$0782/3);
- an 8-bit offset in the Y register;
- index of the relevant memory (the „granted“ type) in the X register;
- operation type in the C flag (C=0 read, C=1 write);
- data to be written in the accumulator (this is ignored when reading).

On error (e.g. no memory for the given index) XRWMEM returns C=1, and C=0 means success.

Peek into SPARTA.SYS memory in BANKED mode:

```
addpos = $0782

    lda #<$4000
    sta addpos
    lda #>$4000
    sta addpos+1
    ldy #0
    ldx #2
    clc
    jsr XRWMEM
    bcs no_access_error
    ...
```

The X and Y registers are preserved across the call, and the accumulator when writing.

This function is used by SpartaDOS commands PEEK, POKE and MDUMP. It is rather slow, so it should not be used to access or transfer larger data blocks.

3.7.3 System extended area access (MALLOC, EXTENDED)

The system bank access is only needed for programs which allocated a part of it for themselves, by calling MALLOC explicitly or implicitly. Explicit usage of MALLOC was described above; an implicit call occurs, when a position independent binary file is loaded, and at least one of its binary blocks is to be automatically loaded to the system extended area. That is – unless there is no free place in it: in that case the main RAM is used instead.

After an explicit allocation, to gain the access to the allocated block, the memory index returned by MALLOC in the X register should be passed in the accumulator to the *V_setme* (\$07F1) function.

After an implicit call, the index of the memory where at least one block was loaded to, can be found in the variable pointed to by the symbol EXTENDED. The program, especially if it is a TSR, should remember that value in a private variable (EXTENDED being zeroed upon return to DOS) and later, during execution, pass that value to *V_setme* (\$07F1) wherever it is necessary to reference the part loaded to the extended RAM:

```
V_setme = $07f1
V_popme = $07f4

    lda EXTENDED
    sta our_mem
    ...
    lda our_mem
    jsr V_setme
    ...
    jsr V_popme
    ...
                                ;interaction with extended RAM bank
```

In any case, switching the bank off and restoring the memory state is, as said above, accomplished by a call to *V_popme* (\$07F4).

3.7.4 Program extended area access (MALLOC, EXTENDED)

The program banks can be accessed using the very same *V_setme* (\$07F1) function mentioned above. The memory index should be passed to it in the accumulator. Of course, it cannot be an arbitrary constant, but a value received from a procedure of allocating a block, as described above in section 3.3 and later.

As said above, switching the bank off and restoring the memory state is accomplished by a call to *V_popme* (\$07F4).

An implicit allocation (by the binary loader) is also possible. What is more, a program may this way fill several banks of extended RAM before execution.

A count of all, position-independent binary blocks loaded is available at EXTENDED+1. To be specific, this is a count of all binary blocks, for which the MALLOC was called by the binary loader. These will be position-independent binary blocks and RAM allocation blocks of all types, that is, these loaded to the conventional memory, system extended area and program extended area.

Notice: while a program is being loaded, its very first block aimed at the program extended area is always loaded at the beginning of a new bank.

A question that remains is how a program can retrieve memory indices for several blocks loaded consecutively. These indices are available in a table located at EXTENDED+3. It is now enough to know the block number (considering the first one to be number 0) and use it as an offset in that table, to retrieve the relevant memory index. For example, this way:

```
ldx EXTENDED+1 ;number of the last block
lda EXTENDED+3-1,x ;its memory index
```

3.8 Memory index to register value conversion (T_)

It may be sometimes useful to convert a memory index to a „real“ value to be stored in the RAM controlling hardware register.

AtariDOS programs, which can not access the symbol list, using the information below can retrieve a list of PORTB banks which are not in use. For starters it is enough to know that the free banks occupy the range of indices from 4 to (COMTAB+\$1D)+3 – if the value of COMTAB+\$1D is not a zero.

The following rules apply here:

3.8.1 Indices \$00 and \$02

- 1) For Axlon extension: add the bank index to an address pointed to by the symbol T_. The resulting number is an address of a byte, where the register value is stored corresponding to the index.

```
;CONVERT SYSTEM MEM INDEX TO AXLON
;
    tax
    lda T_,x
```

- 2) For PORTB extension: do the same as above, except that a value of 4 should be added to the memory index before.

```
;CONVERT SYSTEM MEM INDEX TO PORTB
;
    tax
    lda T_+$04,x
```

3.8.2 Indices \$01 and \$03

These two are reserved and therefore your program should never need to deal with them, the less convert them to anything else. *See also the chapter 19, dedicated to the 65C816 processor extensions.*

3.8.3 All other indices

- 1) For Axlon extension: subtract 3 from the index. The result is the value to be stored in the Axlon controlling register (\$CFFF).

```
;CONVERT USER MEM INDEX TO AXLON
;
    sec
    sbc #$03
```

- 2) For PORTB extension: use table at T_+\$08, the following way (memory index in the accumulator):

```

;CONVERT USER MEM INDEX TO PORTB
;
    pha
    lsr
    lsr
    tax
    pla
    and #$03
    asl
    asl
    ora T_+$08-1,x

```

The result ends up in the accumulator. It should be remembered that the maximum possible index for a PORTB extension is 67, an attempt to convert greater values will return meaningless results.

Programs which cannot access the symbol list, to find the T_ table will have to use *jfsymbol* call described in chapter 16.

How to detect the extension type, it is hinted below.

3.9 Technical information on extended RAM

All the information on the memory configuration is contained in the COMTAB and COMTAB2 tables, pointed to by respective symbols. The former is additionally pointed to by the DOSVEC (\$0A-\$0B) pointer.

Caution: the DOSVEC value must not be considered a constant. The address this vector points to may vary between different SpartaDOS versions (and even between different SpartaDOS X editions) and there is no guarantee that it would not change in the future. Only the offsets relative to the DOSVEC are constants.

COMTAB2-\$02 (_EXTTYP) – if bit 7 set, then the memory extension is Axlon-compatible. This flag is available as of SpartaDOS X 4.46.

The total count of banks (Axlon or PORTB) detected at startup is available at **COMTAB2-\$03** (_EXTBNK). This variable is present as of SpartaDOS X 4.46.

COMTAB+\$1B (_800FLG) has a negative value, the computer is an Atari 800. This means that there is no RAM „under ROM”.

COMTAB+\$1C (PBMASK) contains ones on all bits, which, when set in PORTB, cause memory bank switching at \$4000-\$7FFF. The bit 4 is set too, and it should not be counted; on a stock 130XE the mask's value is \$1C, or %00011100. Less the bit 4, two bits are set, which means four additional memory banks, or 64 KiB. Similarly, on a 320 KiB Compy Shop extension the mask has a value of \$DC or %11011100. Less bit 4, four bits are set, which means 16 additional memory banks. If the extension is Axlon-compatible (or there is no extension at all) the mask is zeroed.

COMTAB+\$1D (NBANKS) contains the number of free banks of the program extended area. If zero, that means the extended RAM either does not exist at all or is completely occupied by DOS components: drivers, RAM-disks etc.

COMTAB+\$1E (BANKFLG): a negative value (e.g. \$FF) indicates that the system is loaded to the banked memory (USE BANKED).

COMTAB+\$1F (OSRMFLG): a negative value (\$FF or \$FE) indicates that the system is loaded to the RAM under the ROM (USE OSRAM). The banked memory, if it exists at all, may be used as a RAM-disk or allocated for other drivers which require it.

4 Error Handling

4.1 Standard error handler (U_FAIL)

Handling an error in SpartaDOS X consists in calling the library function U_FAIL passing the error code in the accumulator. U_FAIL may be explicitly called from the user's program, but most of the time it is done implicitly, when the system library calls it automatically, having found some anomalies during program's execution.

U_FAIL has one unpleasant feature: once called, it never returns. In other words, the calling program is aborted and unloaded, its open files get closed, and the system, having printed a suitable message on the console, hands the control back to the Command Processor. It is convenient when the error that occurred was a critical one and the program cannot continue execution anyway. But it is relatively often necessary to handle an error inside the program, without terminating it.

4.2 Setting a trap (U_SFAIL)

A program willing to take on handling of an error condition can set a trap. A procedure, which is used for that, is labelled U_SFAIL. LSB and MSB of the error handler address should be passed to it as arguments in A and X registers. Example:⁴⁾

```
lda errorp      ;'errorp' is a word containing address
ldx errorp+1    ;of error handler
jsr U_SFAIL
```

More than one trap can be set – when an error occurs, the one will react which has been set as last. But the number of the traps being active simultaneously should not be too big: a program may not maintain more than 10 traps at a time.

When an error occurs with the trap set, the trap is removed (it is a one-time thing). Next, the library loads the stack pointer with a value saved by U_SFAIL, switches the memory banks to the state saved therein, and does a JMP to the address of the last defined error handler, with the error code carried in the accumulator, and with the N flag of the CPU status register set to 1. The rest remains unchanged, especially the files, which the program was keeping open, are not closed.

4.3 Removing a trap (U_XFAIL)

As said, the trap is removed automatically when an error occurs. But when no error occurs, the trap most of the time becomes obsolete and it should be removed „manually“. The procedure, which does it, is U_XFAIL. It accepts no arguments, it simply removes a trap which was set as the last one:

```
jsr U_XFAIL
```

Calling the U_XFAIL does not change the CPU registers A, X and Y.

Up to SpartaDOS X 4.46 (inclusive) U_XFAIL has a bug which results in CPU flag register clobbering upon return.

⁴ Please remember that in SpartaDOS X relocatable binaries addresses cannot be split into separate LSB and MSB – hence no „lda #<error / ldx #>error“ here.

All traps which was set by a program are removed when the program terminates and the control goes back to the DOS.

4.4 Error message (U_ERROR/U_PERROR)

U_ERROR displays the error message corresponding to the error code passed in the accumulator. An EOL character is sent to the screen editor before and after that. It is a part of the the standard error handling procedure initiated by a call to U_FAIL.

As of SpartaDOS X 4.42 two functions subordinate to U_ERROR are available to the programmer. Their symbols are ERR_DMSG and ERR_GMSG. The former one is similar to the U_ERROR, i.e. it displays an error message corresponding to the code passed in the accumulator; but no EOLs are added before or after. The latter writes the message to a memory location, then returns its address in X and Y registers (LSB/MSB). The message is NUL-terminated (i.e. with ASCII 0).

As of SpartaDOS X 4.47 there is U_PERROR. It accepts an error code in the Y register, and in AX registers (LSB/MSB) the address of the buffer, to which it is supposed to write an EOL-terminated message corresponding to that code. The buffer's size may not be smaller than 64 bytes.

4.5 Error trapping example

```
;set the trap to the address contained in the vector
;trapptr, let it point to the place labelled 'trap'

    lda trapptr
    ldx trapptr+1
    jsr U_SFALL

;call system routine where we may expect an error to occur

    jsr ...

;on no error, remove the trap

    jsr U_XFAIL

;when error occurs, DOS removes the trap and hands control to ;this
;place with N=1 and error code in the accumulator

trap    bmi error

;here proceed when no error occurred
    ...
    rts

;handle the error here

error    jmp U_ERROR
```

5 Command Line Processing

5.1 LBUF buffer, BUFOFF index

A command for the DOS' Command Processor, either typed by the user or read from a batch file, gets stored in the *line buffer* LBUF. This buffer consists of 64 bytes and is to be found at COMTAB+\$3F. The current position in that buffer (i.e. the next parameter most of the time) is pointed to by the index called BUFOFF (*buffer offset*, COMTAB+\$0A).

The LBUF buffer and the BUFOFF index provide input data for the library routines designed to process the command line, therefore user programs normally do not need to be interested in what these contain. It can be however required to process a parameter more than once. In such a case the program should save the current state of the BUFOFF index and restore its contents before calling a library function again (all functions processing the command line automatically advance BUFOFF setting it to point to a next thing to be processed).

The first LBUF parameter is the name of the program being invoked. But upon its execution the BUFOFF usually points to the next parameter – that is because the program's name has already been processed by the Command Processor. Zeroing the BUFOFF to fetch that name is not recommended: better leave the things as they are (the reason is explained below).

5.2 COMFNAM buffer

The output place for the command line processing routines – at least for these which produce text results – is at COMTAB+\$21. This buffer is called COMFNAM (*complete file name*), its length is 30 bytes. It has an associated variable called FILLEN (*filename length* COMTAB+\$1A); it contains the length of the parameter copied to the COMFNAM.

While the LBUF contains the entire command line, the COMFNAM's purpose is to hold a single parameter. The LBUF lives immediately past the COMFNAM, these two buffers are virtually joined together. As a result, very long parameters (longer than 30 characters), fetched by the library from the LBUF and copied into the COMFNAM may overwrite the beginning of the LBUF.

That means in practice, that the first parameter of the command line (the one carrying the path and file name of the file to be executed) is only available for the Command Processor, and the executed program can not be sure if that part of the command line has not been already overwritten.

The COMFNAM buffer is terminated with an EOL character (\$9B).

5.3 Parsing command line parameters

A command line consists of a number of parameters, i. e. filenames, switches, options etc. separated with spaces or commas. The order and types of these parameters are usually predefined and known as the command syntax. This means, that the program which is performing the command, while parsing subsequent parameters, most of the time is able to foresee what type of a parameter will be parsed next. Doing that the program does not have to parse the command line the hard way – but, of course, it can when needed – because the library offers functions which facilitate parsing the most common types of para-

meters. We will now discuss such common cases, when a parameter's type is predefined and known, leaving more complex tasks for later.

5.4 Text parameters (U_GETPAR)

U_GETPAR copies the next (i.e. the one pointed to by BUFOFF) parameter from LBUF to COMFNAM, updates (advances) BUFOFF, stores the parameter's length into FILLEN and makes the FILE_P pointer point to COMFNAM.

When BUFOFF was already pointing to the end of the command line before the call – i.e. when all the parameters were already fetched – U_GETPAR does nothing and returns a zero result (Z=1). Otherwise the result will be non-zero (Z=0) and the X register will carry the length of the parameter which just has been copied to COMFNAM (so, X has the same value then as FILLEN has).

A program which displays a list of own parameters on the screen, looks as follows:

```
list    jsr U_GETPAR
        beq exit
        jsr PRINTF
        .byte "%s", $9b, 0
        .word COMTAB+$21      ;COMFNAM
        jmp list
exit    rts
```

The PRINTF function is explained in chapter 9.

5.5 Numeric parameters (U_GETNUM)

A numeric parameter is relatively the simplest one to parse. It may be a number in range from 0 to 65535 given in decimal or hexadecimal notation. A call to U_GETNUM returns a zero (Z=1), if the parameter is not a number. Otherwise (Z=0) in the AX registers there is LSB and MSB of the corresponding binary value, respectively.

As of SpartaDOS X 4.44 U_GETNUM can decode 32-bit numbers (either decimal or hexadecimal, within the range from 0 to 4294967295), but upon return the CPU registers still carry the least significant word of the value – the most significant word is to be found in the divend+2 and divend+3 registers (COMTAB-4 and COMTAB-3, respectively). For backward compatibility it is a good practice to zero out these two bytes before calling U_GETNUM.

U_GETNUM is used for instance by PEEK and POKE commands in the Command Processor.

5.6 Addresses (U_GETADR)

As of SpartaDOS X 4.47 the library contains a new function, similar to the U_GETNUM described above: U_GETADR. Its purpose is to convert a memory address to a binary form. The address may be given by the user in one of four forms:

- a) In a numeric form as a 16-bit, decimal or hexadecimal number.
- b) In a numeric form as above but with memory index specified before the actual address, and separated with a colon, for instance: \$02:\$4000.

- c) In a text form as a symbol name: the symbol, if it exists, will be converted to an address.
- d) In a numeric form as a 24-bit, decimal or hexadecimal number.

A call to `U_GETADR` returns a zero ($Z=1$), if the parameter does not exist, or it cannot be converted to an address. Otherwise ($Z=0$), the Carry flag contains 0 for an address which fits in 6502 16-bit address space, or 1 otherwise. Therefore:

- If $C=0$, there is LSB and MSB of the corresponding binary value in the AX registers, respectively, and the Y register holds the memory index.
- If $C=1$, the AX registers contain LSB and MSB of the low word of the address, respectively, and the Y register contains the 64k segment number (or, in other words, the upper 8 bits of a 24-bit address).

As of SpartaDOS X 4.48, `U_GETADR` automatically evaluates expressions consisting of the address specified as above, the '+' or '-' character, and a numeric constant. It allows to specify addresses for example in this form: `COMTAB2-11`. The arithmetic operation only applies to the low 16 bits of the address, if the result exceeds this limit, the (possible) third byte of the address remains unaffected.

`U_GETADR` is used for instance by `PEEK` and `POKE` commands in the Command Processor.

5.7 ON/OFF switch (`U_GONOFF`)

Some commands are used only to enable or disable something: the user is supposed to type either `ON` or `OFF` as a parameter. The function `U_GONOFF` is designed to parse such a parameter. Upon return, when the parameter is `ON`, the C flag in the CPU status register is set to 1, and to 0, when it is `OFF`. When the parameter is neither `ON` nor `OFF`, the function enters the error 156 (Bad parameter) condition transferring the control to `U_FAIL`, therefore aborting the program. The chapter 4 „Error handling“ explains how to deal with such a problem.

The `KEY.COM` program parses the command line this way.

5.8 Options (`U_SLASH`)

It is sometimes convenient to pass some parameters as one-character „options“ preceded with a slash character `/`. To parse them it is best to use the `U_SLASH` function. All available options must be put into an array. One entry in that array, which corresponds to one option, consists of two bytes, as follows: 1) the flag signaling that the option was given in the command line, and 2) the corresponding characters. For example, if a program wants to react on `/A` and `/X`, the array should look as follows:

```
switches
sw_a    .byte 0, "A"
sw_x    .byte 0, "X"
```

The address of the array must be passed on to U_SLASH in the AX registers (LSB and MSB respectively), and Y should hold its size in bytes. If one of the expected options is found at the command line position being analyzed, the corresponding flag (sw_a and sw_x in the example above) is set to \$FF, or to \$00 otherwise.

As of SpartaDOS X 4.47 it is possible to obtain information on how many times the user has given a particular switch in the command line. For the example above, to know, how many instances of /A was given, do this:

```

lda sw_a
eor #$ff
clc
adc #$01

```

The result is in the accumulator: 0 for none, 1 for /A, 2 for /AA, 3 for /AAA and so on. On older versions the above procedure will only return either 0 (for no switch) or 1 (for any number given).

When an option is encountered in the command line, which is missing in the table, U_SLASH transfers the control to U_FAIL aborting the program with error 156 (Bad parameter). And when the parameter currently being processed is not an option (i.e. it does not start with „/”), U_SLASH does nothing and just returns to the caller.

As of SpartaDOS X 4.47 the ASCII code of the last processed switch is returned in the accumulator – or a \$00 if there were no switches processed.

The MEM command parses its command line this way.

5.9 Keyword (U_GETPAR/U_TOKEN)

Parsing predefined keywords is done with two procedures: U_GETPAR and U_TOKEN.

U_GETPAR, as it has already been explained above, copies the next (i.e. the one being indicated by BUFOFF) parameter from LBUF to COMFNAM, stores its length in FILLEN, and COMFNAM's address in FILE_P.

U_TOKEN fetches a text parameter stored in COMFNAM, and tries to find it in a table of predefined keywords. The address of that table should be passed in registers AX (LSB, MSB, respectively). The keywords must be located in the table one directly after another, the last character of each keyword being in inverse video (i.e. with bit 7 set). The end of the table is marked by a \$00.

When U_TOKEN does not find the keyword in the table, it returns with C=0. Otherwise, when the keyword was found, the C is set to 1, and the Accumulator contains the number (or, in other words, token) of the keyword, counting from \$00.

This is the method used by the batch instruction IF and by the Command Processor.

5.10 Device identifier (U_GETPAR/U_GEFINA)

When the parameter consists of a device identifier alone, for example a disk identifier, a pair of the procedures should be used to parse it: U_GETPAR and U_GEFINA. U_GETPAR, as it has been explained above, fetches a parameter from LBUF and copies it into COMFNAM, and stores the COMFNAM's address in FILE_P.

U_GEFINA fetches the parameter pointed to by FILE_P, analyzes it as a device identifier and sets up the *device* (\$0761) register accordingly. Its less significant nibble contains the unit number (e.g. a number of a disk drive), and the most significant nibble hold the device identifier, for example:

\$0x – device DSK: (disk)
 \$1x – device CLK: (clock)
 \$2x – device CAR: (cartridge)
 \$3x – device CON: (console)
 \$4x – device PRN: (printer)

CAUTION: it should not be assumed that the codes above always correspond to the specified names in the manner above, because the assignment may change when the user de-installs one device driver and installs another one. The method of translating the device value to a text form is described in chapter 18.

When no identifier (or a „:“ alone) was specified, the current device is assumed (current disk, most of the time), its code get fetched from a variable pointed to by the symbol CURDEV. *When the device identifier is incorrect and cannot be decoded, the control goes to U_FAIL with error code 130 (Non-existent device).*

In this way U_GETPAR and U_GEFINA are used in CHKDSK.

5.11 File specification (U_GETPAR/U_GEFINA)

The pair mentioned above: U_GETPAR/U_GEFINA may be as well used to decode a file name. The device identifier, as above, is translated to a value in *device* (\$0761), and the rest of the specification gets divided into pathname copied to *path* (\$07A0, 64 bytes), and the actual file name copied to *name* (\$0762, 11 bytes). The file name is stored there in DOS internal format, i.e. as NNNNNNNNXXX. If either part of the name (i.e. the name's body or the extension) consists of less than 8 or 3 characters, respectively, it is padded with spaces. If there are wild cards there, they get expanded into series of question marks and so on.

When the device identifier is incorrect and cannot be decoded, the control goes to U_FAIL with error code 130 (Non-existent device).

This is the file specification format which already can be passed on to the SpartaDOS X kernel drivers, especially to the DSK: driver contained in SPARTA.SYS. The library functions calling that driver most of the time call U_GEFINA automatically, so the user's program usually only needs to call U_GETPAR beforehand.

5.12 Directory specification (U_GETPAR/U_GEPATH)

When the expected parameter is a directory specification, the procedure is similar to the one described above, with the only exception that U_GEPATH should be called instead of

U_GEFINA. U_GEPATH translates the device specification into *device* (\$0761), and the complete path is copied into *path* (\$07A0, 64 bytes). Nothing is copied into *name* (\$0762, 11 bytes).

When the device identifier is incorrect and cannot be decoded, the control goes to U_FAIL with error code 130 (Non-existent device).

Such a form of specification is useful when kernel functions 16 (kd_chdir) and 17 (kd_cwd) are called. The corresponding library functions (CHDIR and GETCWD) call U_GEPATH automatically, so the user's program only needs to call U_GETPAR before.

5.13 File specification and attributes (U_GETATR)

Some commands, such as COPY or DUMP, accept a file specification as a parameter, plus optionally they allow the user to specify preferred attributes. For example, *DUMP +H FOO.BAR* will display a hidden file (with +H attribute) named FOO.BAR. Without that, a hidden file is ignored.

The analysis of such a parameter is performed by U_GETATR. It does that standalone, i.e. there is no need to call U_GETPAR beforehand. The default attribute (in case the user specifies none) should be passed on in the accumulator. The bit mask should be put together according to the following table:

+\$01: protected only (+P)	+\$10: unprotected only (-P)
+\$02: hidden only (+H)	+\$20: unhidden only (-H)
+\$04: archived only (+A)	+\$40: not archived only (-A)
+\$08: directories only (+S)	+\$80: not directories only (-S)

The most often used default masks are: \$20 (*not hidden*) and \$A0 (*not hidden* and *not directory*). A mask value of \$00 selects all files.

U_GETATR returns a zero (Z=1), when there is too few parameters in LBUF to fetch from – for example, the user has given an attribute, but there is no file name. The decoded attribute is stored as a bit mask in *scan* (\$0779), which constitutes one of the search criteria for file searching in FFIRST and FNEXT (and by that also in all other functions which use these two, especially FOPEN). Despite this detail, U_GETATR works the same way as U_GETPAR, that is, fetches a parameter from LBUF, stores it in COMFNAM, the parameter's length saves in FILLEN and the COMFNAM's address in FILE_P.

5.14 File specification with a default mask (U_FSPEC)

Some commands, like COPY, accept a path as a parameter, plus file specification or a file mask selecting a group of files. The mask may be omitted, if it has to be *'*.*' – COPY FOO>* should copy all files (*'*.*'*) it finds in the directory FOO. The program which performs the command has to know, if the mask was given, and add it, when it finds out, that it was omitted.

This uncomplicated, but troublesome task – one must check a number of conditions – is performed by U_FSPEC. When called immediately after U_GETPAR or U_GETATR, it checks if the parameter in COMFNAM ends with a path separator, and thus meets the conditions to add a mask *'*.*'* to it. When it does, adds the mask updating FILLEN at the occasion.

5.15 Various parameter types combined

The most frequent situation is when a program first parses a file name, then some options marked with a slash. Parsing such a command line is no trouble, the program should first do what is written above on fetching a file spec, then call U_SLASH to parse the options.

A more difficult case is a command like ECHO, which has two versions: ECHO ON/OFF or ECHO TEXT. The former enables or disables the echo of the commands being performed by the Command Processor. The latter one simply displays the given text on the screen. The difficulty is that to parse the ON/OFF switch the U_GONOFF function should be called, and this one, when the parameter is neither ON nor OFF (but TEXT), immediately aborts the program by jumping to U_FAIL.

The correct solution is to set a trap to catch the error generated by U_GONOFF. It is explained in the chapter 4 „Error handling“.

5.16 Getting loader flags (U_GETFLG)

U_GETFLG is a special purpose function originally intended to set the program loading mode FLAG (see 11.1) via a command line parameter. The expected command line parameter is this:

letter|number

where the following letter symbols for the parameter are recognized:

- S – SpartaDOS X default loading mode (INIT vector is called, RUN vector or the first segment starts the program)
- A – AtariDOS mode (INIT called, RUN starts the program)
- I – INIT-only mode (INIT called, the program is not started)
- N – Load-only mode (no INIT, no start)

In addition to the symbols, U_GETFLG can also accept the corresponding numerical values described in 11.1 - or in fact any numerical value, which will then be stored in the FLAG register. For example:

FOO A BAR.EXE

the program FOO should first call U_GETFLG to determine the desired loading mode, then U_GETPAR to fetch the subsequent file name. In this case, the FLAG register will get set to \$40, as explained here and in 11.1. Alternatively this:

FOO \$40 BAR.EXE

provides the same result as the example above. The value is expected to constitute an input parameter for the subsequent U_LOAD call, but, of course, it can be also used for other purposes.

Example of common usage:

```

loadx  lda #0000      ;default FLAG if no parameter given
        jsr U_GETFLG   ;get mode
        jsr U_GETPAR   ;get file name
        beq exit
        jsr PRINTF
        .byte 'Loading, FLAG=%2x', $9B, 0
        .word FLAG
        jsr U_LOAD
exit    rts

```

This call sequence is also used by the commands LOAD and CAR in the Command Processor.

5.17 Other functions (U_PARAM, _CRUNCH)

U_PARAM is one of the functions processing the command line contents. It fetches a parameter from the LBUF and copies it into an intermediate buffer named *copybuf* which lives at COMTAB+\$BF. U_GETPAR and U_SLASH use that buffer to process the command line.

_CRUNCH is a „traditional“ routine to parse command lines, it is used by programs written for SpartaDOS versions older than 4.0. For certain technical reasons it is pointed to by a symbol labelled _CRUNCH, but the main entry point lives at COMTAB+3 and is sometimes labelled ZCRNAME in the relevant literature. That address (as a reminder: it is a result of adding 3 to the DOSVEC vector value) is a common one for all SpartaDOS and DOS XL versions.

The information on how to use that is to be found in the chapter 6 of the „SpartaDOS X User’s Manual“. It can be added here, that the presence of that procedure may be detected by checking, if the byte located at COMTAB+3 contains a value of \$4C (or in other words, a JMP instruction). If so, the ZCRNAME, or a compatible routine, is available.

6 Processing File and Path names

6.1 8+3 to internal format conversion (U_GEFINA)

A conversion of a file name from 8+3 to the internal format consists in space-padding both of the parts (i.e. the filename body and the extension), when necessary, expanding wild cards '*' into question marks and gluing both parts together to form a 11-character string looking like this: NNNNNNNNXXX. That string is stored into a buffer called *name* (\$0762-\$076C).

The conversion is made with a call to U_GEFINA with an 8+3 file name pointed to by FILE_P. Note that this also converts device specification into *device* (\$0761), and the path – if there is any – gets copied into *path* (\$07A0).

When the device specification is invalid, the U_GEFINA function will throw an error via U_FAIL (see chapter 4).

6.2 Helper function PRO_NAME

A helper function PRO_NAME makes the above conversion on a 8+3 name pointed to by *bufadr* (\$15-\$16) plus offset Y+1, and stores it in the internal format at *name* (\$0762-\$076C). When *name* contains characters upon return, the function returns with a non-zero result (Z=0), or a zero-result (Z=1) otherwise. The accumulator contains a delimiter character, if there is one prior to the name being decoded. The Y register contains the updated offset.

PRO_NAME is a helper function used by U_GEFINA and U_GEPATH. Because of that, user programs normally do not need to call it directly.

6.3 Internal format to 8+3 conversion (U_EXPAND)

The reverse translation, i. e. a name stored in *name* (\$0762-\$076C) to 8+3, is made by U_EXPAND. The result will be stored at an address pointed to by the AX registers (LSB, MSB, respectively) at an offset defined by the Y register. The resulting text string is EOL-terminated (\$9B). Its position in the buffer is returned in the Y register.

6.4 Internal format to path string conversion (U_PATH)

An operation complementary to U_EXPAND is performed by U_PATH: this function gets *device* (\$0761) and *path* (\$07A0) as input parameters, then converts them back into the human-readable string, which will be stored at the address pointed to by the AX registers (LSB, MSB respectively), plus the offset in Y.

The string generated is EOL-terminated, upon return the Y register contains an offset of that EOL character; thus a subsequent call to U_EXPAND with the same base address in AX will then append the file name at the end, resulting in a complete pathname.

If the *device* value is invalid (e.g. \$FF), it will get skipped. But if it is valid and points to an unregistered device, this function will jump to U_FAIL.

The U_PATH function is present as of SpartaDOS X 4.50.

7 Environment Variables

7.1 What is an environment variable

SpartaDOS X is the only DOS for Atari 8-bit, which supports environment variables, known otherwise from bigger computers. An environment variable is an ASCII string with an unique name assigned to it. The variables can hold some global settings for the system, the Command Processor or application programs. An example of the first type can be \$PATH and \$DAYTIME, the other \$COPY, the third one \$MANPATH.

The environment variables are held in the extended RAM, in a 256-byte buffer. The library offers three functions facilitating the access to the data contained there.

As of SpartaDOS X 4.47 a pointer to the environment buffer is held in a three-byte array pointed to by the symbol ENV_P: the first two bytes contain an address of the buffer, the third one - an index of the memory where the buffer is held. If these bytes are all zeroed, the environment buffer is not directly accessible (use functions specified below to access the variables).

7.2 Getting a variable by its name (GETENV)

To read an environment variable by its name one should use a function labelled GETENV. The address of the name of a variable to be found should be passed in the AX registers, LSB and MSB, respectively. The name (an ASCII string) should be EOL-terminated. If the function returns a negative value (A=\$FF, N=1), that means that there is no such variable in the environment. Otherwise the result is positive (A=0, N=0), and the variable's value is to be found at the FP package output buffer *lbuff* (\$0580) as an EOL-terminated ASCII string.

7.3 Getting a variable by its number (NUMENV)

Alternatively a variable can be fetched not by its name, but by the number of the place it occupies in the buffer. The function that does it is labelled NUMENV. The number of the variable must be passed in the accumulator. When the result is positive (A=0, N=0), the variable's value is to be found at the same place and in the same format as in the case of GETENV (see above). Otherwise, when the numbers specified does not correspond to any variable, the result is negative (A=\$FF, N=1).

This function is most frequently used to get all the variables consecutively to e.g. display them on the screen (just as it is done by the Command Processor's SET command), searching the entire buffer through etc.

7.4 Putting and deleting variables (PUTENV)

PUTENV stores a variable into the environment buffer. The address of the new variable's value must be passed in the AX registers (LSB and MSB, respectively). At that address there should be an EOL-terminated ASCII string in the form: NAME=TEXT.

This will cause a variable named NAME to be created and assigned the value of TEXT. If such a variable already exists in the buffer, it is deleted first.

Passing a variable name alone, that is, an EOL-terminated ASCII string without an equation sign, will cause the specified variable to be deleted.

As above, a positive result (A=0, N=0) means success, a negative one (A=\$FF, N=1) means an error.

Remark: when using NUMENV and PUTENV combined in one loop to find and delete certain variables, it should be kept in mind, that deleting a variable causes all other variables, which are located past the deleted one, to get their numbers decreased by 1. Therefore, in such a loop, after a call to PUTENV which deletes a variable, one should NOT increment the counter for the next NUMENV call.

8 Accessing Files

8.1 Opening a file (FOPEN)

Opening a file is realized by FOPEN. The required parameters are passed to it as follows:

- 1) file specification should be pointed to by FILE_P
- 2) opening mode should be stored in *mode* (\$0778)
- 3) mask of file search attributes should be stored in *scan* (\$0779)
- 4) when opening a file for writes (modes \$08, \$09 and \$0C) the mask of assigned attributes should be stored in *attr* (\$077A)

The file specification pointed to by FILE_P should be stored in a text form (an EOL-terminated ASCII string). The device should be specified according to SpartaDOS X convenience, and not the Atari OS convenience, that is, for example, A:>KATALOG>PLIK.TXT rather than D1:>KATALOG>PLIK.TXT.

8.1.1 Access mode

mode has the same function as the Atari OS *icax1* of the IOCB channel, when a file is being opened (when the OPEN function of the CIO device D: is being called, the value of *mode* is fetched from nothing else than *icax1*). The value of this variable consists of two nibbles. The least significant one defines the data access mode:

\$x4 – read
 \$x8 – write
 \$x9 – append
 \$xC – read/write

CAUTION: versions up to SpartaDOS X 4.41 (inclusive) contain a bug, which causes that when reading a file opened read/write, an EOF condition never occurs. It has been fixed as of SpartaDOS X 4.42.

The most significant nibble is a bit mask, where setting single bits has the meaning as follows:

\$8x – reserved
 \$4x – reserved
 \$2x – open for read with path searching (\$PATH)
 \$1x – direct directory access

Setting bit 4 causes the specified directory to be available as „raw” data, in the similar manner as it is in the case of a regular file. A directory becomes available for reads or for writes – in that latter case every bug in the program may of course result in destroying the directory.

When opening a directory the system reads its „zero entry” (or the header) and stores it at *dentry* (\$0789-\$079F). Because of that the initial read/write position in the directory points to the beginning of the first „normal” directory entry (FTELL will return a value of 23). To read the header out a FSEEK 0 should be done first.

*CAUTION: due to limitations of the Atari DOS 2.0 file system, when reading a directory from a DOS 2.0, 2.5, MyDOS etc. disks, **the EOF status occurs after the entire directory has been read**, i.e. after 1495 bytes (the header + 64 entries, 23 bytes each), regardless of whether there are any entries in the directory or not. That problem does not occur for a SpartaDOS disks, but, as the program cannot certainly know, what a file system it is dealing with, it should not count on the EOF status occurring in the proper time. Therefore, a directory opened for direct access should be read in portions, 23 bytes each: the signal of the end of the directory is then either an EOF (status 136), or a situation, when the first byte of a portion, after clearing out the bits 0-2 (AND #\$F8), turns out to be \$00.*

When the file access mode is \$04 (read) and bit 5 is set (so the *mode* has a value of \$24), the DOS will search the specified file in all the directories indicated by the \$PATH variable.

8.1.2 File search attributes

The mask of file search attributes *scan* (\$0779) is used when opening a file for reads. The specified file will be found in the directory and opened only when the condition encoded in *scan* is met:

+\$01: protected only (+P)	+\$10: unprotected only (-P)
+\$02: hidden only (+H)	+\$20: unhidden only (-H)
+\$04: archived only (+A)	+\$40: not archived only (-A)
+\$08: directories only (+S)	+\$80: not directories only (-S)

Normally when opening regular files a mask value of \$A0 is used (*unhidden* and *not-a-directory*). Setting bit for example \$01 and \$10 (protected and unprotected) will cause an attempt to select files which met both criteria, but as no file may be protected and unprotected at the same time, such a condition, being self-contradictory, will never be met. Consequently, no file will be selected. The mask which selects all files has a value of \$00.

8.1.3 Attributes assigned

The bits 0-3 in the mask of assigned attributes *attr* (\$077A) have meanings corresponding to the bits in the mask of required attributes; the rest have no meaning. The mask has a meaning only when creating new files, usually \$00 is used here.

When an error occurs, the control is transferred to U_FAIL (see the chapter 4 „Error handling“). When the opening was successful, the file's handle is stored at *file_id* (\$0760).

8.2 Closing files (FCLOSE/FCLOSEAL)

The library offers two functions here: FCLOSE closes a specified file, viz. the one which corresponds to the handle being stored in *file_id* (\$0760).

When calling FCLOSE without a previous FOPEN in SpartaDOS X 4.20 the system sometimes crashes, and in SpartaDOS X 4.22 an error 133 message may appear (File not open).

FCLOSEAL closes all files, which are currently open. To be more specific, whether a file gets closed or not, depends on a „system level“, indicated by a variable labelled SYSLEVEL – the file is closed, when the current SYSLEVEL value is lesser than or equal to the SYSLEVEL value at the time the file was being opened. As the SYSLEVEL value is incremented by each call to U_LOAD, and decremented by each call to U_UNLOAD, therefore FCLOSEAL is able to

close all the files which were opened by the calling program, but it will not close any file opened by the parent program, i.e. the one which called the U_LOAD to load the program which is calling FCLOSEAL (it is the DOS library most of the time).

This mechanism protects the files of one program from being closed by another program. As long as a program is running, a file opened by that program is its private file. At the other hand a parent program can close files opened by its child, when the child terminates.

There is a possibility to protect a file from being closed by FCLOSEAL. To do that, its handle should be written to `file_id` (\$0760), then call `FCLEVEL` should be done with `$FF` in the accumulator. Similar procedure with the current `SYSLEVEL` value in the accumulator will lift the protection.

8.3 Reading and writing single bytes (FGETC/FPUTC)

Reading and writing single bytes from and to a file whose handle is in `file_id` (\$0760), is accomplished through FGETC and FPUTC.

FGETC returns a data byte in the accumulator. When an EOF has been encountered, the accumulator will contain the EOL character (\$9B), the X register will contain \$FF and the flag register will signalize a negative value (N=1). Any other error condition will transfer the control to U_FAIL. The Y register value is preserved across a call to FGETC.

FPUTC sends a byte from the accumulator to a file. Calling the function does not alter A, X and Y registers. Any error condition transfers the control to U_FAIL.

8.3.1 Micro-buffers

The library contains code that can buffer 1-byte data transfers, speeding them up significantly. By default there are 4 buffers, 16 bytes each.

The I/O operations made by FGETC and FPUTC for DSK: and CAR: devices are micro-buffered by default. Thanks to that they are much faster than transferring single bytes with FREAD/FWRITE calls.

The micro-buffers are flushed on FCLOSE, but also whenever a call is being made to FREAD, FWRITE, FNEXT, FSEEK, FTELL or FILELENG. That last one can be therefore used to flush them on demand without incurring side effects.

8.4 Reading and writing text records (FGETS/FPUTS)

Reading and writing text records is accomplished with FGETS and FPUTS. The file's handle should be in `file_id` (\$0760). The parameters are passed in the registers: a buffer address in AX (LSB, MSB respectively), buffer length in Y. A record is an EOL-terminated ASCII string not longer than 255 characters.

When FGETS succeeds, it returns with a positive result (N=0), a zero in the accumulator and the number of bytes read in Y. An EOF is signalized by a negative result (N=1) and a zero in the accumulator. When there is an overflow, the result is negative (N=1) and the accumulator contains an \$FF. This condition corresponds to error 137 in the XL OS (Truncated record). Any other error causes a call to U_FAIL.

When writing with FPUTS, if the string is shorter than the length passed in Y, it should be EOL-terminated. Any error condition causes the control to be transferred to U_FAIL.

FGETS and FPUTS use the FGETC/FPUTC for I/O. Thanks to that these operations are by default micro-buffered for DSK: and CAR: devices.

8.5 Reading and writing binary blocks (FREAD/FWRITE)

Bigger binary blocks are read through FREAD and FWRITE. The parameters, besides the usual handle in *file_id* (\$0760), should be passed as follows:

- low word of *addpos* (\$0782-\$0783): 16-bit address of buffer
- *bytbuf* (\$0785-\$0786): buffer size
- *memreix* (\$0787): index of destination (or source) memory

In versions prior to SpartaDOS X 4.47 the 'memreix variable can be only assigned one of two values: \$00 or \$02, because there are only two valid memory indices.

The buffer size (unsigned) must be greater than a 0. The FREAD always tries to read the amount of bytes indicated by *bytbuf*. When succeeded, the functions return with a positive result (N=0, X=\$00). When FREAD encounters an EOF, the result is negative (N=1, X=\$FF). Any other error causes a jump to U_FAIL.

Upon return from FREAD, *bytbuf* will contain the number of bytes actually read. Neither function changes *addpos* during execution, so this word will also contain the buffer address.

FREAD and FWRITE are not micro-buffered, so they are not suitable for transferring small data portions. For only a dozen of bytes it is much better to use FGETC and FPUTC.

8.6 Obtaining file's length (FILELENG)

FILELENG reads the length of the file which is currently opened and stores the value in *addpos* (\$0782/3/4). The file's handle should be in *file_id* (\$0760) before the call.

As said above, this call can be used to flush the library's micro-buffers on demand without incurring any remarkable side effects.

8.7 Obtaining and changing current file's position (FTELL/FSEEK)

The current read/write position in an open file may be read through FTELL. It stores in *addpos* (\$0782/3/4) the number of the byte (counting from the beginning of the file as position 0), which would be read or written in a next I/O operation. The opposite, i.e. changing that position, is accomplished with FSEEK. The new position must be stored in *addpos* (\$0782/3/4) before the call. In both cases the file's handle must be stored in *file_id* (\$0760) beforehand.

In a (quite rare) case, when a newly created file is first filled with a default value (e.g. zeros), and directly after that – without closing and reopening it – the creating program wants to write some specific data to it, if this operation is to succeed, right before writing the specific data the program should do a FSEEK to position 0 of the file.

When an attempt is made to set the position after the end of the file, and the file is opened for reads, the control goes to U_FAIL with an error 166 (Range error). A similar attempt on a file opened for writes does not cause any error, but any subsequent writes and closing the file causes the file to become a sparse one (with a „hole“, to which no data sectors are assigned).

8.8 Writing formatted text to a file (FPRINTF)

Writing a formatted text to a file is accomplished with FPRINTF. Its operation is identical to the PRINTF explained in the next chapter (FPRINTF and PRINTF are just a different entries to one function), with the only exception, that the writes go to the file, whose handle is in *file_id* (\$0760), and not to the console display.

Similarly to the text record functions, FPRINTF uses FPUTC to do writes, therefore the disk writes are micro-buffered.

9 Console I/O

9.1 Writing single characters to the console (PUTC)

Writing single characters to the console (i. e. to the CON: device – perhaps its a time to remark that the output to CON: can be redirected by the user to any other file) is accomplished with PUTC. The character to be sent should be passed on in the accumulator. The call does not change the A, X, Y registers, nor the value of *file_id* (\$0760) and *device* (\$0761).

9.2 Writing a text record to the console (PUTS)

Writing a text record to the CON: device is done with PUTS. Before the call, the address of the string to be displayed should be loaded to AX registers (LSB, MSB, respectively), and Y should contain the length of the record. If the string is shorter than that, it must be EOL-terminated.

PUTS preserves the Y register and the values of *file_id* (\$0760) and *device* (\$0761).

9.3 Printing formatted text (PRINTF)

The PRINTF function sends to the console texts or other data streams converted to a text form and formatted according to the given formatting pattern. The peculiarity of this function is that all the parameters are passed to it by putting them directly past the call JSR PRINTF. The scheme is as follows:

```
jsr PRINTF
.byte formatting pattern, 0
possible data
...
```

The „formatting pattern” is a text string (before SpartaDOS X 4.48 up to 253 characters in size), NUL-terminated. In the simplest case it is the text to be displayed, for example:

```
jsr PRINTF
.byte "Hello world!", 0
```

Executing this will print the given text on the screen, stopping the cursor at its end. To make the cursor go to the next line, an EOL character (\$9B) should be added to the end of the text line (before the zero).

But the most of the power consists in the fact, that in the given text string (which as you remember, is only a *pattern* formatting the output text) one can put formatting sub-strings. This is the list:

- %% – print single %-character
- %c – print *character* from specified address
- %s – print *string* from specified address
- %p – ditto, just address of string *pointer* is given instead
- %x – print value from specified address as 24-bit *hex* number
- %b – print value from specified address as *byte* (8-bit) decimal number
- %d – print value from specified address as 16-bit *decimal* number
- %e – print value from specified address as *extended* (24-bit) decimal number

- %l - ditto, *long* (32-bit) decimal number (as of SpartaDOS X 4.40)
- %t - print entry from keyword *table* located at specified address (as of SpartaDOS X 4.47)

Each one such formatting sub-string must correspond to a specific pointer. The pointer must be put after the formatting pattern (on the scheme above the pointers are specified as *possible data*).

The usage is very simple. Assume that *value* is a label corresponding to an address of a 16-bit word (LSB, MSB) in the memory. What we want is to convert the value to characters strings representing it in the hexadecimal and decimal system:

```
jsr PRINTF
.byte "VALUE = %d", $9B, 0
.word value
```

When there is a value of \$98AB (or 39083 in decimal) at the memory location labeled *value*, this text will appear on the screen:

```
VALUE = 39083
```

As it was written above, %d is a command to format a „16-bit“ decimal number. Such a number can have up to five digits (0-65535). In the example above all the digits are displayed, this is the default.

An additional feature is that the formatting sequence may be preceded by a numeric value. That value defines the minimum field size, or in other words, the number of characters to be printed. Thus, assuming *value* to be the same as above, this:

```
jsr PRINTF
.byte "VALUE = %2d", $9B, 0
.word value
```

will display the last two digits only:

```
VALUE = 83
```

and this:

```
jsr PRINTF
.byte "VALUE = %10d", $9B, 0
.word value
```

will cause this to be displayed:

```
VALUE =      39083
```

(five spaces before the five-digit number for a total of 10 digits).⁵⁾

⁵⁾ In the current implementation 10 digits is the maximum value for a decimal field, specifying more will cause undefined results.

Preceding the field size with a 0 in that last example:

```
jsr PRINTF
.byte "VALUE = %010d", $9B, 0
.word value
```

will generate zeros instead of spaces:

```
VALUE = 0000039083
```

Finally, the field size does not have to be specified as a constant; it is possible to make the library fetch that value from a variable, this way:

```
jsr PRINTF
.byte "VALUE = %0*d", $9B, 0
.word numlen, value
```

The *numlen* label points to a location in the memory, where the PRINTF will fetch the required length from. Assuming that there was 10 stored in that location, the resulting display will be the same as above:

```
VALUE = 0000039083
```

The field size must be in range from 0 to 255. In fact it may be up to 65535, but the MSB will get ignored – in the case, however, of the variable field size (%*d) the variable should be of a word size for upward compatibility.

Formatting works similarly for decimal numbers: by specifying the field size in %b, %d, %e and %l it is possible to adjust the number of characters displayed.

In case of %c always just one character is printed, all formatting attributes are ignored.

%s will display an ASCII string found in the memory at the specified address. Such a string should be EOL- or NUL- terminated (\$9B or \$00), both characters work the same way in this case, i.e. EOL terminates the text but does not cause the cursor to go into a newline. After the % the string size may be given: if the string is longer, it will get cut down; if it is shorter, it will get space-padded from the right.

%p works the same way the %s does, with the exception that instead of an address of the string an address of a pointer containing an address to that string should be specified.

Space-padding strings may be used to generate a required number of spaces. To do that you should specify the required number of spaces in the formatting string, and an address of a NUL (or \$00) as the address of the string, for example:

```
jsr PRINTF
.byte "%25s", 0
.word *-1
```

The trick works according to the general logic of this function, i.e. the address specified points to an empty string, which, due to its zero-length, is shorter than the specified size. Therefore it will get space-padded using an appropriate (i.e. specified) number of spaces.

%t prints an element selected from the specified table (see chapter 5.9 on the U_TOKEN function). An index to that table (starting from 0) may be given as a constant parameter, for example:

```
jsr PRINTF
.byte "Keyword %0t", $9B, 0
.word tokentable
```

This code will print the first keyword from the specified list. If the parameter is too big, nothing will be printed. The parameter does not have to be a constant; a variable can be given instead using the „*” operator as described above, for example:

```
jsr PRINTF
.byte "Keyword %*t", $9B, 0
.word tokenidx, tokentable
```

The %t may be also used to print out the numeric ASCII strings produced by the Atari Floating Point package’s function FASC (\$D8E6).

As of SpartaDOS X 4.42 PRINTF allows to use control sequences in text strings. These sequences are:

```
\a - bell
\b - backspace
\e - escape
\f - clear screen
\i - toggle inverse video (as of SpartaDOS X 4.48)
\n - new line
\r - the same as \n
\t - tabulator
\z - ASCII 0 character (as of SpartaDOS X 4.48)
\\ - a single „\” character
```

PRINTF does not change the A, X, Y registers, nor the value of *file_id* (\$0760) and *device* (\$0761).

9.4 Reading single characters from the console (GETC)

Getting a byte from the console is accomplished with GETC. The data is returned in the accumulator. When the read succeeded, the result is positive (N=0) and the X register contains \$00. When an end of file was encountered, the function returns with a negative value (N=1) and a \$FF in the X register. Any other status will automatically invoke U_FAIL (see the chapter 4 „Error handling”).

When a byte read from the keyboard is required, the U_GETKEY function should be used instead.

9.5 Reading a text record from the console (GETS)

Reading a text record from the console is done with a function called GETS. A text record is an EOL-terminated ASCII string. As for PUTS, the parameters are passed in the CPU registers: AX holds the buffer address (LSB and MSB, respectively), Y the length in bytes. A record cannot be longer than 255 bytes.

On success the function returns with a positive status (N=0), a \$00 in the accumulator and the number of bytes read in the Y register. When there is too much data for the declared buffer, the result is negative (N=1), and the accumulator contains \$FF. This situation corresponds to Atari OS' error 137 (Truncated record). Any other status will automatically invoke U_FAIL (see the chapter 4 „Error handling“).

Console functions PUTC, PUTS, PRINTF, GETC and GETS are just wrappers to functions FPUTC, FPUTS, FPRINTF, FGETC and FGETS, which were described in the previous chapter. Using console functions instead of the latter ones makes handling I/O redirection painless, because all the related tasks will be done automatically inside the I/O library.

9.6 I/O redirection (DIVIO/XDIVIO)

Console I/O may be redirected from/to any other file using the DIVIO. This is the function invoked by the Command Processor, when the user uses I/O redirection, e.g. DIR >>file.

DIVIO expects the following parameters:

- a) FILE_P vector to point to the specification of a file where to (or where from) the data will be redirected;
- b) Y=\$00, when standard output is redirected (PUTC and such functions), or
- c) Y=\$01, when standard input is to be redirected (GETC and friends).

When Y=\$00, the destination stream is being opened in write mode, and, if such a file already exists on the disk, it is truncated. As of SpartaDOS X 4.50 DIVIO is able to redirect the standard output in append mode: to accomplish that Y=\$02 should be passed to the function. *Passing Y=\$02 on earlier SpartaDOS X versions will cause undefined results.*

Any errors occurring when the redirection is being created, invoke U_FAIL automatically. On success the handle of the file, where data is redirected to (or from) is found at *file_id* (\$0760).

Ending the redirection is accomplished by calling XDIVIO. As a parameter a \$00 or \$01 should be given in the Y register as said above. Calling XDIVIO will close the relevant file and cancel the redirection.

When console input was redirected and an EOF condition occurs, the library will invoke XDIVIO automatically. It also occurs when all the files get closed with FCLOSEAL or when an error occurs.

9.7 Vectored output (PUT_V/VPRINTF)

Another method of controlling consoles output (only) is to use the PUT_V vector. Normally it does not have any reasonable value, the program wishing to use it must set it up so that it points to a routine handling writes. It might for example be a routine which writes data to screen directly, bypassing CON: or E:. Such a routine should end with RTS and do not modify CPU registers.

When the file handle in *file_id* (\$0760) has a „magic“ value of 100 (\$64), calling FPUTC causes an indirect jump (JMP) via PUT_V with a data byte in the accumulator. Because FPUTC is implicitly called by FPUTS and FPRINTF, the output from these functions will get redirected in identical manner.

Instead of FPRINTF, it is more convenient to call VPRINTF – it is yet another (a third one) wrapper over that function, and its operation is identical as in PRINTF, except that the active I/O channel will get automatically changed to 100 before printing out the text, and also automatically switched back after it. A programmer does not need to trouble himself with the contents of *file_id* (\$0760) in this case.

9.8 Reading a byte from keyboard (U_GETKEY)

U_GETKEY waits for a key to be depressed, then returns its code in the accumulator and a positive status (N=0). When the status is negative (N=1), the accumulator contains an error code (128 or 136, i.e. the user pressed the Break key or Control/3, respectively).

Remark: U_GETKEY is implemented as of SpartaDOS X 4.40.

As of SpartaDOS X 4.43 U_GETKEY is expanded with a possibility to read the HELP key and most other keys pressed together with Shift and Control held down simultaneously.

On a key press, when an ASCII code exists for the key depressed, it is returned in the accumulator, and the result is set to non-zero (Z=0). The alpha-numeric keys, pressed together with Shift and Control, return ASCII codes as if the Control key was not held down (for example, Shift/Control/1 will return the „!“ character). If a key does not have a corresponding ASCII code, the result is zero (Z=1), and the accumulator will contain the Escape code (ASCII 27, or \$1B).

The X register contains additional information:

- bit 7 = 1 – Control key held down
- bit 6 = 1 – Shift key held down
- bit 5 – reserved, always 0
- bit 4 = 1 – HELP key held down

The remaining bits are reserved and set to 0.

Remark: not all Shift/Control+letter combos cause a reaction. The following keys, when joined with both Shift and Control keys held down together, are ignored by the Pokey chip, therefore cannot be read:

*HELP, J, K, L, ;, +, *, Z, X, C, V, B*

The Shift/Control+A combo is handled internally, its function is to clear the keyboard's input buffer.

10 Directories

10.1 Searching for files (FFIRST/FNEXT)

Two functions are provided to search a specified directory for files: FFIRST and FNEXT. The first one accepts a complete directory specification plus a file mask appended to it as one of the arguments – this specification should be pointed to by the FILE_P vector. The other argument is a mask of file search attributes in *scan* (\$0779), as follows:

+\$01: protected only (+P)	+\$10: unprotected only (-P)
+\$02: hidden only (+H)	+\$20: unhidden only (-H)
+\$04: archived only (+A)	+\$40: not archived only (-A)
+\$08: directories only (+S)	+\$80: not directories only (-S)

A matching directory entry is loaded to *dentry* (\$0789-\$079F) in the „crude” form of a SpartaDOS directory entry (i. e. in the same form as it exists on a SpartaDOS disk, see „SpartaDOS X User’s Manual”, chapter 7).

FNEXT does not require any additional parameters, it simply finds a next entry matching the criteria given by a previous call to FFIRST.

The end of a directory in both functions is signaled by a return with a negative result (N=1). Any other error will automatically invoke U_FAIL.

*CAUTION: FFIRST, in fact, opens the specified directory for reads, and puts the file’s handle to file_id (\$0760). Therefore, when the search is done, the program **must** call FCLOSE for that handle in order to close the directory’s file.*

10.2 Reading formatted directory (FDOPEN/FDGETC/FDCLOSE)

The library contains three functions, which provide programs with directory contents in a formatted, or human readable, form (for example, in a form which is displayed after the user types in the DIR command at the command prompt).

These are: FDOPEN, FDGETC and FDCLOSE. FDOPEN opens the directory data stream, FDGETC fetches a byte from it and returns it in the accumulator register (setting a positive result in the 6502 flag register), and FDCLOSE closes the directory which was opened using FDOPEN. Errors cause U_FAIL to be invoked, with the exception of an EOF status – on this one, the function returns to the caller with a negative result (N=1).

These three functions are in fact wrappers for corresponding misc_ functions (see chapter 25).

As of SpartaDOS X 4.46 the FDGETC function zeroes the status byte of the last read directory entry (in *dentry* \$0789), when there will be no more entries and the information on the free space (xxxxx FREE SECTORS) has already been written to the output buffer. This allows a program to „catch” the end of the directory list at the point when all the actual entries of files and directories have been already read out.

The parameters for FDOPEN are:

- a) directory specification pointed to by FILE_P
- b) the required formatting mode in *mode* (\$0778)
- c) the required file search attributes in *scan* (\$0779)

DOS versions prior to SpartaDOS X 4.41 knew only two formatting modes: „long” SpartaDOS format (*mode*=\$80) and „short” AtariDOS format (*mode*=\$00). As of SpartaDOS X 4.41 *mode* is interpreted as a mask of bits, which independently select methods of formatting for the components of the directory listing. The meaning of bits:

- \$80 - long directory format
- \$40 - display attributes
- \$20 - insert space between file name and its extension
- \$10 - print dot after the file name (when bit 5=1)
- \$08 - when long directory format (bit 7=1), print time without seconds; when short format, print directory name extensions (instead of [DIR]), and a directory will get marked with a ‘.’ character printed before the name.
- \$04 - 24-hour time format
- \$02 - two spaces before name, ‘*’ to mark a protected file
- \$01 - in short format (bit 7=0) display file’s size in sectors (or none otherwise). In long format, display file’s full size in bytes (up to 10 digits).

For backward compatibility, user supplied \$00 is translated to \$0B, and \$80 – to \$A8.

*CAUTION: FDOPEN implicitly opens a directory file for reading. So: (a) FDCLOSE should be called after reading is completed, and (b) **only one such file may be opened at a time**, because FD... functions are not able to internally maintain more open handles simultaneously.*

11 Loading Binaries

11.1 Loading a program to memory (U_LOAD)

The symbol U_LOAD points to the SpartaDOS X binary loader. The loader performs three functions: (a) loading binaries to RAM and executing them, (b) loading binaries without execution, (c) execution of a previously loaded binary.

Input arguments for U_LOAD are: binary file specification pointed to with the FILE_P vector, and a control value in the FLAG system variable. The specified file is opened for reads with \$PATH searching (*mode*=\$24), and, when it turns out to be a proper binary file, it is loaded to the memory. Any errors will abort loading and invoke U_FAIL.

The FLAG variable decides what to do with the loaded program. When bit 7 equals 0, the program is executed immediately. Otherwise, if it is a relocatable SpartaDOS X program, free memory pointers get updated and the control returns to the caller.

Execution of a previously loaded program occurs when U_LOAD is called again with the same file specification and bit 7 of FLAG having been set to 0. It will work so if the program, which is about to be executed, has defined a symbol consisting of the program's name preceded with a '@' character.

As of SpartaDOS X 4.47 U_LOAD recognizes the following values of FLAG:

- *\$00 – the program will be executed with the use of INIT and RUN vectors. When there is no RUN vector defined, the program will be executed from the beginning of the first segment. This is the default mode for the binary loader in all SpartaDOS X versions.*
- *\$40 – like \$00, but if the program does not set the RUN vector, it will not get executed. This is an AtariDOS compatible mode.*
- *\$80 – the program will get loaded, but INIT and RUN vectors will get ignored. This behavior is compatible with SpartaDOS 3.x – previous versions of SpartaDOS X ignored the RUN vector, but the INIT vectors were used.*
- *\$C0 – the program will get loaded and INIT vectors called, but the RUN vector will get ignored.*

After a program has been loaded, the memory is setup in the standard way, i.e. in the RAM extension area the home bank is present (a value of \$FF in PORTB). The EXTENDED variable contains a memory index for *V_setme* (or, in other words, an index of the extended memory, if any of the binary segments was loaded to the extended RAM, or \$00 otherwise). As it was said above (page 14), a resident program should save this value in a private variable, because after a return to the U_LOAD (or after terminating the program) EXTENDED gets zeroed. A non-resident program may use EXTENDED directly, even if has not been started immediately after loading, but executed after some time through the aforementioned symbol. In such a situation the EXTENDED value is fetched by the library from the symbol's memory index field before starting the program's code.

When U_LOAD is used to load an overlay for a TSR program, before a call to U_LOAD one should decrement the value of the SYSLEVEL.

As of SpartaDOS X 4.46 loading a program using U_LOAD will cause the *path* buffer (\$07A0) to be filled with an absolute path to the directory, where the binary was loaded from. This way a program can easily access a directory where it was loaded from. But: it will not work, if between loading and executing the program the user opens any other file. In this case the *path* buffer contents will get destroyed.

Regaining control imposes no problem, if the executed program ends with an RTS – in such a case the caller simply continues execution at the instruction located directly after JSR U_LOAD.

As of SpartaDOS X 4.47, after a program terminates, the vectors DOSVEC (\$0A-\$0B) and DOSINI (\$0C-\$0D) get reset to a default state.

The matter is a bit more serious, if the program called ends with a JMP (\$000A). Normally, this causes a return to DOS, not to the direct caller. But the DOS will then perform an indirect jump through a vector called *vdos* \$07E3. If it is urgent to regain control, the vector should be taken over and directed at a routine, which has to run after U_LOAD. That routine should before all restore the original value of the vector (from the time before calling U_LOAD).

*It should be kept in mind that programs ending via JMP (\$000A) do this because of changing the stack contents, which in turn makes it impossible to return via RTS. Therefore a program which regains control by taking the *vdos* over, should use JMP (\$000A) to terminate itself as well.*

11.2 Unloading a program (U_UNLOAD)

When a program was executed normally, its unloading occurs automatically after it returns control to the Command Processor. Otherwise (loaded with FLAG > \$7F) in order to remove the code from the memory U_UNLOAD should be called. That function removes everything the program has installed in the memory by means of U_LOAD (including overlays, symbols etc.)

12 File Management Functions

12.1 Rename file (RENAME)

Changing the name of the specified file is done by a function called RENAME. The input parameter is a complete specification of the name to be changed. That specification should be pointed to by FILE_P. After the source specification a new name should follow, separated with a blank space or a coma. The entire string should be EOL-terminated. Possible errors invoke U_FAIL automatically.

A call to RENAME may change the contents of *scan* (\$0779). Changing the name of a directory using this function is not possible.

In older versions of SpartaDOS X it was possible to change names so that more than one file in the directory could eventually have the same name. As of SpartaDOS X 4.40 it is not possible any more.

12.2 Rename directory (RENDIR)

Changing the name of a specified directory can be done with RENDIR. Analogically as with RENAME, the input parameter is a complete specification of a name to be changed, pointed to by FILE_P. After the source specification a new name should follow, separated with a blank space or a coma. The entire string should be EOL-terminated. Possible errors invoke U_FAIL automatically.

A call to RENDIR may change the contents of *scan* (\$0779).

This function is available as of SpartaDOS X 4.42.

12.3 Delete file (REMOVE)

Deleting a file can be done with REMOVE. The required input parameter is a complete specification of a file to be removed, pointed to by FILE_P. Possible errors invoke U_FAIL automatically.

A call to REMOVE may change the contents of *scan* (\$0779).

12.4 Delete directory (RMDIR)

Deleting a directory is done with RMDIR. The required input parameter is a complete specification of a directory to be removed (without the final separator), pointed to by FILE_P. Possible errors invoke U_FAIL automatically.

A call to RMDIR may change the contents of *scan* (\$0779).

The directory must be empty: the deletion of a directory together with files and subdirectories contained in it, is beyond the library's abilities. If such an action is necessary, a program called DELTREE.COM residing on the ROM cartridge should be invoked, with the specification of a directory to be deleted as a parameter.

12.5 Making directory (MKDIR)

A new directory is created with MKDIR. The required input parameter is a complete specification of a directory to be removed (without the final separator), pointed to by FILE_P. Possible errors invoke U_FAIL automatically.

A call to MKDIR may change the contents of *scan* (\$0779).

12.6 Change current directory (CHDIR)

CHDIR changes the current directory to the one which is specified by a full pathname pointed to by FILE_P. An error invokes U_FAIL.

12.7 Get current directory (GETCWD)

GETCWD reads the path from the main directory of a disk to its current directory, and saves the said path in *path* (\$07A0-\$07DF) in a form of a NUL-terminated ASCII string. An empty string (\$00 at the beginning) means that the main directory is the current one. Before making the call the specification of a disk (in an ASCII form, EOL-terminated) should be pointed to by FILE_P. An error invokes U_FAIL automatically.

In older SpartaDOS X versions this function did not return the extensions for directory names. It has been corrected as of SpartaDOS X 4.40.

12.8 Changing attributes (CHMOD)

Changing file's attributes can be done with CHMOD. The required parameters are: file's specification pointed to by FILE_P, a mask of file search attributes in *scan* (\$0779), and a mask of new attributes in *attr* (\$077A).

The mask of required attributes selects files to undergo changing attributes. It works the same way as in other calls, which take the mask into consideration (e.g. FOPEN):

+\$01: protected only (+P)	+\$10: unprotected only (-P)
+\$02: hidden only (+H)	+\$20: unhidden only (-H)
+\$04: archived only (+A)	+\$40: not archived only (-A)
+\$08: directories only (+S)	+\$80: not directories only (-S)

A value of \$00 selects all files.

In the mask of new attributes the bits have meaning as follows (when set to 1):

+\$01: protected only (+P)	+\$10: unprotected only (-P)
+\$02: hidden only (+H)	+\$20: unhidden only (-H)
+\$04: archived only (+A)	+\$40: not archived only (-A)

The S attribute (subdirectory) may of course not be changed this way. An error automatically invokes U_FAIL.

12.9 Set BOOT file (SETBOOT)

SETBOOT selects the specified (by FILE_P) binary file as the one which will get booted at the next system restart. This function of course only works on SpartaDOS disks. An error will automatically invoke U_FAIL.

13 Other Disk Related Functions

13.1 Format disk (FORMAT)

The FORMAT function does nothing else besides invoking the menu of the built-in formatter. The caller does not pass any parameters to it.

In older versions of SpartaDOS X calling the formatter this way required a „manual“ (by writing appropriate hardware registers) switching the cartridge to bank 0. As of SpartaDOS X 4.40 this inconvenience has been lifted, the system selects the appropriate bank automatically, and, besides, the formatter does not reside in bank 0 any more. Nevertheless, if compatibility is the issue, it is better to invoke the formatter by a call to XIO 254.

13.2 Soft formatting (BUILDDIR)

BUILDDIR soft formats a disk, i.e. writes an empty main directory, default bitmap and the bootsector. It is the only method of formatting available for RAM disks and hard disk partitions. The input parameters are:

- 1) Device identifier in *ddevic* (\$0300). \$31 for a disk.
- 2) Unit number in *dunit* (\$0301).
- 3) New disk label pointer in AX (low/high byte, respectively)

A disk label consists of 8 ASCII characters. If it is shorter, it should be space padded to that length. A label **may not** begin with a space.

The function reads the drive's configuration and calculates the rest of the parameters (like sector count etc.) automatically. As of SpartaDOS X 4.40 the bitmap optimization (*Optimize* in the formatter's menu) is enabled automatically too.

13.3 Get disk parameters (GETDFREE)

GETDFREE reads various information about the disk and stores it in *path* (\$07A0-\$07DF). Before the call the FILE_P vector should point to a valid drive specification (in a form of EOL-terminated ASCII string). If an error occurs, U_FAIL is invoked automatically.

The data (17 bytes) is stored in *path* as follows:

- \$00: file system identification code
- \$01: encoded number of bytes per logical sector
- \$02: total number of logical sectors (2 bytes)
- \$04: current number of free logical sectors (2 bytes)
- \$06: disk label (8 bytes)
- \$0E: sequential number
- \$0F: random number
- \$10: reserved byte

The file system identification code allows what a file system is in use on the specified disk, according to the following scheme:

- \$0x - AtariDOS v. x.0 (e.g. \$02 = DOS 2.0) file system
- \$11 - SpartaDOS v. 1.1 disk file system

- \$2x – SpartaDOS v. 2.x disk file system
- \$3C – PC-MIRROR
- \$80 – non-disk file system (e.g. CAR:)
- \$FB – reserved for FAT32
- \$FC – FAT16
- \$FD – FAT12
- \$FE – CP/M 2.0
- \$FF – MyDOS file system

*Remark: GETDFREE calls the kernel function number \$13. As of SpartaDOS X 4.41 the format of the data returned by the kernel upon that call differs completely from what is returned by GETDFREE. Up to SpartaDOS X 4.22 it was not so – the kernel function \$13 in 4.4x is not compatible with the same function in 4.2x. For compatibility, programs should invoke that function **only** through the library (i.e. through GETDFREE), or, if it is not possible, through OS' XIO 47. Both calls convert data returned by the kernel to the „old“ format, compatible with 4.2x.*

13.4 Change volume name (CHVOL)

This function changes the volume name to the specified one. Before the call the name along with drive's specification should be given following the general rule for file and directory names (in other words: CHVOL accepts volume name the same way as e.g. MKDIR accepts directory name).

A volume name can be up to 8 characters long; if it is shorter, it will get padded with spaces; if it is longer, the additional characters will get ignored. The name cannot contain extension: if one is given, it will get ignored. But a volume name may contain arbitrary characters illegal in file names, and inverse video. Specifying a path does not cause an error, but any pathname given will get ignored as well.

A parameter construed so should be pointed to by FILE_P. Errors will automatically invoke U_FAIL.

The CHVOL function appeared in SpartaDOS X 4.43. The CHVOL.COM utility uses it to change volume names.

13.5 Get and change current drive (CURDEV)

Current device specification (a current drive spec, for example) is held in a variable pointed to by symbol CURDEV. Its internal structure is the same as in the kernel parameter device (\$0761, see chapter 18.7).

14 Auxiliary Functions

14.1 32-bit multiplication and division (MUL_32/DIV_32)

The library contains two routines to make unsigned integer calculation: MUL_32 to multiply two 32-bit numbers, and DIV_32 to divide such numbers.

The component numbers must first be written *using the inverted byte order (MSB first!)*, as follows:

- 1) multiplicand (or dividend) into COMTAB+\$FF (4 bytes)
- 2) multiplier (or divisor) into COMTAB+\$0103 (4 bytes)

The result, written in the same unusual word order, can be fetched from COMTAB+\$0107.

DIV_32 has some disadvantage: it only returns the integer part of the result, and it does not care to provide the remainder. Obtaining a remainder requires an additional calculation, as follows:

- 1) *to multiply the result by the divisor,*
- 2) *subtracting the result of that multiplication from the dividend.*

The result of the subtraction will be the required remainder. If however the divisor is a power of two, the calculation may be reduced to:

- 1) *decrementing the divisor by 1,*
- 2) *doing binary AND between the dividend and the decremented divisor.*

The result of the conjunction is the remainder of the division. But, at the other hand, when the divisor is a power of two, the division is simple enough and it is not necessary to go through it with DIV_32.

Both routines, MUL_32 and DIV_32, return with Carry set (C=1) when an overflow occurred.

14.2 Converting characters to upper-case (TOUPPER)

Calling TOUPPER with an ASCII code of a lower-case character in the accumulator, will convert the character to upper case. If the code does not symbolize a lower case letter, the function does nothing whatsoever.

14.3 Check for path separator (CKSPEC)

The function CKSPEC checks, if the character passed on in the accumulator is one of the following: ':", '>', '\', '<' (these are separators which may occur within a pathname), and if so, returns with a zero result (Z=1). The result is non-zero (Z=0) otherwise.

A `jsr CKSPEC` should be used instead of `CMP #'>'` while analysing pathnames.

15 Initializations

15.1 Initializing TSR programs after a RESET (S_ADDIZ)

A part of TSR programs must be reinitialized after the user hits the RESET key. This is required for all sort of CIO drivers, such as CON64.SYS. After a reset these will need to recreate entries in the OS' handler table, for example.

Such TSR programs can be registered using S_ADDIZ. In the AX registers (LSB and MSB respectively) an address of an init routine should be passed. The DOS will write this address to a special queue, and after a RESET it will execute all subroutines registered in that queue. A routine registered there should perform all necessary initializations and then end with an RTS.

The queue has only five free places, if there is no place for more entries, S_ADDIZ returns with Carry bit set (C=1).

15.2 Unconditional exit to DOS (_DOS)

Executing JMP _DOS will do almost the same thing as JMP (DOSVEC) – the calling program will get terminated and removed from the memory, and the control returns to the shell. An important difference is that after JMP _DOS the DOS will get entered unconditionally and the vector vdos \$07E3 will get ignored. Because of that, if a program was invoked from within another program (via U_LOAD), the calling program does not have a possibility to regain control, but it will get terminated as well.

JMP _DOS should not be overused, because technically it is possible to take this call over: it is enough for a calling program to create own symbol _DOS pointing to own routine. Therefore it is better to leave JMP _DOS for exceptional occasions and terminate programs with JMP (DOSVEC), allowing parent programs to regain control in easy way by changing the vdos vector.

A better method of program termination is to use RTS – this allows to return to the calling program (which is not necessarily DOS) without additional trouble.

15.3 SpartaDOS X warm restart (_INITZ)

Calling _INITZ causes a warm restart of SpartaDOS X system. All files will get closed, settings reset, resident kernel drivers reinitialized.

_INITZ is one of the procedures invoked after hitting the RESET key by the user. In older versions of SpartaDOS X executing _INITZ caused all the current paths on all the drives to be reset to main directories. As of SpartaDOS X 4.40 the SPARTA.SYS driver is able to recognize that its init routine is called from within _INITZ and the paths are not reset that time.

16 Manipulating The Symbol List

16.1 Looking up the symbols (S_LOOKUP)

The system contains functions to search the symbol list, adding symbols to the list, and removing them from the list. Programs normally do not use them, because most of the pertinent things are done by the SpartaDOS X binary loader. A list of symbols attached to a program's binary file is automatically translated to addresses, which in turn are automatically put into the proper places of a loaded program. A definition of a new symbol by a program looks similar: that definition is encoded in the structure of a binary file itself, and it is the loader which creates a new symbol and adds it to the global list.

From time to time it is necessary for a program to search the symbol list „manually“. For example, the CAR:Z.SYS driver needs an access to the symbol I_FMTTD (defined by TD.COM), but it cannot be dependent on its presence, because then it would not be possible to load Z.SYS without TD.COM loaded previously (if a symbol is impossible to „resolve“ at the time a program loads, the loading is aborted and an error „Symbol not defined“ occurs).

In such a case one should use the S_LOOKUP function. It requires the name of a symbol to be searched for be written into SYMBOL+\$02 (space padded to eight characters, if necessary), and clearing the EXTENDED byte. Calling JSR S_LOOKUP returns a zero (Z=1), if the symbol being searched for does not exist. Otherwise the result is different than a zero (Z=0), and under SYMBOL+\$0B and SYMBOL+\$0C there are LSB and MSB, respectively, of an address pointed to by the required symbol, and EXTENDED contains the corresponding memory index.

As the S_LOOKUP itself, the SYMBOL structure and the EXTENDED variable are all pointed to by symbols, it is clearer than light that this way of accessing the symbol list is open only for programs which are stored in SpartaDOS X relocatable binaries. Besides, ordinary AtariDOS binaries usually do not need an access to the symbol list. But, just in case an access was necessary, another entry to S_LOOKUP exists, not only available at a fixed address, but also easier to use.

That procedure, labelled *jfsymbol*, was first introduced in SpartaDOS X 4.40. The entry is located at \$07EB. The input parameter is the address of a space-padded symbol name in AX registers (LSB/MSB, respectively). When the symbol is not found, the function returns a zero (Z=1), otherwise the AX registers contain the 16-bit address, and the Y register – the memory index. *The obvious limitation here is that 24-bit addresses cannot be returned this way, so when Y=\$03 upon return, one should fetch the actual Y value from SYMBOL+\$0D.*

CAUTION: after a program is started using X.COM, the symbol list and most of the objects registered on it become UNAVAILABLE. You still however can use jfsymbol to search for RAM structures such as the T_ table.

16.2 Walking through symbol list (S_NEXT)

As of SpartaDOS X 4.46 the system contains a function called S_NEXT which allows a program to scan the entire symbol list. This function is used by SL.COM program (to be found on the Toolkit disk). Before S_NEXT is first called, the third byte of the SYMBOL structure should be zeroed (at address SYMBOL+\$02). When S_NEXT returns with a non-zero result (Z=0) means that the SYMBOL structure now contains a symbol. Any next call to S_NEXT –

without zeroing anything this time – will store the next symbol in the same place. If the procedure returns with a zero result ($Z=1$), the end of the list was reached.

16.3 Adding a symbol (S_ADD)

After calling S_ADD a specified symbol will get added to the global list. The input data should be stored in the SYMBOL structure prior to the call. To remind, the structure looks as follows:

- \$00-\$01: a pointer to a next symbol (2 bytes)
- \$02-\$09: symbol's name (8 ASCII characters)
- \$0A: program level index
- \$0B-\$0D: target address (2 or 3 bytes)

S_ADD requires that the calling program fills the name (padding it with spaces up to eight characters), and setting up the corresponding 16-bit address in SYMBOL+\$0B and SYMBOL+\$0C. The corresponding memory index should be stored in EXTENDED, unless it was already stored there (by S_LOOKUP for example). The program level index and the pointer to a next symbol are filled in automatically.

The symbol is added in the memory location pointed to by *memlo*. That pointer is then raised by 13 bytes.

16.4 Removing symbols (S_CLEAR)

The S_CLEAR procedure removes symbols from the global list. Any symbol is removed, which has been assigned a PID greater than the PID of the calling program. It is a part of U_UNLOAD, and the „current PID” cannot be explicitly specified by an application. Therefore the purpose of exporting S_CLEAR to the global symbol list is not completely clear.

17 Remaining Symbols

Apart from the symbols described above, there are some more which have not been explained. These are variables like `SYSLEVEL`, tables like `H_FENCE`, functions like `_CIO` and `_EDIT`. Most of them can only be used in system drivers and are of no interest to application programmers.

18 Some Programming Techniques

18.1 Example application program

Here is a very simple example program which could serve as a skeleton for your developments.

18.1.1 FA's syntax

FA, or Fast Assembler, is a specialized assembler available on Atari since about 1995.

```

PRINTF  smb 'PRINTF'           ;labels defined as external
GETS    smb 'GETS'             ;symbol references

        blk reloc main         ;relocatable block loaded to
                                ;'main' memory (mem index $00)
; the program starts here

hello   jsr PRINTF
        dta c'Type your name: '
        dta b($00)

        lda whov                ;buffer address in AX
        ldx whov+1
        ldy #32                 ;buffer size (31 chars + EOL)
        jsr GETS                ;GET RECORD from console
        bmi quit                ;quit on EOF (Ctrl/3)

        jsr PRINTF
        dta b($9b)
        dta c'Hello, %s!'
        dta b($9b),b($00)
whov    dta v(who)              ;buffer pointer

quit    rts                    ;just RTS to terminate

        blk empty main 32       ;new block with mem index $80
who     equ *-32

        blk update address
        blk update symbols
        blk update new hello '@HELLO' ;export start address

```

18.1.2 ELSA's syntax

ELSA is an assembler running on Ataris equipped with 65C816 CPU.

```

.out hello.com

.xref PRINTF           ;labels defined as external
.xref GETS             ;symbol references

.rel $00               ;relocatable block loaded to
                        ;'main' memory (mem index $00)

.bss
who .ds 32
.code

```

; the program starts here

```
@hello%                                ;export start address
    jsr PRINTF
    .byte "Type your name: ",0

    lda whov                            ;buffer address in AX
    ldx whov+1
    ldy #32                             ;buffer size (31 chars + EOL)
    jsr GETS                             ;GET RECORD from console
    bmi quit                             ;quit on EOF (Ctrl/3)

    jsr PRINTF
    .byte $9b,"Hello, %s!", $9b,0
whov  .word who                          ;buffer pointer

quit  rts                               ;just RTS to terminate

    .bss *                              ;put BSS here
```

18.1.3 MADS's syntax

MADS is a quite popular cross-assembler running on PC. *Its „feature“, causing in practice much trouble if someone is not accustomed to, is that if a text string is enclosed in single quotes ('...'), MADS generates ASCII codes for it, but the same in double quotes ("...") is generated as ANTIC screen codes.*

```
PRINTF smb 'PRINTF'                    ;labels defined as external
GETS   smb 'GETS'                      ;symbol references

    blk reloc main                     ;relocatable block loaded to
                                        ;'main' memory (mem index $00)

; the program starts here
```

```
hello  jsr PRINTF
    .byte 'Type your name: ',0

    lda whov                            ;buffer address in AX
    ldx whov+1
    ldy #32                             ;buffer size (31 chars + EOL)
    jsr GETS                             ;GET RECORD from console
    bmi quit                             ;quit on EOF (Ctrl/3)

    jsr PRINTF
    .byte $9b,'Hello, %s!', $9b,0
whov  .word who                          ;buffer pointer

quit  rts                               ;just RTS to terminate

who   .ds 32                             ;new block with mem index $80

    blk update new hello '@HELLO' ;export start address
```

18.1.4 Commentary

The program does not load the buffer pointer directly (using *lda #<who* and *ldx #>who*), because the internal addresses of a relocatable program cannot be split up into bytes – the relocater would not be able to fix up such a reference. This is why an indirection – through the pointer *whov* in this case – must be used.

The last line in FA and MADS listings defines symbol *@HELLO* pointing to the label *hello* (in this case to the beginning of the program). This enables the user to load the program to the computer's memory without execution (using the LOAD command), and execute it later just by typing „HELLO” at the Command Processor prompt and pressing Return. The same is accomplished in ELSA by appending a '%' at the end of the label which is to be exported this way.

A program may be executed thus many times, having been loaded only once. It should be remembered though, if the program uses any static variables, that these must be initialized carefully at each execution, or undefined things may happen. It is not enough to just have preinitialized data in the binary file, because these will only be valid during the first execution of the program.

18.2 Example program 2: Reading data from a file

Task: open a file with a name specified by the user in the command line, read its first byte, display it, then quit to DOS. ELSA's syntax:

```
.out gbf.com                ;or "get byte from file"

.xref U_GETATR
.xref U_FSPEC
.xref U_EXPAND
.xref FOPEN
.xref FGETC
.xref FCLOSE
.xref PRINTF

mode    = $0778
name    = $0762
DEfname = $078f

.rel 0                      ;memory index 0 (conventional 64k)

@gbf%  lda #%101000000      ;ignore directories and hidden files
      jsr U_GETATR          ;get user-specified file name
      jsr U_FSPEC           ;assume *.* when nothing specified
      lda #$04              ;access mode: read
      sta mode              ;'scan' is set automatically by U_GETATR
      jsr FOPEN             ;open
      jsr FGETC             ;get first byte
      bmi ?eof              ;EOF – the file is zero-length, do nothing

      sta data

      ldx #$0a
?cp    lda DEfname,x        ;see what we opened
      sta name,x            ;provide input data for following call
```

```

    dex
    bpl ?cp

    lda ffnp                ;convert 'name' to 8+3
    ldx ffnp+1              ;store at ffname
    ldy #$00                ;with offset 0
    jsr U_EXPAND

    jsr PRINTF
    .byte $9b,"File: %s, byte read: $%02x",$9b,0
    .word ffname,data

?eof    jmp FCLOSE          ;close file and return to DOS

ffnp    .word ffname

        .rel $80            ;mem. index 0 + bit 7 set (reserve memory)

data    .ds 1               ;temporary storage for our byte
ffname  .ds 8+1+3+1         ;file name + dot + extension + EOL

```

The file should assemble to 250 bytes. If you run it giving it GBF.COM (i.e. itself) as input file name, it should reply as follows:

```
File: GBF.COM, byte read: $FE
```

18.3 Example 3: A resident program

A resident program is usually a system driver (like RAMDISK.SYS for example) which installs itself permanently in the memory. It consists of a non-resident part, being usually loaded to pages 4, 5, 6, which performs some one-time actions such as checking installation conditions and registering the program in the appropriate system structures; and the resident part, which is loaded by the relocation loader over the *memlo* and optionally to the extended RAM, and may be executed many times as a part of the operating system.

18.3.1 With the use of system extended RAM

ELSA's syntax:

```

    .out tsrex.com

    .xref EXTENDED
    .xref INSTALL
    .xref PRINTF

V_setme = $07F1
V_popme = $07F4

    .abs $0600              ;non-resident part

inst    lda EXTENDED        ;fetch extended RAM information
        sta ext_m           ;store it in private variable for later

        jsr PRINTF          ;print acknowledgment for the user

```

```

        .byte "TSR example v.1.0 installed", $9b, 0

        dec INSTALL          ;make "stay resident" signal
        rts                  ;and quit

; resident part

        .rel 0               ;memory index 0 (conventional 64k)

@tsr%
        lda ext_m            ;export entry point to symbol list
        jsr V_setme          ;load our memory index
        jsr do_tsr           ;enable extended RAM
        jmp V_popme          ;call our code there
        ;disable extended RAM and quit
ext_m   .byte 0

        .rel 2               ;memory index 2 (system extended RAM)

do_tsr  jsr PRINTF           ;print message
        .byte "Hello there!", $9b, 0
        rts

```

The program, when executed, will install itself in the memory. The message „Hello there!“ will appear whenever the user types „TSR“ at the DOS prompt and hits the Return key.

18.3.2 With the use of program extended RAM

The same program as above with the use of the „user“ extended banks, ELSA's syntax:

```

        .out tsrex2.com

        .xref EXTENDED
        .xref INSTALL
        .xref PRINTF

V_setme = $07F1
V_popme = $07F4

        .abs $0600           ;non-resident part
                                ;fetch extended RAM information:
inst    ldx EXTENDED+1       ;count of blocks loaded
        lda EXTENDED+3-1,x   ;memory index of the last one
        sta ext_m            ;store it in private variable for later

        jsr PRINTF           ;print acknowledgment for the user
        .byte "TSR example v.2.0 installed", $9b, 0

        dec INSTALL          ;make "stay resident" signal
        rts                  ;and quit

; resident part

        .rel 0               ;memory index 0 (conventional 64k)

@tsr2%
        lda ext_m            ;export entry point to symbol list
                                ;load our memory index

```

```

        jsr V_setme           ;enable extended RAM
        jsr do_tsr           ;call our code there
        jmp V_popme         ;disable extended RAM and quit
ext_m   .byte 0

        .rel 4               ;memory index 4 (program extended RAM)

do_tsr  jsr PRINTF           ;print message
        .byte "Hello there 2.0!", $9b, 0
        rts

```

The message will appear when the user types „TSR2” at the DOS prompt, then hits the Return key.

Note that this example is rather wasteful, as an entire extension bank (16k) will be allocated for the *do_tsr* part. Therefore this method is useful rather for those resident programs which either are large or need large memory buffers (such as ENV.SYS, for example).

18.3.3 Common optimizations

Needless to say, the two examples above for code clarity do not use some optimization techniques commonly employed in the outside world, primarily to save some bytes in the conventional RAM. So, instead of this:

```

@tsr%   lda ext_m
        jsr V_setme
        jsr do_tsr
        jmp V_popme
ext_m   .byte 0
        ...
do_tsr  jsr PRINTF
        .byte "Hello there!", $9b, 0
        rts

```

we rather do this:

```

ext_m = @tsr+1

@tsr%   lda #$000
        jsr V_setme
        jmp do_tsr
        ...
do_tsr  jsr PRINTF
        .byte "Hello there!", $9b, 0
        jmp V_popme

```

This saves some CPU cycles as well as some bytes of the conventional memory, partially at the expense of the extended RAM.

18.4 Converting ASCII digits to a value

The PRINTF function described in chapter 9.3 converts binary values to decimal and hexadecimal ASCII strings. It is sometimes necessary to do a conversion the other way, i.e. to convert an ASCII string of digits to a binary value; for example, this may be necessary when

reading a number from an environment variable. The library, however, does not offer a suitable function.

But this does not mean that it does not contain a suitable procedure. Such one exists and is used by PRINTF to fetch numeric values which the formatting pattern may contain, and by U_GETNUM. That last function may be used to convert anything.

To remind, U_GETNUM expects an EOL-terminated string of digits in LBUF (COMTAB+\$3F, 64 bytes), at the position pointed to by BUFOFF (COMTAB+\$0A), and a hex number is preceded with a '\$' character. So, to convert that into a binary value, one should do this:

- 1) Save the contents of LBUF and BUFOFF somewhere, if it seems necessary, or in other words, if the program is yet going to parse any more arguments in the command line;
- 2) write an EOL-terminated string of digits into LBUF;
- 3) zero out BUFOFF;
- 4) call U_GETNUM;
- 5) optionally restore LBUF and BUFOFF.

Calling U_GETNUM returns a zero (Z=1), when the parameter is not a number. Otherwise (Z=0) in the AX registers the LSB and MSB of the corresponding binary value will be returned.

As of SpartaDOS X 4.44 U_GETNUM can decode 32-bit unsigned numbers (in range from 0 to 4,294,967,295), but the CPU registers will still carry the lower word only – the upper word is to be found in divend+2 and divend+3 (COMTAB-4 and COMTAB-3, respectively). For backward compatibility with older SpartaDOS X versions it is a good practice to zero out these bytes before making a call to U_GETNUM.

The BUFOFF is automatically incremented, so repeating calls to U_GETNUM (see point 4 above) until Z=1 is returned will allow to convert a series of hexadecimal and decimal numbers separated with commas or spaces.

```
string = $0600                                ;the address of the string to convert

convrt  ldx #$00
        stx COMTAB+$0a                        ;BUFOFF
?loop   lda string,x
        sta COMTAB+$3F,x                      ;LBUF
        inx
        cmp #$9b
        bne ?loop
        jsr U_GETNUM
        beq ?not_num
        sta value_low
        stx value_high
        rts
?not_num
...
```

It is a good example how to use the command line parsing function in a non-standard way. Similarly U_TOKEN will be useful to look-up any keyword tables, and not only containing these keywords which the user typed from the keyboard. This way the function is used by DOSKEY – to look-up a table of aliases.

18.5 SPRINTF

One of the functions SpartaDOS X library lacks is *sprintf*, a function known from the C standard library. It works similarly to the PRINTF described in chapter 9, but the output string is written to the memory instead of to the screen. To simulate this function one can use the mechanism of vectoring the output, described in chapter 9.7, via PUT_V and VPRINTF.

Let us assume that the parameters to the SPRINTF subroutine are passed exactly the same way as to VPRINTF (or PRINTF), just with the exception that before the formatting string a buffer address should be given as the first parameter, where the output string should go into. Let us additionally assume that the destination string will be EOL-terminated.

Here is an example solution:

```
sprintf pla
    sta tmp          ;zero page variable
    pla
    sta tmp+1
    ldy #$01         ;get buffer address
    lda (tmp),y
    sta ?out+1
    iny
    lda (tmp),y
    sta ?out+2
    tya              ;fix return address
    clc
    adc tmp
    tax
    lda #$00
    adc tmp+1
    pha              ;push it back to stack
    txa
    pha
    ldx #$02         ;set PUT_V
?cpv   lda ?outv-1,x
        sta PUT_V-1,x
        dex
        bne ?cpv
        stx ?outx    ;zero out buffer index
        jmp VPRINTF

?outv   .word ?store
?outx   .byte 0

?store  stx ?savx+1
        ldx ?outx
        jsr ?out
        lda #$9B
```

```

        inx
        jsr ?out
        inc ?outx
?savx   ldx #$000
        rts
?out    sta $ffff,x
        rts

```

The subroutine can be significantly simplified if a fixed address of the output buffer is assumed: then passing its address among other parameters is not necessary, and this, in turn, allows to remove 19 initial instructions from the `SPRINTF`, and also allows some simplification in the `?store` subroutine.

18.6 Kernel device types

There are six predefined kernel devices in SpartaDOS X: DSK:, CLK:, CAR:, CON:, PRN: and NUL: The *device* (\$0761) register assigns them values as follows: \$0x, \$1x, \$2x, \$3x, \$4x, \$5x. A part of them, e.g. CAR:, are file devices – these can contain and allow access to multiple data files. Some other are character devices – each one behaves more or less like a separate file. The remaining ones are special devices which are, well, special.

Sometimes it is necessary to make sure, if a file specification given by the user belongs to a file device, to a character device or to a special device. For example, the command COPY requires a file mask in the destination file spec (and "*" is the default) while copying from a file device, and forbids using one while copying from a character device. Therefore there must be a method to check what a device the program is dealing with.

The naive approach, i.e. decoding the name using `U_GEFINA` then comparing the result with the known *device* values, is not a good idea, because the assignments of these codes may change – for example, someone will de-install one of the character devices and install a file system one instead (or vice versa). Not to speak about the fact that two places in the table (corresponding to codes \$6x and \$7x), are free, and a device, that can occupy them, can equally be file oriented or character oriented (or special).

18.6.1 Checking for a special device

Special devices do not provide regular I/O streams, thus do not allow opening them. This is the easiest way of checking for a special device:

```

        jsr U_GETPAR           ;get file spec
        beq ?brak             ;branch if nothing given
        jsr U_FSPEC           ;add default mask
        lda trapv             ;set trap
        ldx trapv+1
        jsr U_SFAIL
        lda #$04              ;read mode
        sta mode
        lda #$000
        sta scan
        jsr FOPEN             ;test
        jsr FCLOSE            ;necessary!
        jsr U_XFAIL           ;remove trap
        lda #$80              ;result: file or character device
?brak   rts

```

```

trap    cmp #146                ;function not implemented
        bne ?error
        lda #$01                ;result: special device
        rts
?error  jmp U_FAIL               ;some error
trapv   .word trap

```

18.6.2 Checking for a file device

A method of checking for a file device can be used before passing the given file specification onto FOPEN (or, in other words, before opening the file or device).

Do it as follows:

```

        jsr U_GETPAR            ;get file spec
        beq ?brak               ;branch if nothing given
        jsr U_FSPEC             ;add default mask
        lda trapv               ;set trap
        ldx trapv+1
        jsr U_SFAIL
        jsr FFIRST               ;test
        jsr FCLOSE               ;necessary!
        jsr U_XFAIL             ;remove trap
        lda #$01                ;result: file device
?brak   rts
trap    cmp #146                ;function not implemented
        bne ?error
        lda #$80                ;result: character or special dev.
        rts
?error  jmp U_FAIL               ;some error
trapv   .word trap

```

The test works so that the character and special devices do not have directories, so a call to FFIRST **must** trigger error number 146 (Function not implemented). This and *only this* error code means that the device we are dealing with is not a file device.

18.6.3 Files vs pseudo-files

Another method of checking if we are dealing with a real file system or a character device can be used when we already passed the file specification to FOPEN, and this one returned no errors, so our program is now owning a handle to an open file. We have to check if that file is a „true” one, located on a file device, or a pseudofile such as CON:, NUL: or such. The method of checking that is similar to the above, with the exception that the function being called is FSEEK:

```

        lda trapv               ;set trap
        ldx trapv+1
        jsr U_SFAIL
        lda #$00
        sta addpos
        sta addpos+1
        sta addpos+2
        jsr FSEEK               ;test
        jsr U_XFAIL             ;remove trap
        lda #$00                ;result: file

```

```

        rts
trap    cmp #146                ;function not implemented
        bne ?error
        lda #$01                ;result: pseudofile
        rts
?error  jmp U_FAIL              ;some error
trapv   .word trap

```

18.7 Decoding device identifier

There are two methods to decode a device identifier, if it is stored in an ASCII form (e.g. D2:, A:, CAR:, NUL: and so on), or in other words converting it to a numeric form. One method is to use U_GEFINA (see page 25).

When the library is not accessible, one can use the *misc_* vector instead, specifically its function number 1 (*misc_gefina*). See chapter 25.

Unfortunately, up to SpartaDOS X 4.42 there was no method of converting a value stored in *device* back into the text form. Problems system programs (such as ARC or MENU) have or had with accessing any device besides the default DSK: (with \$0x identifier) have their origin in this fact.

In SpartaDOS X 4.42 this problem has been solved by adding a symbol DEVNAME pointing to a table of device names. That table contains a list of „long” (three-character) names of devices, such as „DSK”, „CLK” and so on. One entry occupies four bytes, i.e. three characters name and a space as a separator. An empty entry consists of four spaces.

Now it is enough to know that the upper nibble of *device* is the DEVNAME table entry number, and the lower nibble is a unit number of that device (for example, the number of a logical disk drive in the DSK: device). A subroutine doing the translation from *device* code into an ASCII form may look as follows:

```

dev2txt  lda device
        cmp #$ff
        beq error

        pha
        and #$f0
        lsr
        lsr
        tax
        lda DEVNAME,x
        cmp #$20
        beq error
        sta buf
        lda DEVNAME+1,x
        sta buf+1
        lda DEVNAME+2,x
        sta buf+2
        pla
        and #$0f
        ora #$40
        sta buf+3

```

```

        lda #' : '
        sta buf+4
        lda #$9b
        sta buf+5
        clc
        rts
error   sec
        rts

buf     .ds 8

```

To the *buf* buffer the program will store the text name of a device, its unit number, a colon and the terminating EOL character. An error will be signaled by returning with Carry bit set (C=1).

In SpartaDOS X 4.50 this task has been yet further facilitated by adding the *U_PATH* function to the library, which can be used to convert *device* and *path* contents back to the human-readable form.

18.8 Checking for existence of a file

It is sometimes necessary, before opening a file, to check if it exists. Also, when the user has specified an ambiguous file name to open (i.e. with wildcards), it may be crucial to find the first matching file and e.g. ask the user, if this is what he wanted. Also, if the file is about to be saved later, its name must not contain wildcards, because then an error would occur during opening for writes.

Before SpartaDOS X 4.50 the *FOPEN* call might have been used for that, as it returns in *dentry* (\$0789-\$079F) the directory entry of the file being opened – but now it is not recommended because of the presence of symbolic links: if the user has specified a link as the name of the file to open, *FOPEN* will dereference the link and return its target file name. This is most of the time not desired, so to perform the task one should use *FFIRST* instead:

```

        jsr U_GETPAR          ;get file spec
        beq ?brak             ;branch if nothing given
        jsr U_FSPEC           ;add default mask
        jsr FFIRST            ;find file
        php
        jsr FCLOSE            ;necessary!
        plp
        bmi ?fnf
        lda #$01              ;result: file exists
?brak   rts
?fnf    lda #170              ;error
        jmp U_FAIL

```

When the result is positive, the corresponding directory entry will be present at *dentry*, as said above.

18.9 Starting programs with parameters

When a program wants to launch another program, it is sometimes sufficient to use an arbitrary memory location to pass all required data.

Most of the time, however, it is not sufficient, especially when a programmer wants to use programs available on the CAR: device – most of them expects arguments passed in the command line.

Remark 1: regardless of the method of starting, programs stored as SpartaDOS X position independent binaries (see chapter 2) are loaded into the memory buffer pointed to by the memlo vector. If the parent program keeps data in that memory, the data will get destroyed. To prevent that, one has to use MALLOC – this function will raise free memory pointers protecting the data.

Remark 2: the loaded program (or programs) can use the area \$80-\$FF for own purposes; so it is a good practice to assume, that after calling U_LOAD or XCOMLI any data previously stored in that area get destroyed.

Remark 3: about regaining control after loading another program – see U_LOAD function description (chapter 11).

18.9.1 Starting a program directly with U_LOAD

The first method will work on all SpartaDOS X versions. It requires storing a complete command line, including program name and all the parameters, to the LBUF buffer (COMTAB+\$3F, 64 bytes) and zeroing out BUFOFF (COMTAB+\$0A). Next, because the „child” program being started expects that the „parent” (which is COMMAND.COM most of the time) has already fetched its name from the command line, our program should make a call to U_GETPAR. This function does two necessary things: raises BUFOFF as required and sets up the vector FILE_P.

Now what only remains to be done is to clear FLAG and call U_LOAD. If an error is expected during loading, a trap should be set as described in chapter 4. Errors which may always occur are (in case of position independent binaries): 154 (Symbol not defined) and 158 (Out of memory), and in case of AtariDOS binaries – 179 (Memory conflict).

The procedure below is an example of removing the directory C:>WINDOWS using the CAR:DELTREE.COM program.

```
delwin  ldx  #$00
?cmd    lda  command,x
        sta  COMTAB+63,x      ;LBUF
        inx
        cmp  #$9b
        bne  ?cmd
        lda  #$00
        sta  COMTAB+10       ;BUFOFF
        jsr  U_GETPAR
        lda  trapv
        ldx  trapv+1
        jsr  U_SFAIL
        lda  #$00
        sta  FLAG
        jsr  U_LOAD
        jsr  U_XFAIL
```

```

        lda #000                ;success
trap    rts                    ;or not
trapv   .word trap
cmdln   .byte "DELTREE.COM /Y C:>WINDOWS",9b

```

In this case it is not required to specify a full path, because U_LOAD will search through \$PATH by itself. An inconvenience is that one should specify the full program's file name along with the extension, even if it is a *.COM – because adding default file name extensions is normally handled by COMMAND.COM, being skipped in this case.

Automatic handling different file name extensions may turn out to be a complicated matter, because the expected (by an user) behaviour of a program may differ depending on whether some system extension, like RUNEXT or COMEXE, was installed or not. Therefore, instead of getting into this trouble, it is better to use the method described below.

18.9.2 Starting a program through COMMAND.COM

As of SpartaDOS X 4.42 it is possible to use all the features of the COMMAND.COM in own program – or, in other words, to use it as a kind of a library performing ordered tasks, including executing programs. The library function which allows that is pointed to by the symbol XCOMLI.⁶⁾

Its use is very simple, it is enough to store the entire command line in LBUF and zero out BUFOFF as above, then call the symbol using JSR. There is no need to set error traps up, and additionally it is possible to invoke all the COMMAND.COM functions (including internal commands such as DIR or VER, I/O redirections etc.), except batch file interpretation. The example code below uses this method to perform identical task as specified above:

```

delwin  ldx #000
        stx COMTAB+10          ;BUFOFF
?cmd    lda command,x
        sta COMTAB+63,x        ;LBUF
        inx
        cmp #9b
        bne ?cmd
        jmp XCOMLI
cmdln   .byte "DELTREE /Y C:>WINDOWS",9b

```

Specifying the *.COM extension is not necessary. Moreover, this method, contrary to the former one, will also work on programs which require X.COM to be started, i.e. without the I/O library (but that time one must ensure that the screen memory is located outside the area \$A000-\$BFFF, because XCOMLI requires the library, so it simply enables it when necessary).

These are advantages, but there are disadvantages as well: **calling XCOMLI causes the COMMAND.COM to be loaded to the memory**, so one must make sure that there is

⁶⁾ As *eXecute COMmand Line*. To be specific, this existed in earlier versions as well, but the procedure was not pointed to by a symbol (but by a legacy vector \$FFD2, located „under the ROM” and therefore not existing on 400/800), and its address was different in different revisions of the DOS. As SpartaDOS X 4.42 additionally made this feature work better, it is safer to assume that there was no such procedure at all in older versions.

enough free memory over the *memlo* (5 KiB⁷ ought to be enough). Also, if your program uses any zero-page variables, you must preserve them somewhere before the call and re-store afterwards.

18.9.3 Returning status to the parent program

A program started with `U_LOAD` may pass a resulting status to the parent program. To do that it should load the status code to the accumulator register, then call `JMP U_FAIL`. A negative value will automatically cause an error condition to be triggered, and a positive value will be returned in the accumulator to the parent program. The value is passed through `STATUS` variable, which is automatically zeroed after use.

Another method, suitable for overlays, is to store the status code to the `STATUS` variable, then do `RTS`. A positive value will be returned to the calling program in the accumulator, a negative one will trigger the error trap.

The `XCOMLI` symbol does not allow to pass any status code to the caller.

18.9.4 Loading overlays

Loading and executing overlays is performed the normal way, i.e. by calling `U_LOAD`. Because an overlay is usually a part of our program and we have full control over how it accepts parameters, there is no need to fiddle with command lines. It is best to pass all the parameters using a symbol defined *ad hoc*, and put a reference to it in the overlay's code. A convenient side effect is that it will only be possible to start the overlay from „its“ program, and an attempt to execute the overlay binary from the DOS prompt will cause an error.

An example:

```
runovl  ldx #$02
      _sn  lda ovlnamep-1,x
          sta FILE_P-1,x
          dex
          bne _sn
          stx FLAG
          jmp U_LOAD

ovlnamep .word ovlname
ovlname .byte "OUROVL.OVL", $9b

_params .byte 0,1,2,3
```

The symbol must be defined so that it points to the `_params` array (it contains some arbitrary values here). The name of the symbol should be defined so that it does not conflict with anything else that was already defined in the system – it is best to use the main program's name preceded by an underscore character.

The result values – if any – can be passed to the main program a similar way.

⁷ `COMMAND.COM` in SpartaDOS X 4.50 occupies ~3600 bytes of RAM, but one should take into account that it may grow in the future.

18.9.5 Redirecting output of a child program

A program may quite often want to write something to the screen. It can be controlled in several ways, for example using I/O redirection to a file, or to the memory, or by disabling it completely.

18.9.5.1 Redirecting to a file

The I/O redirections are done with the functions DIVIO and XDIVIO described in chapter 9.6 (q. v.). Using them has sense in case of starting a program with the first method described above, i.e. via direct U_LOAD: a call to DIVIO should then be done before loading, and a call to XDIVIO – after it.

With the second method, the trouble of redirecting the output can be put onto the COMMAND.COM – it gets done by adding a „>>” followed by a name of the file (to which the output has to go) at the end of the command line.

The output can also be completely disabled by using „>>NUL:” here.

18.9.5.2 Redirecting to memory

It can be sometimes needed to take over the text being generated by the child program and to store it in the memory for further processing. It can of course be done by redirecting the output to a file and loading the file afterwards. It is however possible to write the data directly to the memory, without using an intermediate file.

The mechanism of vectored output, described in chapter 9.7, can be used for that. As it was mentioned there, changing a file's handle to a value of 100 (\$64) causes the console's output to be switched so that a routine pointed to by PUT_V vector is called instead of the general display handler. It is enough to know now that the output handle to CON: is located at COMTAB+6. The program below invokes the DIR command, writes its results to a buffer, and puts a terminating NUL at the end of the buffer:

```
get2buf  ldx  #$01                ;change PUT_V
?spv     lda  savp,x
          sta  PUT_V,x
          lda  bufp,x
          sta  sav+1,x
          dex
          bpl  ?spv

          lda  COMTAB+6           ;change stdout handle
          pha
          lda  #100
          sta  COMTAB+6

          ldx  #$00
?cmd      lda  command,x
          sta  COMTAB+63,x        ;LBUF
          inx
          cmp  #$9b
          bne  ?cmd

          lda  #$00
          sta  COMTAB+10         ;BUFOFF
```

```

        jsr XCOMLI

        pla
        sta COMTAB+6           ;restore stdout

        lda #$00              ;terminate buffer
sav      sta $ffff
        inc sav+1
        bne ?skip
        inc sav+2
?skip    rts

savp      .word sav
cmdln     .byte "DIR", $9b

```

Because of the reasons explained above, the routine filling the buffer may not use a zero page pointer.

The SPRINTF routine described earlier in this chapter works using the same principle.

19 65C816 Support

SpartaDOS X, in itself, provides very little support for the 65C816-equipped Ataris, as the ROM space dedicated to system code is hardly large enough to secure the basic 6502 services. Thus most of the support for the extended processor comes in a form of loadable modules.

19.1 The 65816.SYS driver

The most important extension module is 65816.SYS, which provides the basic services such as extended memory management, extended binary loader, extended error handler etc., so that the system is enabled to load and run other extension modules and provides access for application programs to the extended memory.

For certain insuperable technical reasons, this driver requires the OS ROM to contain a 65C816-compatible OS, and it will refuse to load without one. There are two of them which are currently being maintained: DracOS (also known as Rapidus OS) and AltirraOS/816. Therefore also the 65816 driver comes in two flavours: 65816.SYS for the former and 65816A.SYS for the latter. The latter occupies slightly more memory to provide additional services which AltirraOS/816 is missing. *Both modules will be referred to as „65816.SYS“ for simplicity.*

It is crucial to load the 65816.SYS driver as the very first binary in the system (even before SPARTA.SYS). Therefore it should be put onto CAR: and invoked as the first „device“ in your CONFIG.SYS. *This driver is available as of SpartaDOS X 4.50.*

The SpartaDOS X Toolkit Disk contains an example CONFIG.SYS file which shows the correct use of the extension modules at startup.

19.1.1 Calling legacy functions

The „legacy functions“ being talked about here are the functions of the 8-bit operating system in ROM as well as the functions provided by SpartaDOS X. There is a twofold difficulty in using them by a program that runs in 65C816 native mode and in the High RAM past the first 64k:

- a) all of them end with the RTS instruction, which uses a 16-bit return address: this means that a call must be done from within the same 64k block of memory as the one where the function is residing (i.e. from the first 64k, \$000000-\$00FFFF);
- b) many of them may be incompatible with the native mode: this means that the caller must first switch the CPU to the emulation mode, and this, in turn, may result in losing some context, i.e. the contents of the CPU registers.

This difficulty may be solved (or levied) in the ways sketched below. *Also note that this problem is mostly solved by the 65816.SYS module itself, but for most part by the EXT816.SYS extension module, which both provide 16-bit alternatives for the essential functions of the SpartaDOS X cartridge library (see further on).*

19.1.1.1 The Rapidus OS style interface

The 65816.SYS module provides an interface to facilitate making calls to the legacy functions, the same one as defined by the Rapidus OS. Accordingly, the call is done using the software interrupt instruction COP. For example, a call to FCLOSE may look like this:⁸⁾

```
.xref FCLOSE

pea #FCLOSE      ;push function address
cop #$00         ;call
rep #$20         ;switch to 16-bit accumulator
pla              ;pop argument
```

This is convenient enough in most cases, although it should be remembered that *all the possible memory-resident parameters and results for the function called this way will still be located in the first 64k of the address space.*

Sometimes it is crucial, upon return from the call, to preserve all the register contents. For example, a call to U_GETKEY will return data in A and X, the status code in Y and additional information in the NZ bits of the CPU status register. The way to preserve all that and pop the function address off the stack without engaging static variables in the memory may look like this:

```
.xref U_GETKEY

sep #$30         ;assume all registers 8-bit
.rb
pea #U_GETKEY    ;push function address
cop #$00         ;call
sta $01,s        ;save A to stacked LSB of function address
php              ;save status register
pla              ;pop it to accumulator
sta $02,s        ;store it to stacked MSB as above
pla              ;pop accumulator contents
plp              ;pop status register
...
```

If preserving the CPU flag register is not very crucial, the code above may be somewhat simplified:

```
.xref PUTC

sep #$20         ;assume 8-bit accumulator
.ab
pea #PUTC
cop #$00
sta $02,s        ;save A to stacked MSB of function address
pla              ;pop LSB to accumulator
pla              ;pop accumulator contents
...
```

⁸⁾ All code examples in this chapter use ELSA's syntax, ELSA being the only assembler (at the time of writing this) capable of generating SpartaDOS X binaries with 65C816 extensions.

The COP #00 call is relatively costly, because it must carefully save all the context, including the processor status and the stack pointer (which might be lost when the CPU gets switched into the emulation mode), and later restore all that.

Therefore sometimes it may be necessary to use custom gateway code to avoid performance penalties. **However the COP #00 should always be considered the regular way of calling the legacy functions as incautious use of custom call gateways may cause stack corruption.**

Also, if the stack was relocated, a legacy call done in this manner will preserve it, but the emulation mode stack will get corrupted. Therefore it is best to assume that, once a program switched to the native mode, there is no return to the 6502 mode; and when the program is about to return to DOS, it should do this:

```
dosvec = $0a
    pea (dosvec)
    cop #000
```

or, if the zero page was relocated, this:

```
dosvec = $0a
    rep #$20
    lda !dosvec
    pha
    cop #000
```

or, when DBR is not 0, even this:

```
dosvec = $0a
    rep #$20
    lda >dosvec
    pha
    cop #000
```

rather than RTS. The COP #00 call here will reset the SP and DP to their default values (\$01FF and \$0000, respectively), then return the control to the DOS.

19.1.1.2 Stack-indirect calls

This method is especially useful to resident programs, because it allows to avoid base 64k RAM allocation for the gateway code, being at the same time much faster than while using the COP. There is also a drawback: *everything must be done on the default, 6502-compatible stack*, or, when the application maintains its own stack, it is fully responsible for switching from one stack to the other and back.

The 65816.SYS module defines 4 locations within the first 64k for use in such stack-indirect gateways:

```
.xref _65816
```

```

_toemu = _65816+0      ;switch to emulation, jump to address in first 64k
_toemu_c = _65816+2    ;switch to emulation, preserve C, jump as above
_tonat = _65816+5      ;switch to native, jump within 16 MB
_tonat_c = _65816+8    ;switch to native, preserve C, jump as above

```

The `_toemu` and `_tonat` procedures switch the CPU to the emulation and to the native mode, respectively. What they are doing next, is to jump to the stacked address: the former one uses a 16-bit address to jump to a location within the first 64k, the latter one uses a 24-bit address to jump to a location anywhere in the memory. The „jump” instructions used are RTS and RTL, respectively.

The „_c” versions do the same with the preservation of the C flag, which is normally lost during the mode switch.

A call consists of stacking several addresses in the proper order, then doing a long jump (JML) to the location labelled as `_toemu`, for example:

```

.xref FCLOSE

_fclose
    pea #_tonat-1      ;target function return point
    pea #FCLOSE-1      ;target function entry point
    pea #$0000         ;optional: reset DBR, if not $00
    plb                ;optional: reset DBR
    plb                ;optional: reset DBR
    rep #20            ;optional: reset DP, if not $0000
    .aw                ;optional: reset DP
    lda #$0000         ;optional: reset DP
    tcd                ;optional: reset DP
    sec                ;required (the next instr. is XCE)
    jml _toemu

```

There can of course be variances depending on current requirements, i.e. whether the DBR/D need to be reset or not, whether the registers must be saved or not, or which ones if so etc. All in all, only the first two and the last two instructions must always be present (or, to be precise, C must be set to 1 before the JML by any means).

The subroutine – for this is a subroutine – shown above must always be called using the JSL instruction:

```
jsl _fclose
```

or, if the relocation rules do not allow a far jump, using the combination of PHK and JSR:

```
phk
jsr _fclose
```

It should be kept in mind that this call resets all register sizes to 8-bit and all their previous contents are lost, so all this must be saved and restored if necessary.

19.1.1.3 Custom jump tables

Application programs can of course simplify all that by setting up a jump table in the low RAM and making direct far calls to it. This is especially recommended when speed is the concern:

```
.xref FCLOSE

_fclose                ;in RAM below $0100000
sec                   ;call with jsr _fclose
xce
jsr FCLOSE
clc
xce
rtl
```

As above, this method is best if the stack was not relocated from the 6502-compatible area (\$0100-\$01FF) – otherwise the stack pointer must be preserved across the switch to the emulation code and restored afterwards to avoid stack corruption.

19.1.2 Symbol remarks (S_LOOKUP/S_ADD)

The symbols can point to any place within the 24-bit address space. When the two topmost bits of the **program level index byte** in a symbol are both set (%11), or, in other words, the program level index value is equal to or greater than 192, the target address is considered 24-bit.

As said in the respective chapter, this is of little significance to a programmer, because symbol references are usually resolved by the system automatically in a transparent manner. The exception occurs when functions like S_LOOKUP, S_NEXT or S_ADD are called to explicitly manipulate the symbol database.

Accordingly, if after a call to S_LOOKUP the EXTENDED byte is equal to \$03, then the address returned is 24-bit with its highest significant byte at SYMBOL+\$0D. And S_ADD will take this byte into account in this case (i.e. when EXTENDED = \$03) while adding a symbol to the database.

The 65816.SYS driver loads the entire symbol list to the 65C816 High RAM. On machines where this RAM is accelerated this has positive influence on the speed with which the system loader (U_LOAD) executes the relocatable binaries.

19.1.2.1 Symbol manipulation (H_SFIRST/H_SNEXT/H_SADD)

The 65816.SYS module provides three functions which are equivalent to the 8-bit S_LOOKUP, S_NEXT and S_ADD: these are H_SFIRST, H_SNEXT and H_SADD, respectively.

All these must be called in the native mode with a JSL instruction, and the register sizes are preserved across the call. Otherwise the semantics is almost identical: for example, the operation of H_SADD is sufficiently described by the synopsis of S_ADD. (chapter 16.3). As for H_SFIRST and H_SNEXT it is enough to say that the memory index associated with the symbol is returned by them not in EXTENDED, but in SYMBOL+\$0A. The value returned is multiplied by 64.

19.1.3 Binary file format remarks (U_LOAD)

This section is describing the general extensions to the binary format and to the operation of the standard U_LOAD routine which are applied when the 65816.SYS module is loaded. The EXT816.SYS module provides further extensions, which will be explained in the relevant chapter below.

19.1.3.1 Headers

The position-independent binary blocks in presence of the 65816.SYS driver can be loaded directly to the 65C816 High RAM and run there. The decision if they should be loaded there is done at assembly time by declaring the memory index \$x3 for them (the higher nibble is explained below).

This also means that the **control byte** in the binary headers will carry this information. The control byte's high nibble also has few more bits defined:

- bit 7 = 1 – BSS block (as in the standard loader)
- bit 6 = 1 – force alignment to a 256-byte boundary (as in the standard loader)
- bit 5 = 1 – force alignment to a 64k boundary (bit 6 is ignored then)
- bit 4 = 1 – load to TSR memory, if available.

These correspond to the X parameter of the MALLOC call (see chapter 19.1.5.2 below). Example directives and their meaning:

```
.rel $00      ;conventional RAM code block
.rel $02      ;system RAM code block
.rel $03      ;65C816 flat RAM code block
.rel $04      ;user ext RAM code block
.rel $80      ;conventional RAM BSS block
.rel $82      ;system RAM BSS block
.rel $83      ;65C816 flat RAM BSS block
.rel $84      ;user ext RAM BSS block
```

As said above, adding \$40 enables alignment to a 256-byte boundary.

Adding \$20 enables alignment to a 64k boundary – this attribute is only valid for the memory index \$x3, i.e. the 65C816 flat RAM; in this case the bit 6 is ignored and it should be kept 0 for upward compatibility.

Adding \$10 selects the special TSR memory area; bit 5 is ignored in this case and should be kept 0 for upward compatibility.

The „TSR memory” is a special area, of size about 60-64k depending on the exact underlying memory layout, reserved by the 65816.SYS driver for resident programs at the top of the physical memory. Thanks to that these do not hog the low part of the High RAM (addresses \$010000 and on), most exposed to experiments and hacking around.

19.1.3.2 The relocation scheme

The same relocation scheme is being used as described in chapter 2, and it mostly works as it could be expected, with few less or more obvious limitations.

The limitation which is probably not very obvious, is this: the references which use long absolute addressing modes may not be made to the same program block which contains the referring instruction. In other words, references **internal** to a program block loaded to the High RAM may be 16-bit only.

Or, to put it in a yet simpler way: the near pointers (16-bit addresses) are to be used within a block of a program, whereas the far pointers (24-bit addresses) are to be used to reference things which are outside a program's block.

Technically, while the loading and fixing-up a program is performed, the following rules apply:

1) a program block cannot be larger than 65535 (sic!) bytes – this is the upper limit which cannot be exceeded; a program may contain up to 128 blocks; this results in a practical limit of $128 \times 64k = 8 \text{ MB}$ in program's size;

2) a program block is guaranteed by the system to be loaded completely within a 64k-segment space (so that the code does not span a 64k boundary), not necessarily at its beginning, though – the programmer must use alignment bits in the block header to force that, if necessary;

3) as said already, code references which refer to the block itself will use 16-bit (two-byte) fix-ups, exactly as on 6502; this means that the internal references, which use the long absolute addressing modes **and** are made within a block loaded to the High RAM, **would not be resolved properly** – the internal references within a block may only be done using the short absolute addressing modes;

4) code references to any blocks loaded to the memory segment 0 (first 64k, 6502 compatible) made from the outside of that segment will use 16-bit fix-ups as well; this means that data accesses to the first 64k may be done using both long and short absolute addressing modes – the latter ones will only work when the Data Bank Register (DBR or B) has been reset by the programmer to 0 while the access is being performed;

5) external code references between any two blocks, neither of which has been loaded to the segment 0, are resolved using 24-bit (three-byte) fix-ups; this means that these references **must always** be done using long addressing modes, no matter where you might think both blocks have been physically loaded to;

6) **references to symbols** are resolved according to the symbol definition: „old” symbols pointing to the segment 0 will cause a 16-bit fix-up to be used on the referring word, whereas „new” symbols defined to point to outside the segment 0 will cause a 24-bit fix-up to be used on the referring long word; this means that symbol references are backward-compatible, but not upward-compatible: references to all symbols may be made using long absolute addressing modes, but the old symbols, residing within the 6502-compatible, 16-bit address space, cannot be safely replaced with new ones residing in 24-bit address space, because old programs (including SpartaDOS X itself) use short addressing modes to reference them;

7) a program is executed using the *runad* \$02E0 OS vector, and this vector is 16-bit: therefore any program which loads blocks of code to the 65C816 High RAM must also load at least one block of code to the conventional memory (first 64k), and that block will be handed the control to by the system upon execution of the program; the code contained in that block may then call the code loaded to the high RAM using the long subroutine call (JSL) or by means equivalent; *the CPU must be switched into the native mode beforehand, then after the return, just before exiting to the DOS, the emulation mode must be restored together with its context, especially the DBR/DP/SP registers must be reset to defaults.*

The H_LOAD function implemented by the EXT816.SYS module overcomes the limitations mentioned in the paragraph 7 above.

Therefore, while writing a program it may be advisable to obey the following rules:

- 1) while writing code to be loaded to the high RAM, do not split that code into multiple blocks; or, if you do, only use long absolute addressing modes to make calls and other references between these blocks;
- 2) go allocate another code block only when you exhaust 65535 bytes of the current one; never use short absolute addressing modes to make references from one block to another (your assembler should prevent that anyway);
- 3) do not use .rel \$83 or .rel \$A3 blocks, unless you want to access them using long absolute addressing modes only.

19.1.3.3 Extended relocatable binary block

To facilitate using the high memory, and especially to facilitate declaring uninitialized variables in it (BSS areas), a new type of relocatable block has been defined. In principle, it is a normal position independent binary block, in which the amount of bytes to allocate has been separated from the amount of bytes to load:

```
$00-$01:  signature $FFF9
$02:      block number
$03:      control byte, $00 + index of the memory to be allocated
$04-$05:  amount of bytes to load
$06-$07:  amount of bytes to allocate
```

While processing such a header, the loader will request from the memory manager the amount of bytes indicated by the bytes \$06-\$07; then, if this succeeded, it will load to it the number of bytes indicated by the bytes \$04-\$05.

The former value (bytes to allocate) must of course be greater than or equal to the latter (bytes to load). If the former is greater, the remaining space may be used for program's variables. And note that in this situation these variables not only may, but even **must** be referenced using the short absolute addressing modes from the code, whose size is indicated by bytes \$04-\$05 of the header, because all this is one program block.

The offset value (described in sections 2.2, 2.3, 2.6) for this block type is internally assumed \$0000 by the loader.

This block type is selected automatically, when a `.rel $x3` directive is used to generate a block of code and at least one `.ds` directive follows, for example:

```

        .rel $03

        .rb
foo      ldx  #$0f
?lp      stz  buf,x
        dex
        bpl  ?lp
        rtl

buf      .ds  16

```

In this case the allocated block size will be 25 bytes, the code size loaded being 9 bytes, leaving 16 bytes for the BSS variables.

19.1.4 Error handling

19.1.4.1 Setting a trap in 24-bit address space (H_SFALL)

The `U_SFALL` function described above accepts a two-byte address as an argument, therefore it can only work in the 6502 16-bit address space: on error, the control can only be returned to code located somewhere between `$0000` and `$FFFF` and no beyond.

To enable error handlers to reside anywhere in the 24-bit address space of the 65C816 processor, the 65816.SYS driver defines the `H_SFALL` function. Its operation is identical as in the case of `U_SFALL`, except that the argument it accepts is 24-bit: the low word of the handler's address in the accumulator, and its topmost byte in the X register. *The call must be done in the native mode and the register sizes are preserved across the function.* Example:

```

.xref H_SFALL

rep  #$20      ;set accumulator to 16-bit
.aw
sep  #$10      ;set X/Y to 8-bit
.ib
lda  #error    ;address of error handler
phk          ;handler is in the same program segment
plx          ;as the call
jsl  H_SFALL
...           ;proceed with your program

```

Cancelling an unused trap can, as usual, be accomplished by a call to `U_XFAIL`:

```

.xref U_XFAIL

pea  #U_XFAIL
cop  #$000
pla
pla

```

or to `H_XFAIL` as shown below.

19.1.4.2 Removing a trap (H_XFAIL)

H_XFAIL is functionally identical to U_XFAIL, as described in section 4.3. The only difference is that H_XFAIL can be called directly using the JSL instruction, in this manner:

```
.xref H_XFAIL
jsl H_XFAIL
```

This method is recommended for the native mode programs.

19.1.4.3 Standard error handler (H_FAIL)

The H_FAIL functionality is identical to the U_FAIL described in section 4.1. The only difference is that H_FAIL can be called directly using the JML instruction, in this manner:

```
.xref H_FAIL
jml H_FAIL
```

As with the U_FAIL, this call terminates the program and transfers the control to the error handler last installed with U_SFALL or H_SFALL.

19.1.5 Memory management

19.1.5.1 65C816 High RAM

As of SpartaDOS X 4.46 there is some support for computers equipped with the 65C816 and additional memory, the so called High RAM or linear RAM (which is short for any RAM available in the address space from \$010000 to \$FFFFFF). COMTAB2 contains the following related values:

COMTAB2+\$00 (_816FLG) – an \$FF here means that CPU is 65C816 (or 65C802). An \$FE – that the OS ROM contains native interrupt services, and therefore it is safe to switch the CPU into the native mode. An \$FC – that the kernel is able to perform I/O on the high RAM. If the CPU is 6502, there is a \$00 here, or \$02 if 65C02.

COMTAB2-\$04 (_SEGCNT) – a count of *additional* 64 KiB segments available past the first 64 KiB. For a full 16 MiB (256 segments) this would be \$FF.⁹ If only the segment 0 is available (i.e. the 6502-compatible memory), this byte contains a \$00.

COMTAB2-\$05 (_SEGBEG) – a number (or, in other words, the most significant byte of the address) of the first additional segment available in the linear memory. If there are no such segments, there is a \$00 here.

SpartaDOS X normally does not use this type of memory nor contains functions to manage it. By default it only makes available the information above. This situation changes when the 65816.SYS driver's version at least 3.0 is loaded. Among other things, it provides extension to the memory management functions so that the high RAM becomes available to the system in the standardized way.

⁹ In theory. In practice the last RAM segment, i.e. from \$FF0000 to \$FFFFFF, is reserved for future I/O devices, so that the DOS will stop the RAM sizing at most at the segment \$FE, seeing 16320 instead of 16384 KB RAM, even if this much memory is physically accessible.

19.1.5.2 High RAM allocation: memory index \$03 (MALLOC)

Loading the 65816.SYS driver assigns memory index \$03 to the high RAM. This is both the index for the requested memory and for the granted one. If the high RAM does not exist (or the driver was not loaded), a MALLOC call referencing that memory index will return an error.

Because of the nature of that RAM and some other requirements, a MALLOC call will expect slightly different parameters than it does in the case of the conventional 64k and the banked extensions.

Note: MALLOC is a legacy function, the calls must always be done in emulation mode.

So, for the index \$03, MALLOC requires the following parameters:

- the Y register must be zeroed;
- the X register must contain \$x3;
- *bytbuf* (\$0785/6) must contain the *low word* of the requested amount of bytes;
- the Accumulator must contain the highest byte of the 24-bit requested amount of bytes;
- *scan/attr* (\$0779/\$077A) must contain a 16-bit word carrying additional attributes (\$0000 is best for normal applications);

Valid X register values are currently as follows:

\$03 – allocate application memory
 \$43 – allocate application memory, page-aligned
 \$23 – allocate application memory, 64k-aligned
 \$13 – allocate TSR memory.

The „TSR memory” is explained above (section 19.1.3.1).

MALLOC returns with a positive value on success (N=0), *addpos* (\$0782/3/4) will then contain the address of the allocated block, and the X register – the index of the allocated memory (\$03). On an out of memory condition the result is negative (N=1).

The allocator in its current implementation can only manage up to 128 memory blocks of any size (the „TSR memory” is exempt from this limit, however). Reaching this limit will cause all further allocation attempts to fail regardless of how much memory is physically free. It is therefore advisable, while writing a program, to allocate the memory in portions rather larger than smaller.

The memory allocated so will be released when the program, which called MALLOC, terminates (unless the system is told otherwise via the symbol INSTALL).

19.1.5.3 Checking how much free RAM is available (MALLOC)

Passing all the parameters as for allocation, with the *amount* parameter equal to \$FFFFFF (= -1) will cause MALLOC to return the total amount of free high RAM in *addpos* (\$0782/3/4). Analogically, passing \$FFFFFFE (= -2) will return the size of the largest free

block – which for practical purposes may be much more useful information than the other one.

In both cases no actual allocation takes place.

19.1.5.4 Freeing the allocated blocks explicitly (MALLOC)

As said above, the blocks allocated by a program are automatically deallocated when the program owning them terminates. But, unlike it is with the conventional RAM and the banked extensions, the high RAM blocks can also be deallocated in an explicit manner (and in random order). The deallocation of a block is also accomplished by a call to the MALLOC function, with the parameters as follows:

- the Y register must be zeroed;
- the X register must contain \$83;
- *bytbuf* (\$0785/6), Accumulator and *scan/attr* (\$0779/\$077A) are do-not-cares;
- *addpos* (\$0782/3/4) must contain the address of a block to be deallocated. This is the value which had been previously returned by a MALLOC call which allocated that block.

Notice that this type of the memory management, allowing you to freely allocate blocks of requested size and deallocate any of them at any time, is by its nature prone to cause the phenomenon known as „memory fragmentation“. Therefore it is advisable, if possible, to allocate the memory blocks so that the probability of creating „holes“ between them could be minimized.

19.1.5.5 Wrappers for MALLOC (H_MALLOC, H_MFREE)

To make the memory management calls less painful the 65816.SYS driver provides two wrappers which can be called from the native mode.

1) H_MALLOC to allocate the High RAM. It accepts the following arguments:

- the 32-bit long word in AX registers specifies the amount of bytes to allocate;
- the Y register contains the alignment parameter in the bits 0-3, other bits being 0;
- *scan/attr* (\$0779/\$077A) contain a 16-bit word with additional attributes (\$0000 is best for normal applications);

The *alignment parameter* is equal to the high nibble of what is passed to MALLOC in the X register: 4 for page alignment, 2 for 64k-alignment, 1 for TSR RAM. The actual memory index is always assumed \$x3. Thus to allocate 1 MB of High RAM you can do this:

```
.xref H_MALLOC

.rw
lda #1048576&$0000FFFF      ;low word
ldx #1048576>>16           ;high word
ldy #$0002                  ;64k-alignment
jsl H_MALLOC
...
```

If N=0 upon return from the call, the 32-bit long word in AX contains the result as expected from the MALLOC call (i.e. the address of an allocated block or the amount of free memory, as described in the previous sections). Otherwise N=1 indicates an error condition.

2) H_MFREE to release what had been allocated with H_MALLOC. The argument is a 32-bit long word specifying the address of a block to release. For example, if that address, returned by H_MALLOC, was stored at *adr*, the call may look like this:

```
.xref H_MFREE

.rw
lda adr
ldx adr+2
jsl H_MFREE
```

As above, N=0 means success, N=1 indicates an error condition.

19.1.5.6 High RAM access

The high RAM is directly available all the time, so the use of *V_setme* and *V_popme* calls, described in the previous sections, is not required; although calling *V_setme* with A=\$03 does no harm, except for imposing some performance penalty (and when done it must be, as always, paired by the subsequent *V_popme* call).

For further technical details – addressing modes, segmentation registers, and the relevant instructions – please refer to the 65C816 CPU documentation.

19.1.5.7 XRWMEM

The legacy function XRWMEM, described in chapter 3.7.2, accepts 24-bit addresses when memory index \$03 is specified. To peek into the address \$ABCDEF do this:

```
addpos = $0782
```

```
.xref XRWMEM

rep #$21                ;this "contains" CLC
.aw
lda #$abcdef&$00ffff
sta addpos
ldx #^$abcdef
stx addpos+2
ldy #$00
ldx #$03
pea #XRWMEM
cop #$00
plx
plx
bcs error_no_access
...
```

In practice this is useful only for legacy 6502 programs wishing to peek or poke the High RAM without using the extended addressing modes, because 65C816 can access such an address directly with just one instruction.

19.1.5.8 Copying RAM blocks (H_MEMCPY)

The 65816.SYS driver defines the symbol H_MEMCPY, which points to a function copying memory blocks of arbitrary size around the full, 24-bit address space of the 65C816 processor.

H_MEMCPY accepts three 32-bit (sic!) words as arguments: the source address, the destination address, and the byte count. The topmost significant bytes of all these values must be zero, making them effectively 24-bit words. *32-bit words are used to facilitate internal pointer arithmetic.*

The arguments must be passed via the stack, and be pushed in this order: source, destination, byte count. After that, a call must be made using JSL. *The call must be done in the native mode. The DP and DBR register contents and the AXY register sizes are preserved across the function.*

Upon return it is not necessary to pull the arguments off the stack or correct the stack pointer, because the H_MEMCPY function cleans this all up upon exit.

*Contrary to what the function's name may suggest, as of 65816.SYS v. 3.2, the blocks to be copied **may** overlap and the memory contents will not be destroyed.*

For the following example, let us assume that the source address, the destination address and the amount of bytes to copy are stored in 32-bit variables on the zero page, named *src_ad*, *dst_ad* and *blk_sz*, respectively.

Also assume that the CPU is already in the native mode and all the registers are set to be 16-bit wide.

```

        .xref H_MEMCPY           ;define symbol reference

        .zp $80                 ;define zero-page variables
src_ad  .ds 4                   ;source address
dst_ad  .ds 4                   ;destination address
blk_sz  .ds 4                   ;block size
        .code

        pea (src_ad+2)          ;parameters assumed on zero-page
        pea (src_ad)
        pea (dst_ad+2)
        pea (dst_ad)
        pea (blk_sz+2)
        pea (blk_sz)
        jsr H_MEMCPY            ;call
        ...                     ;proceed with your program

```

The code does not check if the memory block to be written to is spanning the 16 MB boundary.

19.1.5.9 Filling RAM block with a constant (H_MEMSET)

H_MEMSET is somewhat similar to H_MEMCPY, except that it does not copy the memory contents, but rather fills it with the given constant value. Again, blocks of any given size and located anywhere in the 24-bit address space can be processed.

The arguments must be passed via the stack, and be pushed in this order: block address, block size, fill pattern. Out of these, the two former ones are 32-bit words (with topmost bytes zeroed), and the fill pattern is a 16-bit word (the MSB is ignored).

After pushing these arguments onto the stack a call must be made using JSL. *The call must be done in the native mode. The DP and DBR register contents and the AXY register sizes are preserved across the function.*

Upon return it is not necessary to pull the arguments off the stack or correct the stack pointer, because the H_MEMSET function cleans this all up upon exit.

For the following example, let us assume that the block address, the block size and the fill pattern are stored in variables on the zero page, named *blk_ad*, *blk_sz* and *fill_p*, respectively.

Also assume that the CPU is already in the native mode and all the registers are set to be 16-bit wide.

```

        .xref H_MEMSET           ;define symbol reference

        .zp $80                 ;define zero-page variables
blk_ad  .ds 4                   ;block address
blk_sz  .ds 4                   ;block size
fill_p  .ds 2                   ;fill pattern
        .code

        pea (blk_ad+2)           ;parameters assumed on zero-page
        pea (blk_ad)
        pea (blk_sz+2)
        pea (blk_sz)
        pea (fill_p)
        js1 H_MEMSET             ;call
        ...                     ;proceed with your program

```

The code does not check if the memory block to be written to is spanning the 16 MB boundary, nor it returns anything.

19.1.5.10 Initializing TSR programs after a RESET (H_SADDIZ)

H_SADDIZ works in a similar way to the S_ADDIZ function described in the chapter 15.1. The differences are as follows:

- a) H_SADDIZ expects a 24-bit address in AX registers (A – low word, X – high byte);
- b) if the high byte of that address is not a zero, the routine being registered will, at reset time, be called in the native mode and should end with an RTL instruction; and, when that

byte is zero, the routine will be called in the emulation mode and should end with an RTS instruction;

c) prior to the call the DBR register will be set to the segment number where the respective TSR program is residing (or, in other words, to the high byte of the destination address).

The call must be done in the native mode, the register sizes are preserved across the function.

19.2 The ENV816.SYS module (H_GETENV/H_PUTENV/H_NUMENV)

The ENV816.SYS module provides extended environment support. First, it hooks into the legacy functions GETENV, PUTENV and NUMENV, to provide faster access to the environment space to old programs in a transparent manner.

Second, it defines three new functions which, unlike those, are designed to be called in the native 65C816 mode and to work across the entire 24-bit address space. These new functions also define new limits for the environment: there can be maximum up to 65536 variables, 32767 bytes each (plus an EOL at the end). The old limits – maximum 256 variables, 127 bytes each (plus EOL) – are still in effect for the legacy calls.

19.2.1.1 H_GETENV

This function finds the specified variable in the environment, then puts its value to the specified buffer (see GETENV, chapter 7.2). The function accepts three arguments on the stack, quite in a manner similar to H_MEMCPY, in this order of pushing:

- a) a 16-bit word containing the maximum size of the result buffer (must be > 0).
- b) a 32-bit pointer to a RAM buffer where the result will be written to;
- c) a 32-bit pointer to the variable's name. The name must be EOL-terminated.

The pointers are 32-bit to facilitate internal pointer arithmetics. The topmost bytes of both must be zero.

```
.xref H_GETENV

rep # $20          ;assume 16-bit accumulator
.aw
pea #BUF_SIZE      ;buffer size
lda bufp+2         ;output buffer pointer
pha
lda bufp
pha
lda varp+2         ;variable name pointer
pha
lda varp
pha
jsl H_GETENV
...
```

The function returns A=\$0000 and N=0 on success, or A=\$FFFF and N=1 otherwise, i.e. when the requested variable is not found or it does not fit in the buffer specified.

The call must be done in the native mode, the register sizes are preserved across the function.

19.2.1.2 H_PUTENV

This function gets the variable from the memory buffer and puts it into the environment buffer. Any variable with the same name, previously living there, gets deleted. When the user specifies a name alone, the specified variable is deleted from the buffer and no further action is taken (see PUTENV, chapter 7.4).

The function accepts one argument on the stack: the (32-bit) address of the variable to be set.

```
.xref H_PUTENV

rep #200                ;assume 16-bit accumulator
.aw
lda varp+2              ;variable name pointer
pha
lda varp
pha
jsl H_PUTENV
...
```

The function returns A=\$0000 and N=0 on success, or A=\$FFFF and N=1 otherwise (e.g. no more place in the environment buffer).

The call must be done in the native mode, the register sizes are preserved across the function.

19.2.1.3 H_NUMENV

This function allows to fetch a variable by its number (see NUMENV, chapter 7.3. It is most often used to enumerate all the variables set in the system, as in the Command Processor's command SET.

The function accepts three arguments in this order of pushing:

- a) a 16-bit word containing the maximum size of the result buffer (must be > 0).
- b) a 32-bit pointer to a RAM buffer where the result will be written to;
- c) a 16-bit word specifying the variable's number.

```
.xref H_NUMENV

rep #200                ;assume 16-bit accumulator
.aw
pea #BUF_SIZE           ;buffer size
lda bufp+2              ;output buffer pointer
pha
lda bufp
pha
pea #VAR_NUMBER         ;variable number
jsl H_NUMENV
...
```

The function returns A=\$0000 and N=0 on success, or A=\$FFFF and N=1 otherwise (e.g. no such variable in the buffer).

The call must be done in the native mode, the register sizes are preserved across the function.

19.3 The SIO816.SYS module (H_SIOADD)

The SIO816.SYS module replaces the generic SIO.SYS: it contains a port of the SpartaDOS Sector I/O communication routines which handle the communication with serial floppy disk drives, printers, RS-232 interfaces etc. as well as with parallel hard disks. Its use is mostly transparent to the user and the program is backwards compatible with the SIO.SYS.

The SIO816.SYS module contains one extra function: it allows to register extra SIO drivers which are subordinate to itself rather than to the original SIO subsystem. The gain here is that these drivers do not have to raise the *memlo* pointer to reserve the low memory for a switch to the native mode and a long call to the high RAM (because they are invoked in the native mode and directly from the high RAM itself).

To register a driver, you have to load a 24-bit address of its main communication routine's entry point (see chapter 22), then make a call to H_SIOADD:

```
.xref H_SIOADD

.aw
.ib
lda sioptr
ldx sioptr+2
jsl H_SIOADD
bcs error
...
```

```
sioptr .long do_sio
```

The function returns C=1 when no more drivers can be registered. The call must of course be done in the native mode, the register sizes get preserved across the call.

A driver registered so will get invoked using a JSL instruction, in the native mode, with all registers set to 8-bit, and DBR equal to PBR. It should return using RTL. Your driver will be called before PBI and thus also before the serial routines. Also, out of all the subordinate drivers installed, the one installed most recently will be called first (i.e. it has the highest priority out of all of them).

Besides, most rules apply, which are specified in chapter 22 (and especially 22.4), with the following exceptions:

a) the logical-to-physical drive number translation (also known as „swap“, see chapter 22.4.1) is already done inside SIO816.SYS, so your subordinate driver does not have to trouble itself with this, nor with restoring that upon return – just use the value already stored in *dunit* \$000301.

b) the I/O memory (see chapter 22.4.3) is likewise already enabled, so no need to enable it again or restore afterwards.

19.4 The EXT816.SYS module

Unlike 65816.SYS or SIO816.SYS, the EXT816.SYS module is not a system driver: it does not occupy any extra bit of the 6502-compatible RAM. It just defines a number of symbols pointing to functions that may be useful to application programs: most of them are 16-bit replacement functions for the old SpartaDOS X cartridge library. There are no special requirements regarding its usage (except that 65816.SYS and ENV816.SYS must be loaded before), it can be loaded at any time, although it is advisable to put it somewhere in your CONFIG.SYS, and as early as possible, because other 65C816-related modules may already want to use its functions.

19.4.1 Binary loader (H_LOAD)

The H_LOAD function is a replacement for the standard U_LOAD, with the following extensions:

a) everything described in chapter 19.1.3 above – in fact, H_LOAD uses the loading procedure contained in 65816.SYS, so all the remarks which apply to that, also apply here;

b) programs may be not only loaded, but also executed directly from the High RAM, without intermediate blocks loaded to the first 64k – if the first binary block of a program gets loaded to the High RAM, the program will get executed at its beginning, in the native mode, with AXY registers set to 8-bit, the DBR register equal to the PBR register (or B=K), and DP and SP as set by the caller;

c) if the first binary block of a program is loaded to the first 64k, it will get executed in the emulation mode (for backward compatibility);

d) the user symbols which begin with the „@“ character can also be defined directly in the High RAM – invoking such a symbol defined by a program being held in memory using the LOAD command may be done directly from the Command Processor prompt – the rules are as with the programs actually loaded and executed: if the symbol points to the first 64k, it will be executed in the emulation mode; otherwise, it will get executed in the native mode, with AXY registers 8-bit, the DBR equal to PBR, and DP and SP as passed from the caller;

e) unlike U_LOAD, which only allows chaining and overlaying programs running in the first 64k and in the emulation mode, H_LOAD allows it for the programs running outside the first 64k and in the native mode. As hinted in the chapter 19.1.1.1, a program loaded and executed using U_LOAD, after the transition to the native mode, can only be terminated by a call to the address pointed to by the DOSVEC vector (\$0a). The same applies to any of its „child“ programs (or overlays) running in the native mode, because there is no reliable method of returning from the native mode code segment to the U_LOAD loader. But terminating an overlay with a call through the DOSVEC vector terminates the overlay's parent program as well – and that is a serious limitation. Now H_LOAD allows to overcome this limitation: an overlay loaded directly to the High RAM and executed from there may just end with an RTL instruction and this will cleanly resume execution of its parent program.

The H_LOAD hooks up into the system and replaces the standard U_LOAD function once the EXT816.SYS module is loaded – this is why it does not allow loading native mode program segments to the first 64k: it is done so for backward compatibility, so that the usual SpartaDOS X applications and utilities (such as COMMAND.COM for example) still work.

19.4.2 Opening files (H_FOPEN, H_FILE_P)

The H_FOPEN function is an equivalent to FOPEN and both work seemingly in the same way. Hence the remarks about the latter to be found in chapter 8.1 still apply.

Additionally H_FOPEN features an extension: it uses the word pointed to by symbol H_FILE_P as the most significant portion of the regular FILE_P pointer. By default its value is \$0000, but changing that enables your program to store the filename for H_FOPEN anywhere in the memory, not just in the first 64k.

A word value is used to facilitate pointer arithmetic using 16-bit registers. The low byte of this word serves as the most significant byte of a 24-bit address; the high byte is ignored and should be kept \$00 for upward compatibility.

Example code – open a file named „FOO.BAR” from the current directory:

```
.xref FILE_P
.xref H_FILE_P
.xref H_FOPEN

fname .byte "FOO.BAR", $9b

        .rb
open_foo_bar
        lda #$04                ;for reads
        sta >mode               ;assume we're outside first 64k
        lda #$00                ;all attributes
        sta >scan
        lda #$00                ;push bits 24-31 of H_FILE_P
        pha
        phk                     ;push bits 16-23 of H_FILE_P
        rep #$20
        .aw
        lda #fname              ;set low word of filename pointer
        sta >FILE_P
        pla
        sta >H_FILE_P           ;set high word if filename pointer
        sep #$20
        .ab
        jsr >H_FOPEN
        rts
```

H_FILE_P is taken into account by all calls that also use FILE_P: H_GEFINA, H_GEPATH, and in turn by everything that uses these, such as H_FOPEN. Consult the table in section 19.5 to see which calls take H_FILE_P into account.

19.4.3 Accessing files

19.4.3.1 Using FREAD/FWRITE on the High RAM

In the presence of the 65816.SYS driver it is possible to read data from a file to the 65C816 High RAM (located past the address \$00FFFF) using FREAD, and write data from there to a file using FWRITE.

The parameters, besides the usual handle in *file_id* (\$0760), are as follows:

- *addpos* (\$0782-\$0784): 24-bit buffer address
- *bytbuf* (\$0785-\$0786): buffer size (65535 maximum)
- *memreix* (\$0787): a value of \$03

Both are legacy calls, therefore must be called in the emulation mode or via COP #\$00.

Upon return from FREAD, *bytbuf* will contain the number of bytes actually read.

The availability of this extension may be tested in a program by checking *_816FLG* (COMT-AB2+\$00): if its bit 7 is set, and bit 1 cleared, the kernel is able to perform this operation. Another method is to use *MALLOC* with the memory index \$03 to allocate the memory area to be read or written: the 65816.SYS driver guarantees, that if this memory can be allocated, it also can be accessed with FREAD/FWRITE.

An attempt to use this function without the said extension loaded will cause undefined results.

19.4.3.2 Large data transfers (H_FREAD/H_FWRITE)

As hinted above, the regular FREAD and FWRITE functions are somewhat limited being able to transfer only up to 65535 bytes per call. *This limitation is caused by the API's definition, which provides only 16 bits to specify the transfer size.*

To overcome this limitation, the EXT816 module defines two new functions to facilitate reading and writing blocks of data larger than the said 65535 bytes: *H_FREAD* and *H_FWRITE*, respectively.

The calls must be done in the native mode. The DP and DBR register contents and the AXY register sizes are preserved across the functions.

Both accept parameters in the same manner: the transfer size and the buffer address stored on the stack in this order as 32-bit values (with highest bytes zeroed). There are no special requirements to the registers' size at the time of the call, any will do.

Both *H_FREAD* and *H_FWRITE* give the value of *memreix* \$0787 special treatment, as follows:

- a) the memory index for the I/O operation requested will be \$03 (= 65C816 flat memory), if the buffer address specified is greater than \$00FFFF;
- b) otherwise, if the buffer address specified fits within the first 64k (= 6502-compatible memory), the said *memreix*'s value will be respected as passed by the caller.

*CAUTION: you **may not** specify parameters to both of these functions so that the buffer begins in the 6502-compatible memory and crosses the boundary over to the byte \$010000 and above – this will not work. The 64k boundaries may be crossed by the data blocks being read or written only if the buffer address is equal to or greater than \$010000.*

The values returned in the CPU registers and other features are the same as in the regular FREAD and FWRITE (see there); and the number of bytes actually read or written is returned as a 32-bit value on the top of the stack, it must be popped from there before proceeding (the buffer address is removed from the stack automatically by the function):

```

        .xref H_FREAD          ;define symbol reference

        .zp $80                ;define zero-page variables
blk_sz  .ds 4                  ;block size
blk_ad  .ds 4                  ;block address
        .code

        pea (blk_sz+2)         ;arguments assumed on zero-page
        pea (blk_sz)
        pea (buf_ad+2)
        pea (buf_ad)
        js1 H_FREAD            ;call
        rep #$30
        pla                    ;low word of transfer size
        plx                    ;high word of transfer size
        ...                    ;proceed with your program

```

As said above, the calling convention for H_FWRITE is identical.

19.4.3.3 Text records (H_FGETS/H_FPUTS)

These two are 24-bit counterparts to the regular FGETS and FPUTS. The operation is generally very similar, except that the H_ instances accept 32-bit buffer address in the AX registers (low/high word, respectively) and 16-bit record length in the Y register.

These calls must be done in the native mode. The DP and DBR register contents and the AX/Y register sizes are preserved across the functions.

The return values in A, X and Y are also 16-bit: A=\$FFFF means truncated record condition, X=\$FFFF means EOF, and Y carries the actual record length.

Attention should be paid to the fact that both functions restore the MX bits (i.e. the register sizes) to the state prior to the call, so that, when the function was called with 8-bit registers (which is perfectly possible), the values in the X/Y registers will be cut down to 8 bits upon return.

19.4.4 Formatted text output

The original PRINTF, FPRINTF and VPRINTF functions (see chapter 8.8, 9.3, 9.7) by their design cannot be called from the memory located past the first 64k and may be incompatible with the native mode, which may seriously impair their use in larger 65C816 programs. Therefore the EXT816 module provides four replacement functions described below.

These calls must be done in the native mode. The DP and DBR register contents and (unless noted otherwise) the XY register sizes and contents are preserved across the functions.

19.4.4.1 Output to the console (H_PRINTF)

This function is an almost exact clone of the regular PRINTF and its operation is nearly the same, with the following differences:

- a) the call must be done in the native mode using the JSL instruction;
- b) the formatting string and the 16-bit pointers to arguments are fetched from the program segment (pointed to by the K, or PBR register), but the arguments themselves are fetched from the data segment (pointed to by the B, or DBR register).

The latter means, that all arguments for a single call to H_PRINTF must be located in the same 64k data segment.

Example:

```
.xref H_PRINTF

jsl H_PRINTF
.byte $9b, "Hello world!", $9b, 0
```

In comparison to the regular PRINTF, there is also an additional formatting command: %f, which accepts and prints a floating point number in the Atari format. Its size may be controlled by normal means (e.g. %4f will set the resulting string length to 4 characters).

Upon exit the 16-bit A register contains the number of characters printed, except the terminating NUL.

19.4.4.2 Output to a file (H_FPRINTF)

This is the counterpart to the regular FPRINTF. The operation is identical as in H_PRINTF described above (see there), except that the resulting string gets written to an open file.

Upon exit the 16-bit A register contains the number of characters printed, except the terminating NUL.

19.4.4.3 Vectored output (H_VPRINTF, H_PUT_V)

As above, this is the counterpart of the regular VPRINTF, the same remarks apply as to H_PRINTF.

Upon exiting H_VPRINTF the 16-bit A register contains the number of characters printed, except the terminating NUL.

Note that the PUT_V vector, which is in use here, is 16-bit and therefore cannot in itself point outside the first 64k. To cure this problem the EXT816 module defines H_PUT_V, which – similarly to the H_FILE_P described above – constitutes the high significant word of the PUT_V. Both values – PUT_V and H_PUT_V will be used to setup a long vector. That vector in turn will be used to call your custom display routine.

The routine in question must meet the following requirements:

- a) the call will be made in the native mode;

- b) the SP points to the application's stack;
- c) the DBR contains the application's data segment number;
- d) all registers are 8-bit;
- e) the register contents does not have to be saved at the beginning and restored upon exit;
- f) the routine should end with RTL.

19.4.4.4 Output to a buffer (H_SPRNTF)

This function has no real counterpart in the standard SpartaDOS X Library, although it can be simulated using the V_PRINTF call (see chapter 18.5).

Whereas H_PRINTF produces character output to the console, H_SPRNTF writes them to a memory buffer; thus it also must accept more parameters. One is the address of the said buffer: it is a 16-bit word located before the formatting string, directly after the JSL call. The other is that buffer's index, expected in the accumulator. Example:

```
.xref H_SPRNTF

rep #20
.aw
lda #0000
jsl H_SPRNTF
.word _outbuf
.byte $9b, "Hello world!", $9b, 0
```

As it is with the other parameters, that buffer pointer will be fetched from the program segment, but the buffer itself will be expected in the data segment.

The sense of the „buffer index” parameter will become clear, when we learn that it is not only required, but also returned in the accumulator. The returned index is updated with the number of characters actually written to the buffer, so that another H_SPRNTF call may write new data to the buffer starting at the place where a previous call left it:

```
.xref H_SPRNTF

rep #20                ;assume 16-bit accumulator
.aw
lda #0000              ;initial index
jsl H_SPRNTF           ;returns updated index in A
.word _outbuf
.byte $9b, "Hello ", 0
jsl H_SPRNTF           ;continues writing where prev. call left
.word _outbuf
.byte "world!", $9b, 0
```

The buffer will then contain the complete string plus the terminating 0. *The said output buffer **may** span a 64k boundary and reach past the end of the current data segment, but in any case its size cannot exceed 64k (as the index being passed on in the accumulator is only 16-bit).*

Remark: the EOL character normally terminates a substring – therefore, to allow the contents of the output buffer to be passed on to subsequent calls (such as H_PRINTF) as a substring, all EOL characters are translated into the '\n' sequences.

19.4.5 Integer multiplication and division

Though the SpartaDOS X Library does contain routines which perform 32-bit integer multiplication and division (MUL_32 and DIV_32, see chapter 14.1), these have some limitations:

- a) they accept input and produce results in big-endian format, which requires byte- and word-swapping to and fro;
- b) the calculations are unsigned;
- c) the calculations are always 32-bit;
- d) the DIV_32 routine returns only the integer part of the quotient, losing the remainder;
- e) and finally, they are located in an external ROM (namely: the SpartaDOS X cartridge), which on accelerated machines makes them slow, because such ROMs are clocked at stock speed of 1.77 MHz.

Therefore the EXT816.SYS module provides eight routines which are more friendly.

These calls must be done in the native mode. The DP and DBR register contents and the AX/Y register sizes are preserved across the functions.

19.4.5.1 16-bit unsigned integer division (H_DIVU16)

This function accepts two 16-bit unsigned integers and returns a 16-bit unsigned quotient and 16-bit unsigned remainder. All arguments and results are passed via the stack:

```
.xref H_DIVU16

Int0    .zp $80          ;define zero-page variables
        .ds 2            ;dividend
Int1     .ds 2            ;divisor
Result  .ds 2            ;quotient
Rem      .ds 2            ;remainder
        .code

        rep #$20         ;assume 16-bit accumulator
        .aw
        pea (Int1)        ;divisor first
        pea (Int0)
        jsr H_DIVU16
        pla
        sta Result
        pla
        sta Rem
        bcs error        ;error if C=1 (e.g. division by 0)
        ...
```

19.4.5.2 16-bit signed integer division (H_DIVS16)

This is analogous to H_DIVU16, with the exception that the dividend, the divisor and the result are signed integers:

```

        .xref H_DIVS16

Int0    .zp $80                ;define zero-page variables
        .ds 2                  ;dividend
Int1    .ds 2                  ;divisor
Reslt   .ds 2                  ;quotient
Rem     .ds 2                  ;remainder
        .code

rep #20                ;assume 16-bit accumulator
        .aw
pea (Int1)              ;divisor first
pea (Int0)
jsl H_DIVS16
pla
sta Reslt
pla
sta Rem
bcs error                ;error if C=1 (e.g. division by 0)
...

```

19.4.5.3 16-bit unsigned integer multiplication (H_MULU16)

This function accepts two 16-bit unsigned integer factors and produces a 32-bit unsigned integer result.

```

        .xref H_MULU16

Int0    .zp $80                ;define zero-page variables
        .ds 2                  ;factor 1
Int1    .ds 2                  ;factor 2
Reslt   .ds 4                  ;product
        .code

rep #20                ;assume 16-bit accumulator
        .aw
pea (Int1)
pea (Int0)
jsl H_MULU16
pla
sta Reslt
pla
sta Reslt+2
bcs error                ;error if C=1 (e.g. overflow)
...

```

19.4.5.4 16-bit signed integer multiplication (H_MULS16)

Analogically to H_MULU16, this function accepts two 16-bit signed integer factors and produces a 32-bit signed integer result:

```

        .xref H_MULS16

Int0    .zp $80                ;define zero-page variables
        .ds 2                  ;factor 1
Int1    .ds 2                  ;factor 2
Reslt   .ds 4                  ;product
        .code

```

```

rep #20 ;assume 16-bit accumulator
.aw
pea (Int1)
pea (Int0)
jsl H_MULS16
pla
sta Reslt
pla
sta Reslt+2
bcs error ;error if C=1 (e.g. overflow)
...

```

19.4.5.5 32-bit unsigned integer division (H_DIVU32)

This function accepts two 32-bit unsigned integers and returns a 32-bit unsigned quotient and a 32-bit unsigned remainder:

```

.xref H_DIVU32

.zp $80 ;define zero-page variables
Int0 .ds 4 ;dividend
Int1 .ds 4 ;divisor
Reslt .ds 4 ;quotient
Rem .ds 4 ;remainder
.code

rep #20 ;assume 16-bit accumulator
.aw
pea (Int1+2) ;divisor first
pea (Int1)
pea (Int0+2)
pea (Int0)
jsl H_DIVU32
pla
sta Reslt
pla
sta Reslt+2
pla
sta Rem
pla
sta Rem+2
bcs error ;error if C=1 (e.g. division by 0)
...

```

19.4.5.6 32-bit signed integer division (H_DIVS32)

This is analogous to H_DIVU32, with the exception that the dividend, the divisor and the result are signed integers:

```

.xref H_DIVS32

.zp $80 ;define zero-page variables
Int0 .ds 4 ;dividend
Int1 .ds 4 ;divisor
Reslt .ds 4 ;quotient

```

```

Rem      .ds 4                      ;remainder
        .code

        rep #20                     ;assume 16-bit accumulator
        .aw
        pea (Int1+2)                ;divisor first
        pea (Int1)
        pea (Int0+2)
        pea (Int0)
        js1 H_DIVS32
        pla
        sta Reslt
        pla
        sta Reslt+2
        pla
        sta Rem
        pla
        sta Rem+2
        bcs error                   ;error if C=1 (e.g. division by 0)
        ...

```

19.4.5.7 32-bit unsigned integer multiplication (H_MULU32)

This function accepts two 32-bit unsigned integer factors and produces a 64-bit unsigned integer result:

```

        .xref H_MULU32

        .zp $80                     ;define zero-page variables
Int0     .ds 4                      ;factor 1
Int1     .ds 4                      ;factor 2
Reslt    .ds 8                      ;result
        .code

        rep #20                     ;assume 16-bit accumulator
        .aw
        pea (Int1+2)
        pea (Int1)
        pea (Int0+2)
        pea (Int0)
        js1 H_MULU32
        pla
        sta Reslt
        pla
        sta Reslt+2
        pla
        sta Reslt+4
        pla
        sta Reslt+6
        bcs error                   ;error if C=1 (e.g. overflow)
        ...

```

19.4.5.8 32-bit signed integer multiplication (H_MULS32)

Analogically to H_MULU32, this function accepts two 32-bit signed integer factors and produces a 64-bit signed integer result:

```

.xref H_MULS32

Int0    .zp $80                ;define zero-page variables
        .ds 4                  ;factor 1
Int1    .ds 4                  ;factor 2
Reslt   .ds 8                  ;result
        .code

rep #20                ;assume 16-bit accumulator
        .aw
        pea (Int1+2)
        pea (Int1)
        pea (Int0+2)
        pea (Int0)
        jsl H_MULS32
        pla
        sta Reslt
        pla
        sta Reslt+2
        pla
        sta Reslt+4
        pla
        sta Reslt+6
        bcs error          ;error if C=1 (e.g. overflow)
        ...

```

19.4.6 Floating point support

The EXT816.SYS module contains some routines doing floating-point calculations. They are intended as a replacement for the respective Atari ROM routines, so the same memory locations and the same Atari FP number format are in use. Also the results are interchangeable between EXT816 and the Atari ROM.

All these calls must be done in the native mode. The DP and DBR register contents and the AXY register sizes are preserved across the functions.

19.4.6.1 32-bit integer to FP conversion (H_IFP32/H_IFP32S)

For H_IFP32 the 32-bit unsigned integer must be stored in FR0 (\$0000D4-\$0000D7) prior to a JSL call to the function. Upon return, if the C=0, the FR0 (\$0000D4-\$0000D9) will contain the corresponding number stored in Atari 6-byte FP format. *This is a replacement for the Atari OS call IFP \$D9AA, which accepts a 16-bit unsigned integer as input.*

H_IFP32S works identically as the H_IFP32, with the sole exception that the input integer is interpreted as a signed value. The returned FP number will accordingly have its sign bit set or clear. As above, C=1 means an error.

19.4.6.2 FP to 32-bit integer conversion (H_FPI32/H_FPI32S)

H_FPI32 performs the conversion from a 6-byte FP number stored in FR0 (\$0000D4-\$0000D9) to a 32-bit unsigned integer returned in the first four bytes of that same memory location (\$0000D4-\$0000D7), provided that C=0 upon return. *This is a replacement for the Atari OS call FPI \$D9D2, which produces a 16-bit unsigned integer as a result.*

H_FPI32S is the corresponding routine which can convert an FP value, even if it is negative, into a signed 32-bit integer. As above, the input number should be passed in FR0 (\$0000D4-\$0000D9) and the resulting integer, if C=0 upon return, will be stored in \$0000D4-\$0000D7.

19.4.6.3 FP addition and subtraction (H_FADD/H_FSUB)

H_FADD is a faster replacement to the regular Atari ROM FADD \$DA66, and H_FSUB is a faster replacement of the respective FSUB \$DA60. *Both routines have been "borrowed" from Turbo-BASIC XL.*

H_FADD adds FR0 (\$0000D4-\$0000D9) to FR1 (\$0000E0-\$0000E5) leaving the result in the former. Likewise, H_FSUB subtracts FR1 from FR0 leaving the result in the latter. Upon return, if C=1, there was an error.

19.4.6.4 FP multiplication and division (H_FMUL/H_FDIV)

These are faster replacement functions to the respective FMUL \$DADB and FDIV \$DB28 from the Atari ROM. *Both routines have been "borrowed" from Turbo-BASIC XL.*

H_FMUL multiplies FR0 (\$0000D4-\$0000D9) by FR1 (\$0000E0-\$0000E5) leaving the result in the former. Likewise, H_FDIV divides FR0 by FR1 leaving the result in FR0. Upon return, if C=1, there was an error.

19.4.7 Other support

19.4.7.1 Stopwatch (H_TIME)

The stopwatch function is useful while benchmarking longer processes such as I/O transfers, memory moves, drawing picture elements on screen etc. The H_TIME has two functionalities:

a) start the timer: the current VBL timer (rtclock) state is remembered internally, no results are returned. Example:

```
.xref H_TIME

clc                ;C=0 - start timer
jsl H_TIME         ;call function
```

b) catch the intermediate time: the current VBL timer state is read, and the initial state of the timer, remembered above, gets subtracted from it. Next, the value is converted to floating point format and recalculated to seconds. At the end the result is written to the memory at the address passed to the function as a parameter. Example:

```
.xref H_TIME

sec                ;C=1 - stop timer
rep #20           ;set A to 16 bit
lda #timev        ;load destination address
jsl H_TIME         ;call function
```

The result is stored at the specified 16-bit address within the current data segment, i.e., in the example above, it will be *DBR:timev*. It consists of 10 bytes. The first four bytes is the

number of VBL ticks elapsed, in the form of a 32-bit integer. The next six bytes is the number of seconds as a floating point number in Atari format.

The function can measure time as long as 33554431 VBL ticks, i.e. about 6-7 days (depending on the TV system in use).

Please keep in mind that the the finest granularity possible is 0,02 sec. (0,0167 sec. on NTSC systems). Also, the result returned may well be 0, which is to be considered before passing it to further calculations, especially as a divisor.

The call must be done in the native mode. The DP and DBR register contents and the AXY register sizes are preserved across the function.

19.4.7.2 Pseudo-random number generator (H_RANDOM)

The difficulty with the actual hardware pseudo-random number generator located at the address \$00D20A is that it is actually rather slow. It is a shift register, so if any new value is read before the old value has been completely shifted out, the two values may be too closely correlated to obtain a satisfactory randomness. Therefore, in fact, even on a stock Atari it should not be read more often than at intervals of 8 bus cycles.

On a vanilla 6502 two consecutive states of that register may be obtained with a minimal interval of 4 bus cycles. Even on a non-accelerated 65C816 this interval can be shrunk to 3 bus cycles. On an accelerated processor it is possible to read that register's two consecutive states 1 bus cycle apart. This is not enough to obtain good pseudo-randomness and, due to this, ill phenomena may be observed even in Turbo-BASIC XL.

One solution is to wait for the shift-register to shift out its current value, but – since there are no good means to measure time intervals at this granularity – it usually involves either wasting CPU time grossly (such as when writes to WSYNC \$00D40A are used) or relying on delay loops, which are unreliable almost by definition.

The H_RANDOM function cures all that by providing satisfactory pseudo-randomness without much hassle – it is enough to call this function to obtain an 8-bit pseudo-random number in the low byte of the accumulator:

```
.xref H_RANDOM
jsl H_RANDOM
```

This generator is implemented in software, but it is initialized and automatically randomized using the hardware generator, so that it can be fairly certainly guaranteed to never produce the same sequence of numbers.

The call must be done in the native mode. The DP and DBR register contents, the contents of the MSB of the accumulator, the XY register sizes and contents are preserved across this function.

19.4.7.3 TOUPPER/TOLOWER

The EXT816.SYS module provides two functions: H_LOWERC and H_UPPERC, which convert a character passed in the accumulator into lower case or upper case, respectively. The

latter is a replacement for the library function TOUPPER (see chapter 14), the former has no standard counterpart. Note that ELSA the assembler allows you to define arbitrary aliases to symbols, so nothing prevents your from aliasing both as:

```
.xref H_LOWER
.xref H_UPPER
```

```
H_TOLOWER = H_LOWER
H_TOUPPER = H_UPPER
```

and using as if the system provided symbols H_TOLOWER and H_TOUPPER directly.

19.5 Replacement calls for the cartridge functions

There is a number of H_ functions which use the same inputs and return the same output data as their 8-bit counterparts located in the SpartaDOS X cartridge (thus the descriptions in the first 18 chapters of this document still apply). Their purpose is to make the 65C816 programming easier and programs more efficient by avoiding the costly and involved calls to the (slow) cartridge ROM. The differences are that:

- a) all calls must be done in the native mode and with the JSL instruction,
- b) upon exiting a function the CPU register sizes are restored to the state prior to the call,
- c) the input and output data may be passed on in a different way, more suitable for a 16-bit processor (see „Remarks” in the table below).

Whenever the table below speaks of a value „in AX”, it means the low word in the accumulator and the high byte (or word) in the X register.

New call	Location	Cart. call	Chapter	Remarks
H_CHDIR	EXT816.SYS	CHDIR	12.6	Uses H_FILE_P
H_CHKDSK	EXT816.SYS	GETDFREE	13.3	Uses H_FILE_P
H_CHMOD	EXT816.SYS	CHMOD	12.8	Uses H_FILE_P
H_CHVOL	EXT816.SYS	CHVOL	13.4	Uses H_FILE_P
H_CKSPEC	EXT816.SYS	CKSPEC	14.3	
H_DIVIO	EXT816.SYS	DIVIO	9.6	Uses H_FILE_P
H_ERROR	EXT816.SYS	U_ERROR	4.4	
H_EXPAND	EXT816.SYS	U_EXPAND	6.3	Expects full 24-bit address in AX
H_FCLOSE	EXT816.SYS	FCLOSE	8.2	
H_FCLSAL	EXT816.SYS	FCLOSEAL	8.2	
H_FDCLS	EXT816.SYS	FDCLOSE	10.2	
H_FDGETC	EXT816.SYS	FDGETC	10.2	
H_FDOPEN	EXT816.SYS	FDOPEN	10.2	Uses H_FILE_P
H_FGETC	EXT816.SYS	FGETC	8.3	
H_FFIRST	EXT816.SYS	FFIRST	10.1	Uses H_FILE_P

New call	Location	Cart. call	Chapter	Remarks
H_FLEN	EXT816.SYS	FILELENG	8.6	Returns file's length in AX and <i>addpos</i>
H_FNEXT	EXT816.SYS	FNEXT	10.1	
H_FPUTC	EXT816.SYS	FPUTC	8.3	
H_FSEEK	EXT816.SYS	FSEEK	8.7	
H_FSPEC	EXT816.SYS	U_FSPEC	5.14	
H_FTELL	EXT816.SYS	FTELL	8.7	Returns file's position in AX and <i>addpos</i>
H_GEFINA	EXT816.SYS	U_GEFINA	5, 6	Uses H_FILE_P
H_GEPATH	EXT816.SYS	U_GEPATH	5.12	Uses H_FILE_P
H_GETADR	EXT816.SYS	U_GETADR	5.6	Returns full 24-bit address in AX, mem index in Y
H_GETC	EXT816.SYS	GETC	9.4	
H_GETCWD	EXT816.SYS	GETCWD	12.7	Uses H_FILE_P
H_GETFLG	EXT816.SYS	U_GETFLG	5.16	
H_GETKEY	EXT816.SYS	U_GETKEY	9.8	
H_GETNUM	EXT816.SYS	U_GETNUM	5.5	Returns full 32-bit value in AX, not in <i>divend</i>
H_GETPAR	EXT816.SYS	U_GETPAR	5	Zeroes H_FILE_P
H_GETS	EXT816.SYS	GETS	9.5	See chapter 19.4.3.3
H_GONOFF	EXT816.SYS	U_GONOFF	5.7	
H_LOWERC	EXT816.SYS	-	-	See chapter 19.4.7.3
H_MKDIR	EXT816.SYS	MKDIR	12.5	Uses H_FILE_P
H_PARAM	EXT816.SYS	U_PARAM	5.17	
H_PATH	EXT816.SYS	U_PATH	6.4	Expects full 24-bit address in AX
H_PERROR	EXT816.SYS	U_PERROR	4.4	Expects full 24-bit address in AX
H_PUTC	EXT816.SYS	PUTC	9.1	
H_PUTS	EXT816.SYS	PUTS	9.2	See chapter 19.4.3.3
H_REMOVE	EXT816.SYS	REMOVE	12.3	Uses H_FILE_P
H_RENAME	EXT816.SYS	RENAME	12.1	Uses H_FILE_P
H_RENDIR	EXT816.SYS	RENDIR	12.2	Uses H_FILE_P
H_RMDIR	EXT816.SYS	RMDIR	12.4	Uses H_FILE_P
H_SBOOT	EXT816.SYS	SETBOOT	12.9	Uses H_FILE_P
H_SLASH	EXT816.SYS	U_SLASH	5.8	Expects full 24-bit address in AX
H_TOKEN	EXT816.SYS	U_TOKEN	5.9	Expects full 24-bit address in AX, returns word A
H_UPPERC	EXT816.SYS	TOUPPER	14.2	See chapter 19.4.7.3
H_XDIVIO	EXT816.SYS	XDIVIO	9.6	

20 Using SORTDIR.OVL

20.1 Function

While playing with the SpartaDOS X you may notice that the CAR: device contains several hidden OVL files. Most of them are system modules and have no direct use to you. There are exceptions, though, and one of them is CAR:SORTDIR.OVL: a module specialized in sorting directories, designed to be used in application programs. At the time of writing this there are two utilities which take advantage of its presence: SORTDIR.COM and Sparta Commander. The former is an utility program to sort directories on a disk permanently. The latter uses the OVL module to display directories, which are physically unsorted, in sorted manner.

In the current implementation it takes SORTDIR.OVL about 7.2 seconds to sort a full-size directory (1424 entries = 32 KB), which is fast enough in most cases considering that an average directory is hardly longer than 5% of that.¹⁰⁾

The module is able to automatically detect if a directory was already physically sorted by the same algorithm. Therefore e.g. the Sparta Commander does not sort directories which have been physically sorted using SORTDIR.COM.

20.2 Initializing

The SORTDIR.OVL module is a standard, SpartaDOS X executable binary. To initialize the module for use with your program it is enough to run it:

```

        ldx #$02
_ld     lda sdv-1,x
        sta FILE_P-1,x
        dex
        bne _ld

        stx FLAG
        jsr U_LOAD

```

where *sdv* is defined as follows:

```

sdv     .word sdname
sdname  .byte "SORTDIR.OVL", $9b

```

After the call to U_LOAD returns, the module is ready to use. The sorting routine is loaded to the RAM located „under“ the SpartaDOS X cartridge, with the entry point at \$A000. Therefore, in your program, you can define a label for it, for example:

```
do_sort = $a000
```

Of course, whenever your program uses that same RAM area for other purposes and still wants to sort directories, the SORTDIR.OVL will have to be reloaded.

¹⁰ The original SORTDIR.COM shipped on the ICD's SpartaDOS Toolkit disk took several *minutes* (sic) to sort a full-size directory.

20.3 Memory usage

The module, apart from the memory located under the cartridge bank, also uses 30 bytes of the zero-page, at \$80-\$9D. That area is automatically preserved across the calls, the only limitation is that these bytes should not be used by interrupt handlers while any of the SORTDIR.OVL functions is performing its tasks.

20.4 Preparing directory data

The directory data to be sorted must be first loaded to RAM. A directory can be as long as 32 KB, therefore it is best to reserve this much (using MALLOC, see chapter 3), especially when writing an utility like SORTDIR.COM, for which it is kind-of necessary to be able to sort any directory in the system, even the longest one.

However, a shell such as the Sparta Commander may allocate a smaller buffer counting on the fact that most directories are much shorter. And indeed, Sparta Commander only sorts directories containing up to 711 entries (= 16 KB), leaving longer ones unsorted. The buffer may be located in the conventional RAM or in the banked memory. The only real requirement is that it must be continuous: you may not, for example, load one directory to two consecutive extended RAM banks and expect the SORTDIR.OVL to be able to sort that.

For now let us assume that the buffer has already been allocated, its address is being held in *bufbot* and *bufbot+1*, its size is 32752 bytes, and its memory index is 0 (= conventional RAM):

```
mode      = $0778
scan      = $0779
attr      = $077A
addpos    = $0782
bytbuf    = $0785
memreix   = $0787

        jsr U_GETPAR      ;get directory name from command line
        lda #$14          ;raw directory, read mode
        sta mode
        lda #$08          ;directories only
        sta scan
        lda #$00
        sta attr
        jsr FOPEN         ;open raw directory
        ldx #$01
_buf      lda bufbot,x     ;load buffer address to addpos
        sta addpos,x
        dex
        bpl _bp
        lda #<32752       ;load buffer size to bytbuf
        sta bytbuf
        lda #>32752
        sta bytbuf+1
        lda #$00          ;memory index to memreix
        sta memreix
        jsr FREAD         ;read directory contents
        jsr FCLOSE        ;close file
```

The number of bytes actually read is left by FREAD in *bytbuf*, get it from there:

```
lda bytbuf
clc
adc bufbot
sta buftop
lda bytbuf+1
adc bufbot+1
sta buftop+1
```

The *buftop* holds the address of the first byte past the buffer.

20.5 Invoking the sorting procedure

The sorting procedure's parameters must be passed via the stack and pushed in the following order:

- 1) Data memory segment, 1 byte. On stock, 6502 Ataris this parameter must always be zero: passing anything else will cause undefined results.
- 2) Buffer end address, 2 bytes. In our example above it is called *buftop*. MSB must be pushed first.
- 3) Buffer start address, 2 bytes (MSB first); or *bufbot* in the code above.
- 4) Sorting mode, 1 byte. This decides by what criteria the directory should be sorted: \$0x means sort by name, \$4x – by size, \$8x – by time, \$Cx – by extension. Two lowest bits select additional functions: \$x0 – sort in ascending order, \$x1 – sort in descending order, \$x2 – disable sorting. Other values are reserved for future definition.

When the parameters have been stacked, the RAM under the SpartaDOS X cartridge must be activated using the Hwctrl interface (see chapter 26). Then the data buffer's memory index should be loaded to the accumulator; and after that, the sorting routine proper is invoked **using the JMP instruction** (not JSR!):

```
jhwctrl = $070F

sortme lda #$00      ;segment 0: 6502-compatible addresses
pha
lda buftop+1        ;buffer end
pha
lda buftop
pha
lda bufbot+1        ;buffer start
pha
lda bufbot
pha
lda #$00            ;sort by name, ascending order
pha
ldx #hw_ram         ;cart off
jsr jhwctrl
lda #$00            ;memory index $00 = conventional RAM
jmp do_sort
```

The procedure *do_sort* will pull all the data from the stack, setup the memory banks as necessary, perform the ordered task, then call *jhwctrl* to activate the SpartaDOS X cartridge and return to the place where the *sortme* subroutine was invoked from. Therefore it is not necessary to do anything more or clean-up after its return.

20.6 Accessing buffered directory

Considering that the directory may be buffered in an extended memory bank, an access to the contents of the buffer may be a problem for application programs which cannot control their position in the memory and therefore cannot easily ensure that the bank-switching code is located outside the bank-switching area.

Therefore the SORTDIR.OVL contains a function to copy directory entries between the directory buffer and the application's memory. This function is defined as:

```
move_entry = $a003
```

The parameters are passed as follows:

- 1) three-byte *zptr* buffer pointer on the stack; this pointer must contain an absolute address to the actual entry stored inside the buffer;
- 2) memory index of the buffer in the accumulator;
- 3) copying direction in the C flag: C=0 from the buffer (read), C=1 to the buffer (write).

When the C=0, the entry pointed to by the memory index and *zptr* will get transferred to the location in the main memory pointed to by a 16-bit address stored in *addpos* \$0762-\$0763. When C=1, the directory entry stored in *dentry* \$0789-\$078F will get transferred to the location pointed to by *zptr* and the memory index. Always 23 bytes are transferred.

The *zptr* address is calculated according to the following formula:

$$zptr = buffer_address + (entry_number * 23)$$

The *buffer_address* value your program obtains from the MALLOC call (chapter 3) issued to allocate the buffer. The entry number 0 is the directory header, the user entries are numbered from 1 onwards.

As above, the procedure must switch off the SpartaDOS X cartridge bank before being called. **Contrary to the above, the procedure must be called using a JSR.** Apart from that it cleans up the stack, does all the memory configuration as appropriate and, upon exit, restores the cartridge state automatically. Example:

```
ldx #hw_ram      ;cart off
jsr jhwctrl
lda ptr+2        ;on 6502 this must be 0
pha
lda ptr+1
pha
lda ptr
```

```

pha
lda buf_index
sec                ;write dentry to address specified by 'ptr'
jsr move_entry
...

```

20.7 65C816 remarks

In the presence of the 65816.SYS driver (see chapter 19) it is possible to store the directories in the memory past the first 64k (memory index \$03). This is also what Sparta Commander does, as the presence of the extra RAM enables it to sort full-size directories (1424 entries each).

When the directory buffer was allocated from the memory past the first 64k, or in other words, its address is greater than 16-bit, the following rules apply:

- 1) the *Data memory segment* parameter for the sorting procedure must be equal to the top-most byte of the 24-bit address of the buffer;
- 2) the entire buffer may not cross a 64k boundary. It is therefore best to make it either 32k or 64k-aligned (take it as a kind of speed optimization);
- 3) the memory index for the *do_sort* is a don't-care, it is best to pass a 0 for it. However, for the *move_entry* procedure the memory index passed for the 65C816 memory should be equal to \$03.

21 Using SCANDIR.OVL

21.1 Function

Another user library, after SORTDIR.OVL explained in chapter 20, is CAR:SCANDIR.OVL. As its name may already suggest, it scans directories, or, to name it more accurately, its function is to traverse directory trees recursively so that a program, which wants to do such an operation, does not have to implement this internally.

This library is used by the commands DELTREE, DU and FIND. The first one is an utility program to delete entire subdirectories, the next one – that scans the specified directories to gain the information on how much disk space is taken by them. The last one is an utility to find the specified file(s) in the specified file system.

21.2 Principles of operation

The principles are very simple. The library defines two main functions, named SD_FIRST and SD_NEXT, which are analogous to FFIRST and FNEXT (see chapter 10).

The difference is that the latter ones just return consecutive entries of a specified directory, whereas the former ones do the same only as long as these entries belong to regular files or symbolic links; otherwise, when a subdirectory entry is encountered, the SCANDIR.OVL library enters that subdirectory and starts returning its entries (with full path) to the calling program. And when that subdirectory ends, the library returns to the parent directory and continues.

21.3 Usage

Using this library may seem somewhat more complicated than SORTDIR.OVL. Libraries cannot be automatically loaded in SpartaDOS X, worse even, when a program loads a library with U_LOAD, it cannot use references to the symbols defined by that library, because at the time of loading the program these symbols do not yet exist.

The solution is to split the program into a loader part (which will load the necessary libraries first) and the main part, which will be loaded and executed as last.

21.3.1 Example loader code: FIND.COM

As an example code for the „loader” part we can use CAR:FIND.COM, or rather its version a bit simplified for clarity (MADS syntax):

```
FILE_P  smb 'FILE_P'
FLAG    smb 'FLAG'
U_LOAD  smb 'U_LOAD'

    opt ?+

    blk reloc main

find    ldx #$01                ;setup for SCANDIR.OVL load
?sf     lda sdlibp,x
        sta FILE_P,x
        dex
        bpl ?sf
```

```

        lda #$00
        sta FLAG
        jsr U_LOAD                ;load SCANDIR.OVL without execution

        ldx #$01                  ;setup for FIND.OVL load
?so     lda ovlp,x
        sta FILE_P,x
        dex
        bpl ?so

        lda #$00
        sta FLAG
        jmp U_LOAD                ;load and run FIND.OVL

sdlbip  .word sdlbip
ovlp    .word ovln

sdlbip  .byte 'SCANDIR.OVL', $9b
ovln    .byte 'FIND.OVL', $9b

        blk update new find '@FIND'

```

The first U_LOAD call loads the SCANDIR.OVL. This not only loads the code to the memory, but also adds symbols, which are being exported by the library, to the global symbol database. The library is loaded without execution: thanks to that it is held in the memory instead of being unloaded otherwise. The final U_LOAD loads and runs the main program, which can be dependent on the symbols defined by the library.

Note that this sequence of calls does not disturb the command line in any way, the command line gets passed on to the main program without any changes.

When the main program exits, everything, i.e. the main program, the library (or libraries, if more of them have been loaded) and the loader part, all get removed from the memory.

21.3.2 Initialization (SD_INIT)

The first call a program should invoke is SD_INIT. Its purpose is to setup the parameters for the incoming directory scanning. The parameters are:

- 1) the mask of file search attributes stored in *scan* (\$0779). It may be set arbitrarily (see chapter 8.1.2) or taken from the command line (see chapter 5.13).
- 2) the complete path specification (along with the file mask at the end) pointed to by FILE_P. Again, this may be set to an arbitrary buffer or just inherited from a call to U_GETPAR (see chapter 5.4).

The beginning of the main program wishing to use the library may look like this:

```

        lda #$00
        jsr U_GETATR              ;get file specification and attributes
        jsr U_FSPEC               ;add default file mask if none given
        jsr SD_INIT               ;initialize SCANDIR.OVL

```

The call to SD_INIT does not return any status codes.

21.3.3 The main loop (SD_FIRST/SD_NEXT)

After successful initialization the library is ready to use. The main loop's main components are calls to SD_FIRST and SD_NEXT. These two functions behave like their „flat” counterparts, FFIRST and FNEXT, in particular:

- a) SD_FIRST opens a file and leaves its handle in *file_id* \$0760. This file must be closed after use.
- b) an End-Of-File condition causes both functions to return a negative status (N=1);
- c) all other errors invoke U_FAIL automatically (see chapter 4.1).

However, since SD_FIRST and SD_NEXT are supposed to scan the entire subdirectory tree, they will, while doing that, close and open directories as needed. Therefore, after a call to SD_NEXT, the *file_id* variable **may contain no longer the same handle!**

And example of the program's main part:

```
loop      jsr SD_FIRST           ;find the first matching file
          jsr SD_NEXT           ;find a next matching file
          bpl loop              ;if success, call again
          jsr FCLOSE            ;close on EOF
```

This displays the contents of the specified directory tree. Nothing else is needed, because both functions automatically print on the screen the paths to the files and directories, which match the initial criteria. How to suppress that, it will be explained later on.

21.3.4 Internal variables (SD_VARS)

The symbol SD_VARS points to an internal table containing internal control variables and some additional information the library collects while performing its tasks.

Offset	Size	Label	Function
-3	byte	dinh	Do not display (inhibit) directories, if bit 7 = 1 (but still scanning them).
-2	byte	finh	Do not display files, if bit 7 = 1 (but still counting them).
-1	byte	flat	Do not dig into subdirectories, if bit 7 = 1.
0	4 bytes	dsize	The size of all matching directories in bytes.
4	4 bytes	fsize	The size of all matching files in bytes.
8	4 bytes	tsize	The total size of all matching objects in bytes (dsize+fsize).
12	4 bytes	dcnt	The count of matching directories.
16	4 bytes	fcnt	The count of matching files.

All these fields are zeroed by the SD_INIT function. The five latter fields are (obviously) intended to be read-only.

21.3.5 Paths (SD_DPATH/SD_FPATH)

These two symbols point to current working paths:

- a) SD_DPATH – full path to the directory currently being visited
- b) SD_FPATH – full path to the file currently being processed

Both strings are EOL-terminated and are valid no earlier than after the first call to SD_FIRST.

21.3.6 Call-backs (SD_INCB/SD_EXCB)

The library provides two call-back vectors, where your program may insert own procedures performing additional tasks:

Symbol	Size	Synopsis
SD_INCB	word	Called when a directory is being entered.
SD_EXCB	word	Called when a directory is being exited.

Initially, these two addresses just point to an RTS and do nothing. They are also initialized to this state by a call to SD_INIT.

To register own procedure, you have to overwrite a 16-bit word pointed to by the respective symbol with an address to your subroutine. That subroutine must end with an RTS. The A/X/Y register contents or CPU flags do not have to be preserved.

The SD_INCB is called, when SD_DPATH and SD_FPATH are already valid for a new directory about to be entered, but the directory itself has not yet been opened or its path displayed.

The SD_EXCB is called, when the directory about to be left has already been closed, but SD_DPATH and SD_FPATH are still pointing to it. You can use the callback to e.g. delete that directory, if required.

22 SpartaDOS X SIO Interface

22.1 Function

The „SIO” here stands for „Sector I/O” – this subsystem is intended to be used as a functional replacement for the OS’s SIOV (or JSIOINT \$E459). The SpartaDOS X SIO interface – also known as LSIO – contains a high-speed driver for devices attached to the serial bus; it also handles the regular parallel bus interface (PBI) devices and ramdisks. Virtually any Sector I/O device can be handled by that subsystem, such as non-PBI hard disks etc.

22.2 Calling convention

There are two separate entry points to the SIO Interface, slightly differing in functionality. One entry, labelled *lsio*, compatible to SpartaDOS 3, is located at COMTAB-11. Another one, *jsector_io*, lives at \$070C. Only the latter one is used internally by the system.

Both accept the transfer parameters in DCB (\$0300-\$030B) or XDCB (\$0300-\$030F), both return the status information in the N flag of the CPU status register: N=1 means an error, N=0 no errors. The exact status code is then present in the accumulator (for backward compatibility with SIOV it is also copied to the Y register).

One additional input parameter is *memreix* \$0787 which specifies the banked memory configuration to be selected for the time of the data transfer itself.

The operation of both entry points is almost identical with the one exception that the former always zeroes *memreix* \$0787 at its beginning, therefore doing the requested transfers to and from the main memory only; whereas the latter does not touch *memreix* causing the requested transfer to be performed on the memory region indicated by that variable.

22.2.1 XDCB structure

DCB, or Device Control Block, is a widely documented SIO parameter structure located at \$0300-\$030B. Setting bit 7 of *ddevic* \$0300 to 1 (making the device code greater than 127) signalizes that XDCB use is requested from the SIO driver.

The XDCB was first introduced about 2004 in some parallel HDD interfaces to break the 16-bit sector number limit immanent to the standard SIO protocol. It is currently supported by IDE+2.0 and U1MB PBI (and possibly some others) and allows them to access sectors on (parallel) HDD partitions greater than 32 MB.

The XDCB is defined so that the first 12 bytes are equal in function to the corresponding bytes of the DCB. The entire structure looks as follows:

Addr.	Label	Size	Function
\$0300	<i>ddevic</i>	1	Device identification code, e.g. \$31 for a disk drive.
\$0301	<i>dunit</i>	1	Device unit number.
\$0302	<i>dcmnd</i>	1	Command byte.
\$0303	<i>dstats</i>	1	Transfer direction (input) and transfer status (output).
\$0304	<i>dbufa</i>	2	Buffer address, bits 0-15.

Addr.	Label	Size	Function
\$0306	<i>dtimlo</i>	1	Timeout value.
\$0307	<i>dunuse</i>	1	Not used, reserved. Should be kept 0 for compatibility.
\$0308	<i>dbyt</i>	2	Sector size.
\$030A	<i>daux1</i>	1	Sector number, bits 0-7.
\$030B	<i>daux2</i>	1	Sector number, bits 8-15.
\$030C	<i>daux3</i>	1	Sector number, bits 16-23.
\$030D	<i>daux4</i>	1	Sector number, bits 24-31.
\$030E	<i>dbfx1</i>	1	Buffer address, bits 16-23.
\$030F	<i>dbfx2</i>	1	Reserved, should be kept \$00 for upward compatibility.

It should be kept in mind that the regular Serial I/O protocol only allows for 16-bit sector numbers. Therefore 32-bit sector numbers may effectively be used only with parallel devices. Also, the extra bits of buffer address are to be used on systems equipped with the 65C816 CPU and extra memory past the first 64k.

In practice the XDCB usage has so far been limited to device \$31 (or \$B1), i.e. the disk device, because it is mainly disks which require larger ranges of sector numbers and memory buffers.

22.3 Adding your device to the system

22.3.1 Control tables and variables

The *sio_vector* \$0718-\$071F is a table which contains vectors to the SIO drivers registered in the system. This table is 8 bytes long, it contains 4 entries, two bytes each, for four possible SIO drivers. One entry is normally occupied by the regular SIO.SYS.

Another table associated with SIO is *sio_index*. It is located at COMTAB2+2 and contains 15 one-byte entries. It is used to „swap“ the logical disk drive numbers with accordance to the user's preferences (see the Command Processor's SWAP command).

The CURDEV variable contains the default boot device identifier during boot (e.g. \$01 = drive D1: or A:, or \$21 = CAR: etc. depending on the boot configuration). If your driver is a parallel storage manager (like e.g. SIDE.SYS), it may want to update this location in order to change the actual „boot drive“ number, i.e. the number of the drive where the AUTOEXEC.BAT file will be searched for. The system will accordingly update the \$BOOT\$ variable value upon the CONFIG.SYS's completion.

22.3.2 The entry point

The entry point registered in the *sio_vector* table is just the beginning of the routine that performs the transfer. That entry point must be located in the main RAM, for the entries in the SIO vector table contain flat 16-bit addresses only and no memory indices. But nothing prevents the bulk of the driver's code to be located in the extended RAM, whether it is physically located under the ROM or in banked RAM (this will also work when the „extended RAM“ is in the main memory in USE NONE configuration).

On the account of the above the typical entry point to a SIO driver looks like this:

```

V_setme = $07F1
V_popme = $07F4

oursio lda ext_m           ;a copy of EXTENDED, see below
      jsr V_setme         ;switch the ext RAM on
      jsr do_sio          ;call the code located there
      jmp V_popme         ;switch the ext RAM off and return
ext_m  .byte $00          ;a copy of init-time EXTENDED

```

22.3.3 Installing your driver

It is best to have the installation code loaded as an absolute segment (signature \$FFFA) to the area \$0400-\$06FF. It is a 768-byte space which is large enough to initialize most drivers. The code loaded there will not be resident – the to-be-resident parts may of course not rely on any code or data present in that section, as this is just a scratch-pad.

It is to be kept in mind, though, that the area \$0580-\$05BF is used by the library functions dealing with the environment variables: GETENV, PUTENV and NUMENV. If the initializing code of your driver calls any of these functions, please make sure that there is no conflict of that buffer with your code.

If the driver loads any its parts to the system extended area (memory type \$02), the first thing the initializing code should do is to read out the variable pointed to by the symbol EXTENDED and remember its value in a private, resident variable. Let us say, for future reference, that that variable's name is *ext_m*:¹¹⁾

```

lda EXTENDED
sta ext_m

```

The actual value of EXTENDED will depend on the global memory configuration, predefined in the CONFIG.SYS file (USE BANKED/OSRAM/NONE). As explained in the chapter 3, when the memory assigned to the memory index \$02 is not available (full or not present), the system will fall back to index \$00 (= conventional RAM). Also, needless to say, the state of EXTENDED must be remembered, because EXTENDED is a global system variable, which may (and surely will) be changed as soon as any other program gets loaded.

Next, the driver should determine, whenever possible, if no conflicting driver has already been loaded – and exit immediately, if it has.

Then it should determine – in a device-specific manner – if the I/O device it is supposed to handle is physically available (and, of course, exit if it is not).

When all this is passed, the driver should install itself in the *sio_vector* table as shown below.

At the end, it should set \$FF to the system variable being pointed to by the symbol INSTALL, then exit with RTS. INSTALL is zeroed before starting a program, so the common practice is to decrement that variable straight before quitting the installation routine:

¹¹⁾ For clarity the code examples in this chapter will not contain any obvious size-optimizations.

```
dec INSTALL
rts
```

This will be a signal for the loader that the memory being occupied by the relocatable blocks of the program, that has just exited, should not be released, but the reverse – that the program should be permanently held in the memory.

When a SIO call is being executed, the *sio_vector* table is scanned forwards by the system, i.e. towards its positions located at the higher addresses in the memory. Thus the priority is higher towards the beginning of the table and lower towards its end. Drivers with higher priority will be (naturally) called first, thus getting an opportunity to mask out any possible driver located at a lower position (including the default SIO.SYS).

This is why a new driver should always get installed at **the last free** position in the table. To find a free position it is enough to make sure that its most significant byte is \$00:

```
sio_vector = $0718

        ldx #$06
_srch   lda sio_vector+1,x
        beq found
        dex
        dex
        bpl _srch
        ...                ;error: no more drivers can be installed
        rts

oursiov .word oursio

found   lda oursiov
        sta sio_vector,x
        lda oursiov+1
        sta sio_vector+1,x
```

The *oursiov* location should contain a 16-bit address pointing to the entry to the main communication routine of the driver, as shown in the previous passage. That location will get called by the system using the JSR instruction in the circumstances also sketched out above.

22.4 Basic operation

The routine being entered at the aforementioned entry point is the one which directly interprets the (X)DCB and *memreix* \$0787 contents and performs the transfer.

Once done, it should load the appropriate status code to both the accumulator (for the system) and Y register (for the backward compatibility with SIOV), clear the Carry flag of the CPU flag register, then execute RTS.

That C=0 is a signal for the system that the operation has been completed by the appropriate driver, that the accumulator contains a valid status code, and that the control should be handed back to the one who had called the SIO.

Otherwise, when the driver returns with $C=1$, the system will attempt to invoke the next driver registered in the *sio_vector* table. When no more drivers are available, the call returns with a negative error code.

It should be kept in mind that the state of the C flag returned by the driver is an information intended for the system only; the user code may only rely on the state of the N flag and on the contents of the accumulator.

22.4.1 The function of *sio_index*

The *sio_index* table, as said above, translates the user-selected logical drive number into the „actual“ logical drive number. They are usually the same, unless the user has changed the default assignment using the SWAP command in the Command Processor. The only thing the command does is to change the contents of the *sio_index* table.

To take that into account, every SIO driver should do the following at the very beginning:

```
ddevic = $0300
dunit = $0301

do_sio  ldx dunit
        lda ddevic          ;sio_index is only for device $3x
        and #$70            ;(i.e. disk storage device)
        cmp #$30
        bne no_swap

        cpx #$10            ;and for drives 1-15
        bcs no_swap

        lda COMTAB2+2-1,x
        tax
no_swap
```

After this the X register contains the valid unit number to be used further on within the driver. It should **not** be stored back to *dunit* \$0301, this being an input variable only. In case when this is required for a reason, the value of *dunit* \$0301 should be preserved on the stack and restored afterwards.¹²⁾

Remark: in SpartaDOS X 4.2 the sio_index table was located at \$070F and contained 9 entries for 9 possible drives. In SpartaDOS X 4.4 the number of drives has been increased to 15, which forced the sio_index table, now too short, out of its old place.

¹²⁾ Yes, all this *could* be done by the system automatically before a call to the appropriate driver. The reason why this was delegated to the drivers is that embedding this in the LSIO would raise the *memlo* value by several dozens of bytes; whereas, when it is done by the drivers, it can occupy extended memory, saving the RAM space between *memlo* and *memtop*. This is a good example of give-and-take tradeoffs you have to constantly decide upon while programming an 8-bit operating system.

22.4.2 Global RAM locations your driver may use internally

If you are writing a SIO driver, the area on the zero page from \$30 to \$3F is (almost) all yours. Its structure is basically defined by the OS documentation as follows:

Addr.	Label	Size	Function
\$30	<i>status</i>	1	Status: it will be later copied to <i>dstats</i> and respective CPU registers.
\$31	<i>chksum</i>	1	Data block checksum (if applicable).
\$32	<i>bufr</i>	2	16-bit buffer pointer (a copy of <i>dbufa</i>).
\$34	<i>bufen</i>	2	16-bit end buffer pointer (<i>dbufa+dbyt</i>).
\$36	<i>ltemp</i>	2	May be used by the OS, should get preserved across your driver.
\$38	<i>bufrfl</i>	1	„Buffer full” flag for serial I/O.
\$39	<i>recvnd</i>	1	„Receiving done” flag for serial I/O.
\$3A	<i>xmtdon</i>	1	„Transmit done” flag for serial I/O.
\$3B	<i>chksnt</i>	1	„Checksum sent” flag for serial I/O.
\$3C	<i>nocksm</i>	1	„No checksum” flag for serial I/O.
\$3D	<i>bptr</i>	1	Cassette buffer index.
\$3E	<i>gaptyp</i>	1	Type of gaps between cassette record (long or short).
\$3F	<i>feof</i>	1	EOF flag for cassette.

As said above, this is so by the OS's definition, but in fact these variables are internal to your driver, so as long as you do not touch *ltemp* \$36-\$37, their exact use is completely up to you. So the table shown above serves just to exemplify one of possible usages. For instance, if you are writing a ramdisk or a parallel device driver, you may want to skip calculating checksums or handling record gaps, because this stuff only applies to (specific) serial devices.

22.4.3 Memory management within the driver

A SIO driver must be able to perform data transfers to and from any memory type that is being managed by SpartaDOS.

Transfers to or from the extended memory (banked RAM at \$4000-\$7FFF controlled by PORTB \$D301 or Axlon \$CFFF registers) will be requested by other system components in the *memreix* \$0787 by setting there the required memory index.

The responsibility of setting up the appropriate memory configuration (using *V_setme* \$07F1 and *V_popme* \$07F4 system calls) **and of buffering data whenever necessary, lies on the SIO driver which is about to perform the transfer**, in the manner explained further on.

It is important to understand the difference between the two types of memory: the memory the driver has been loaded to, and the so called I/O memory, i.e. the one on which the transfer is about to be performed.

For practical purposes of making as much as possible of the main RAM (memory index \$00) available to programs, most resident drivers tend to load bulk of their code to the extended RAM. But when the actual data transfer is about to be made, the memory must most of the time be switched to make the I/O memory available. As this usually involves banking the extended RAM, the code which is about to switch the banks must be obviously located outside the banked area (\$4000-\$7FFF).

This also means that the specific routines responsible for the actual data transfers must be located in the main memory (index \$00) under the *memlo* pointer, and below \$4000. These are usually just two loops, one copying from a data port to the memory (let us call this one *_io_read*), and the other copying from memory to a data port (*_io_write*). The exact nature of them is of course up to the device, so we will not go into details on how these work internally: let us just consider them black boxes which transfer the ordered data block in the ordered direction.

Let us get back to the driver's entry point. As demonstrated above, it usually consists of just a memory switch and a call to the portion of the code which was loaded to the extended RAM. The following „listing” will show schematically, what are the things which the code should do in order to perform the transfer correctly:

```
status = $30
dstats = $0303

do_sio ...                ;setup unit no. acc. to sio_index
...          ;dispatch (X)DCB input parameters
...          ;setup timeouts and the like
lda #$01     ;init status value to default
sta status
bit dstats   ;check transfer direction
bpl _rd

...          ;setup buffer pointer

jsr _io_write ;if dstats=$80 or $c0, write
bmi _error

bit dstats
_rd bvc _cpl

...          ;setup buffer pointer again

jsr _io_read  ;if dstats=$40 or $c0, read
bpl _cpl

_error sta status
_cpl  lda status
      tay
      sta dstats ;write status back to DCB
      clc        ;operation done
      rts        ;end
```

The *_io_read* and *_io_write* must, as said above, be unconditionally located in the main memory (index \$00). And, whatever they do, it must be done in the scope of another pair of *V_setme/V_popme* calls, in this manner:

```
memreix = $0787
V_setme = $07f1
V_popme = $07f4

_io_read lda memreix      ;get the index of I/O memory
        jsr V_setme      ;switch to I/O memory
        ...              ;perform transfer, set status in A
        jmp V_popme      ;switch banks back, return
```

Remark: it should not be assumed that if memreix=\$00, switching the banks to change the current memory arrangement would not be required – one should just remember that the currently active memory may not necessarily be that of index \$00.

Setting buffer pointers twice is only necessary when your driver supports commands which can perform both read and write in one shot. If it always will be either read or write (like most disk drive I/O commands are), the buffer pointer may be set up only once. Similarly, some devices (such as ramdisks) would not need setting up timeouts etc.

22.4.4 (X)DCB interpretation steps

1) The first thing to check is the device code itself. If lower 7 bits of *ddevic* \$0300 indicate a device which your driver does not handle, it should return immediately with C=1.

2) The same thing should be done when *ddevic* \$0300 indicates „your” device, but *dunit* \$0301 (after the translation done against the *sio_index* table, see above) points to a unit outside the range known to your driver.

3) Now check the bit 7 of *ddevic* \$0300. As said above, if bit 7 is set to 1, this means that the caller is requesting an XDCB operation. If your driver only supports the regular DCB, the safest approach is to just return with C=0 and error code 176 (Access denied) or 184 (Wrong architecture) in A and Y registers.

4) If both *ddevic* \$0300 and *dunit* \$0301 indicate „your” device, check two top bits of *dstats* \$0303: \$4x means a read operation, \$8x a write operation, \$Cx a write+read operation, and \$0x – no data transfer operation. If your device is read-only, *dstats* \$0303 bit 7=1 should cause the driver to return an error 139 (NAK) – but if it is generally writable, just write-protected at the moment, the error should be 144 (Write protected). If the device does not support write+read operations, the proper reaction on \$Cx at *dstats* is to return error 139 (NAK).

5) Next, dispatch the command byte *dcmnd*.

Floppy disk drivers should be able to accept at least these commands: !, P, R, W, S, plus N and O when more than one density is supported, plus " (ASCII 34) when Atari 1050 compatibility is required.

*Hard disk drivers should support !, P, R, W, S, and N (note: not O). In that case, ! may be dummy (do nothing, return success), and W may be equal to P, but neither one must be left unimplemented. **For the actual operation of all these commands, please refer to the regular SIO documentation.***

6) When a valid command has been accepted, verify if the buffer size *dbyt* \$0308-\$0309 is suitable for the command. If not, return error 176 (Access denied).

7) Next, if applicable (i.e. for the commands P, R, W), check the sector number against the bounds allowed for the specified logical device. If the sector number is outside the range, return error 139 (NAK). *Note that sector number 0 is never valid in Atari systems: if a disk contains 720 sectors for example, they are counted from 1 up to 720.*

8) Next, setup your buffer pointer (like *buf*) and other buffer parameters if necessary (e.g. *bufen*, buffer counter etc.) If the XDCB-passed buffer pointer is greater than 16-bit and the CPU is 6502 (see chapter 19.1.5), return error 184 (Wrong architecture).

9) Otherwise, if the CPU is able to perform the transfer using 24-bit addresses, setup the 24-bit pointers and use the appropriate machine code instructions and addressing modes to accomplish the task. Skip this on 6502.

Note that, if the system or the user requests a SIO transfer to be performed on the specific I/O memory, no SIO driver should trouble itself with the question if that memory really exists: it should assume that the calling program already has that verified, so no additional checks are necessary here.

10) Do any necessary calculations required by the physical device (such as logical to physical sector address translation etc.), if applicable.

11) Perform the data transfer, if applicable.

12) Return status bits to the Operating System.

22.5 65C816 support

For extra information related to the 65C816-enabled systems, please see chapter 19.3.

23 Block Data Management Interface (Block I/O)

23.1 Function

The block data management interface, also known as the block I/O interface, is a wrapper over the SIO Interface (see chapter 22). Its primary purpose is to perform block I/O operations on behalf of the SPARTA.SYS and its subordinate file system drivers, such as ATAR-IDOS.SYS or FATFS.SYS. All sector reads or writes, requested by these file system drivers, are going through the block I/O vector.

In theory, the block data management interface is designed to provide an external sector caching mechanism for the file system drivers. The system, however, does not provide anything like that by default: the block I/O interface is a stub, which, to perform the caching, has to be expanded with an external caching module.

Considering, though, that the hard disks for Atari, even those operating in pure PIO mode, tend to be significantly faster than RAM-disks, adding an external caching module may result in performance penalty.

It may, however, still be considered for slow storage (such as serial drives and serial drive emulators). And caching parallel hard drives may still make sense on accelerator boards, where the RAMs operating at 10, 14, 20 or even 40 MHz should still be significantly faster than a hard disk drive attached to the Atari bus clocked at 1.77 MHz.

This mechanism is used in a 65C816 caching module, named BIO/816. On accelerated machines its use makes many file system operations go more smoothly even if a fast storage (i.e. a PBI HDD) is in use.

The block I/O interface's code is physically located inside the SPARTA.SYS system file and is not available before that one gets loaded into the memory.

23.2 Calling convention

The block data management interface entry lives at `jblock_io` \$0706. The function code for a call must be passed in the Y register, e.g.:

```
jblock_io = $0706

    ldy #bm_read
    jsr jblock_io
```

The rest of the parameters belongs to the (X)DCB \$0300-\$030F and `memreix` \$0787 as detailed below. The status codes are returned in the accumulator.

23.3 Function codes

The list of the function codes is defined as follows:

Code	Label	Input	Output	Function
0	<i>bm_read</i>	<i>ddevic</i> , <i>dunit</i> , <i>daux1</i> , <i>daux2</i> , <i>dbufa</i> , <i>memreix</i>	A	Read sector number specified in <i>daux1/2</i> from drive specified in <i>dunit</i> to memory address specified in <i>dbufa</i> and <i>memreix</i> . If the sector is already in the cache, the request will be satisfied from there. If not, the sector will be read from the drive and placed both in the specified buffer and in the cache.
1	<i>bm_write</i>	<i>ddevic</i> , <i>dunit</i> , <i>daux1</i> , <i>daux2</i> , <i>dbufa</i> , <i>memreix</i>	A	Write sector number <i>daux1/2</i> to the cache. Depending on the cache configuration, this function may also update the media immediately (write-through cache) or wait for the <i>bm_flush</i> command before doing that (write-back cache). In the former case the cached sector should be internally either marked as „clean” to avoid getting flushed again later with <i>bm_flush</i> . Alternatively – again, depending on the cache configuration – this function may write the sector directly to the media also invalidating its cache entry, if any.
2	<i>bm_flush</i>	<i>ddevic</i> , <i>dunit</i>	A	Flush all updated sectors to the drive.
3	<i>bm_abort</i>	<i>ddevic</i> , <i>dunit</i>	A	Abandon all buffers on the drive. This is also aliased <i>bm_initz</i> as it is the first function which should get called for a drive on which a media change has been detected.
4	<i>bm_fread</i>	<i>ddevic</i> , <i>dunit</i> , <i>daux1</i> , <i>daux2</i> , <i>dbufa</i> , <i>memreix</i>	A	Like <i>bm_read</i> , but it automatically recognizes and remembers the sector size for the specified disk (see <i>bm_set</i> below). Also, the request is by default not satisfied from the cache. Instead, it is always read directly from the disk, and its cached instance, if any, is abandoned.
5	<i>bm_set</i>	<i>ddevic</i> , <i>dunit</i> , <i>dbyt</i>	A	Set sector size on a drive <i>dunit</i> to the size specified in <i>dbyt</i> .

The *ddevic* \$0300 should be set to \$31 or \$B1, respectively, depending on whether a DCB or an XDCB transfer is being requested (see chapter 22.2.1).

The rest of (X)DCB values are filled in automatically: *dtimlo* is \$08, *dstats* is preset to \$40 or \$80 depending on data transfer direction, *dbyt* according to what was previously set by *bm_set* or *bm_fread*.

24 SpartaDOS X Kernel Interface

24.1 Function

Chapters 3-19 of this document describe the functions and use of the SpartaDOS X library. As it is clear, the library contains a collection of functions performing various tasks generally related to system management. Out of that, its I/O and file system management functions – such as FOPEN, FGETC, FPUTC, FREAD, FWRITE, FCLOSE, RENAME, DELETE etc. – all end up calling the SpartaDOS X kernel, which is directly responsible for dispatching I/O requests, handing them over to the appropriate device drivers, and receiving the statuses. All I/O requests in general – no matter if coming from the library, from the CIO, or made by any other means – end up in the kernel.

As hinted in the previous chapters, the kernel can maintain up to 8 subordinate device drivers, being identified using three-letter identifiers such as CON: or PRN:. Some of them are predefined, whereas others require external device drivers to be loaded to the memory.

The kernel devices can be further subdivided into character devices, special devices and file devices. The differences will be explained later on, for now it will suffice to say that COM:, CON:, NUL: and PRN: are character devices, DSK:, CAR: and PCL: are file devices, and CLK: is a special device.

All devices can maintain up to 15 units, 1-15. Unit 0 is undefined, invoking it will cause undefined results.

24.2 Calling convention

The kernel entry point is located at *jkernal* \$0703. This call requires function code to be loaded to the Y register. For example:

```
jkernal = $0703

    ldy #kd_read
    jsr jkernal
```

Passing just the function code is however not enough, as all kernel calls require additional input to be provided by the caller. The additional parameters can be passed via the kernel parameter table described below.

The status code for all the kernel calls is returned in the accumulator only. It is a positive value (including 0) on success or a negative error code otherwise.

24.3 Page 7 kernel parameter table

The kernel parameter table occupies 128 bytes on page 7, from \$0760 to \$07DF. It is defined as below:

Addr.	Label	Size	Function
\$0760	<i>file_id</i>	1	File handle.
\$0761	<i>device</i>	1	Device identifier. The byte consists of a device number in the high nibble and the actual device unit in the low nibble. Together with <i>path</i> and <i>name</i> this byte constitutes the full file specification for the kernel calls.
\$0762	<i>name</i>	11	File name in the internal format NNNNNNNNXXX. Together with <i>device</i> and <i>path</i> it constitutes the full file specification for the kernel calls.
\$076D	<i>x_name</i>	11	Alternative file name, in the same format as above. Only used in renaming.
\$0778	<i>mode</i>	1	Mode byte. For file I/O its function is as described in section 8.1 of this manual.
\$0779	<i>scan</i>	1	Scan mode, i.e. file searching attributes as described in sections 8.1 and 12.8 of this manual. <i>This byte may also be used to pass additional parameters to other system functions (such as MALLOC), which does not belong to the kernel.</i>
\$077A	<i>attr</i>	1	Attributes to be set in a file being created or changed, as described in sections 8.1 and 12.8 of this manual. <i>This byte may also be used to pass additional parameters to other system functions (such as MALLOC), which does not belong to the kernel.</i>
\$077B	<i>date</i>	3	Date as DD/MM/YY (bytes in this order). YY values 0-79 mean years 2000-2079, values 80-99 are years 1980-1999. <i>This location may undergo constant updates when a program, such as TD, is displaying the current date and time on the screen.</i>
\$077E	<i>time</i>	3	Time as HH:MM:SS (byte order as above). <i>This location may undergo constant updates when a program, such as TD, is displaying the current date and time on the screen.</i>
\$0781	<i>dateset</i>	1	Date/time control variable as described in section 6.8 of the SpartaDOS X User's Manual.
\$0782	<i>addpos</i>	3	Address or file position. For seek/tell operations, this is the file position. For read/write operations, this is the address of the buffer: 16-bit for 6502-compatible memory, 24-bit for 65C816 High RAM. <i>Note that this location is also used by MALLOC, which does not belong to the kernel.</i>
\$0785	<i>bytbuf</i>	2	Amount of bytes to read or write, or, after the I/O, the amount of bytes which actually was read or written. The number specified simply means what it does: an \$FFFF means 65535 bytes, and a 0 means 0 bytes. <i>As above, this location is also used by MALLOC.</i>
\$0787	<i>memreix</i>	1	Memory region index. This byte informs the low level I/O drivers, in which memory region the requested data buffer is located, or, in other words, what memory index should be used to gain the access to the said buffer for the time of I/O transfer. Note that <i>memreix</i> =\$03 may enable <i>addpos</i> to be interpreted as a 24-bit address.
\$0788	-	1	Unused, reserved, should be kept \$00 for upward compatibility.

Addr.	Label	Size	Function		
\$0789	<i>dentry</i>	23	Raw directory entry buffer. The content is defined as follows:		
			Label	Size	Function
			<i>DEsta</i>	1	Status byte of directory entry
			<i>DEsmap</i>	2	Sector number of first sector map
			<i>DEnbyt</i>	3	Number of bytes in file
			<i>DEfnam</i>	8	File name, space-padded up to 8 characters
			<i>DEext</i>	3	File name extension, space-padded up to 3 characters
			<i>DEdat</i>	3	Date of creation/update
			<i>DEtim</i>	3	Time of creation/update
			Within one disk the <i>DEsmap</i> value will be different for every file and directory – so this value sort of uniquely identifies a data file on that disk.		
\$07A0	<i>path</i>	64	The ‘raw’ path. Together with <i>device</i> and <i>name</i> it constitutes the full file specification for the kernel calls. Also, the kernel function 19 <i>kd_check</i> will write its output to this buffer.		

24.4 General distribution of kernel function codes

Kernel functions 0-9 (\$00-\$09) are reserved for character device drivers.

Range	Function
0-7	File operations: a valid handle must then be specified in <i>file_id</i> .
8	Device driver initialization. This function should be invoked when the device driver has been just loaded, also is always invoked on each warm reset. If the driver does not need initializations, this function may safely be left unimplemented (i.e. consisting of an RTS only).
9	Open a new file, its handle gets returned in <i>file_id</i> .

Kernel functions 0-31 (\$00-\$1F) are reserved for file devices. The range 0-9 is identical as shown above, plus there is function 10 which also returns a valid handle.

Function codes 11-255 (\$0B-\$FF) do not operate on open files, and so when they are being called, *file_id* gets implicitly forced to \$FF. Therefore, in an application program, it is best, just after opening a file, to save the value of *file_id* in a private variable and restore it to *file_id* whenever a file operation is about to be performed.

Functions 32-249 (\$20-\$F9) are reserved for special devices. As an example, the kernel functions 100-111 (\$64-\$6F) are reserved for (and partly defined by) clock drivers. Notice that character device drivers and file device drivers also can define „special“ functions in this range.

Kernel functions 250-255 (\$FA-\$FF) are reserved for driver housekeeping. For now the only one defined in this range is the kernel function 255 (\$FF) *kd_xdrv*: it should return

status code 1, if the device driver is capable of performing direct read/write operations on 65C816 High RAM; i.e. if the driver is able to regard *addpos* as a 24-bit address of the buffer when *memreix*=\$03. Otherwise – and this is what happens „normally” with vanilla 6502 I/O drivers – this function should return status 146 (Function not implemented): in this case the kernel extension 65816.SYS will take care of buffering such reads or writes (with performance penalty, of course); and in its absence nothing will happen because in such a case the High RAM would not get allocated, and therefore not referenced either.

All function codes a driver does not implement should return status 146 (Function not implemented) in the accumulator. The only exception is *kd_initz*, which, when not implemented, may consist of just the RTS instruction: the DOS initialization routine, which is its main „user”, does not expect any status code from it.

24.5 Character devices

The most common type of a kernel device is a character device. A character device appears to the application program as a single pseudo-file (so it behaves in quite similar manner as the OS’s devices „E:”, „K:” and such). As such it typically provides only one data stream, even if it can possibly be opened multiple times simultaneously.

Character devices do not have directories, so any path, file name, or file mask specified while calling them will be ignored: only the device name (and possibly the unit number) matters here.

A typical character device has only one unit. The perfect example is CON: – it will legitimately respond as CON1: too, but if you try e.g. CON2:, in the current implementation you will get the same results. It is so because the current driver does not pay any attention to the unit number (i.e. to the low nibble of *device* \$0761), so CON2: will in fact be the same as CON1: and both will be equal to CON: as well. But the units outside the legitimate range (i.e. other than CON: or CON1:) should of course be considered as performing „undefined” operations, only to be possibly defined in the future.

That aside, a character device of course *can* take the unit numbers (up to 15) into account and act differently as different unit specified. An example of a character device which acts this way is NUL: – namely, when reading from it, NUL: (or NUL1:) returns EOF, NUL2: returns infinite number of zeros, NUL3: returns infinite number of random numbers and NUL4: – an infinite number of EOL characters (units outside this range would, as usual, return undefined results).

24.5.1 Character device definition

A character device typically implements these functions:

Code	Label	Input	Output	Function
0	<i>kd_read</i>	<i>file_id</i> , <i>addpos</i> , <i>bytbuf</i> , <i>memreix</i>	<i>bytbuf</i>	Read <i>bytbuf</i> bytes from the file <i>file_id</i> to the address <i>addpos</i> of memory region <i>memreix</i> . Return the amount of bytes read in <i>bytbuf</i> and status in A.
1	<i>kd_write</i>	<i>file_id</i> , <i>addpos</i> ,	<i>bytbuf</i>	Write <i>bytbuf</i> bytes to the file <i>file_id</i> from the address <i>addpos</i> of memory region <i>memreix</i> . Return the amount of

Code	Label	Input	Output	Function
		<i>bytbuf</i> , <i>memreix</i>		bytes written in <i>bytbuf</i> and status in A.
7	<i>kd_close</i>	<i>file_id</i>	-	Close the file, return status in A.
8	<i>kd_initz</i>	-	-	Optional: perform cold or warm initializations. No status.
9	<i>kd_open</i>	<i>device</i> , <i>mode</i> , <i>scan</i> , <i>attr</i>	<i>file_id</i>	Open file, return file handle in <i>file_id</i> and status in A.

A call to any other function code – especially to those which are dealing with directories (see below) – should return error 146 (Function not implemented). A driver wanting (and being able to) support 65C816 should of course respond accordingly to *kd_xdrv*.

24.6 File devices

File devices, or file management system devices, are entities such as DSK:, CAR: or PCL:, with the ability to provide (random) access to many named files and directories. Their general outline is as in a character device, just more functions are available. A file device also typically allows to open more than one file at a time: the maximum global limit is 16, but the actual number may depend on the specific driver or its configuration.

The kernel devices of this type should not be confused with the actual file system drivers (such as ATARIDOS.SYS or FATFS.SYS), which are plug-ins to the default FMS device DSK: and use its internal API.

24.6.1 File device definition

A device of this type may want to implement these functions:

Code	Label	Input	Output	Function
0	<i>kd_read</i>	<i>file_id</i> , <i>addpos</i> , <i>bytbuf</i> , <i>memreix</i>	<i>bytbuf</i>	The back end for FREAD (therefore also for FGETS, FGETC, GETS, GETC): read <i>bytbuf</i> bytes from the file <i>file_id</i> to the address <i>addpos</i> of memory region <i>memreix</i> . Return the amount of bytes read in <i>bytbuf</i> and status in A. The „file“ may as well be a directory.
1	<i>kd_write</i>	<i>file_id</i> , <i>addpos</i> , <i>bytbuf</i> , <i>memreix</i>	<i>bytbuf</i>	The back end for FWRITE (therefore also for FPUTS, FPUTC, FPRINTF, PUTS, PUTC, PRINTF): write <i>bytbuf</i> bytes to the file <i>file_id</i> from the address <i>addpos</i> of memory region <i>memreix</i> . Return the amount of bytes written in <i>bytbuf</i> and status in A. The „file“ may as well be a directory.
2	<i>kd_pos</i>	<i>file_id</i> , <i>addpos</i>		The back-end for FSEEK: set the next read/write position of a file (or directory) as defined in <i>addpos</i> , return status in A. In a file opened for writes (or opened read/write) it is legal to set the position past the file's end: any write will create a gap within the file and the file's size will get updated accordingly. In a file opened read-only it is legal to set the position 1 byte past the end, but attempts to set it further will return error 166 (Range error).
3	<i>kd_gtpos</i>	<i>file_id</i>	<i>addpos</i>	The back-end for FTELL: read the current read/write posi-

Code	Label	Input	Output	Function
				tion of a file (or directory) and store it in <i>addpos</i> . Return status in A.
4	<i>kd_gtlen</i>	<i>file_id</i>	<i>addpos</i>	The back-end for FILELENG: read the length of a file (or directory) and store it in <i>addpos</i> . Return status in A.
5	<i>kd_status</i>	<i>file_id</i>		Function not implemented, reserved for future use.
6	<i>kd_next</i>	<i>file_id</i>	<i>dentry</i>	The back-end to FNEXT: get a „next” (after <i>kd_first</i>) directory entry. Status in A.
7	<i>kd_close</i>	<i>file_id</i>		The back-end to FCLOSE: close the file (or directory). Status in A.
8	<i>kd_initz</i>	-	-	Optional: perform cold or warm initializations. No status.
9	<i>kd_open</i>	<i>device</i> , <i>mode</i> , <i>scan</i> , <i>attr</i> , <i>name</i> , <i>path</i>	<i>file_id</i>	The back-end to FOPEN: open file, return file handle in <i>file_id</i> , status in A. A file being opened for reads (<i>mode</i> =\$x4) must exist and match the specified search criteria, otherwise an error will occur. A file being opened for writes (<i>mode</i> =\$x8), if it exists, will be truncated to 0-length, if it does not exist, it will get created – in both cases a write handle will be returned. A file being opened read/write (<i>mode</i> =\$xC), if it does not exist, will get created, and if it exists, its read/write position will be set to 0 – in both cases a read/write handle will be returned. A file being opened for append (<i>mode</i> =\$x9), if it does not exist, will get created, and if it exists, will get opened and its read/write position will be set to the value of its length (i.e. past the last byte of the data). When a new file gets created, the attributes specified in <i>attr</i> are applied. If opening a directory in direct mode (<i>mode</i> =\$1x), the <i>dentry</i> will afterwards contain the directory header (i.e. the entry referring to that directory itself) and the read/write position in the directory will be set to the first user-visible entry.
10	<i>kd_first</i>	<i>device</i> , <i>scan</i> , <i>name</i> , <i>path</i>	<i>file_id</i> , <i>dentry</i>	The back-end to FFIRST: open directory, return file handle in <i>file_id</i> , the first user-visible directory entry in <i>dentry</i> , and status in A. The functionality of this call is equal to the following call sequence: a) <i>kd_open</i> with <i>mode</i> =\$14 b) <i>kd_next</i>
11	<i>kd_rename</i>	<i>device</i> , <i>name</i> , <i>x_name</i> , <i>scan</i> , <i>path</i>		The back-end to RENAME: rename the file specified in <i>device</i> , <i>name</i> , <i>scan</i> , <i>path</i> to a new name given in <i>x_name</i> . Return status in A.
12	<i>kd_delete</i>	<i>device</i> , <i>name</i> , <i>scan</i> , <i>path</i>		The back-end to REMOVE: delete the specified file, return status in A.
13	<i>kd_setatr</i>	<i>device</i> , <i>name</i> ,		The back-end to CHMOD: in a file specified by <i>device</i> , <i>name</i> , <i>scan</i> , <i>path</i> , apply the attributes specified by <i>attr</i> , re-

Code	Label	Input	Output	Function
		<i>scan,</i> <i>attr,</i> <i>path</i>		turn status in A.
14	<i>kd_credir</i>	<i>device,</i> <i>name,</i> <i>scan,</i> <i>path</i>		The back-end to MKDIR: create the specified directory, return status in A.
15	<i>kd_deldir</i>	<i>device,</i> <i>name,</i> <i>scan,</i> <i>path</i>		The back-end to RMDIR: delete the specified directory, return status in A. The directory to be deleted must contain no further entries, otherwise the call will fail (error 167 „Directory not empty“).
16	<i>kd_chdir</i>	<i>device,</i> <i>name,</i> <i>path</i>		The back-end to CHDIR: change the current directory on the specified <i>device</i> to the one specified by the <i>path</i> and <i>name</i> . Return status in A.
17	<i>kd_cwd</i>	<i>device</i>		The back-end to GETCWD: read the current directory on the specified <i>device</i> and store it in <i>path</i> . Return status in A.
18	<i>kd_boot</i>	<i>device,</i> <i>scan,</i> <i>name,</i> <i>path</i>		The back-end to SETBOOT: mark the specified file as bootable. Return status in A.
19	<i>kd_check</i>	<i>device</i>		The back-end to GETDFREE: return the information on the device specified and store it in <i>path</i> . Return status in A.
20	<i>kd_chvol</i>	<i>device,</i> <i>name</i>		The back-end to CHVOL: change the name of the specified device to <i>name</i> . Return status in A.

The exact implementation depends on a specific device and its nature. For example, the CAR: device, being read-only and containing no directories, implements only this subset: *kd_read*, *kd_pos*, *kd_gtpos*, *kd_next*, *kd_close*, *kd_open*, *kd_first*, *kd_chdir*, *kd_cwd* and *kd_check*, and a part of these – namely *kd_chdir* and *kd_cwd* – are stubs doing really nothing essential but just returning the status 1. Also, CAR: allows only one file to be opened at a time. The PCL: device, being not bootable, implements everything except *kd_boot*. Finally DSK:, the default FMS device, implements the whole table.

As usual, a driver wanting (and being able to) support 65C816 should of course respond accordingly to *kd_xdrv*.

24.7 Special devices

A special device is everything that is not a character device or a file device. In particular, a special device does not perform explicit file transfers, therefore no standard file handles are in use. It does not mean however that there are no data transfers involved.

For example, the only special device that has been defined so far, CLK:, does not allow to open files, but it is of course able to transfer the time and date stamp to and from the clock hardware. To accomplish that it uses the special *date* and *time* registers on the kernel parameter table.

It is also possible for a driver like this to read/write data blocks (for example, in a clock driver, current date and time in text form etc.) from or to the memory. In this case the *ad-dpos*, *bytbuf* and *memreix* will be in use as the buffer address, the buffer size and the buffer memory index, respectively.

Otherwise, a special device is free to use other locations on the kernel parameter table, which do not openly conflict with other device drivers or the kernel itself. Especially *name*, *x_name*, *mode*, *scan* and *attr* may be redefined for your driver's internal purposes (or e.g. *addpos* as a pointer to a larger parameter table and so on).

24.8 Adding your device to the system

24.8.1 The entry point

Your driver will have one main entry point: a procedure which dispatches the function codes passed by the kernel in the Y register, invokes the corresponding functions, then returns the control to the operating system afterwards.

The entry point may look quite similarly to the entry point of a SIO device driver (see chapter 22):

```
V_setme = $07f1
V_popme = $07f4

ourdev lda ext_m           ;see below
      jsr V_setme          ;switch the ext RAM on
      jsr do_dev           ;call the code located there
      jmp V_popme          ;switch the ext RAM off and return
ext_m  .byte $00           ;a copy of init-time EXTENDED
```

The dispatcher, or the procedure labelled here as *do_dev*, will receive the function code in the Y register, as said. Upon return the operating system will receive the status code in the accumulator. All other input or output data depend on the nature of the device, but the obvious place for additional input data and results is the kernel parameter table.

The internal structure of the function dispatcher is completely up to you; just about the only requirement is that the driver should return valid status codes in the accumulator; and when invoked with an unimplemented opcode, should return error code 146 „Function not implemented“.

24.8.2 Setting up driver's memory

The entire process is quite similar to installing any other resident program. The installing code should be stored in a non-resident segment, preferably loaded to page 6, with expansion to pages 5 and 4, if one page is not enough.

The first thing that installation code should do is to save the value of the EXTENDED system variable for later use:

```
lda EXTENDED
sta ext_m           ;see above
```

If the device driver is to be communicating with an externally added hardware, it may be good idea to check now for the presence of the hardware. The way to do that is completely dependent on the device (you may for example call the driver's *kd_initz* function invoking it directly). Also the reaction to the lack of response depends on the device: although in that case most of the time the driver should probably not be installed and exit instantly, some devices (like PCL: for example) do allow the installation in the hope that the lack of the communication is of transient nature and the external device will start responding in the future.

24.8.3 Registering the driver

The key entity here is the table pointed to by the symbol *DEVNAME*. That table contains names for all installed devices, in ASCII form. The table's size (in entries) is indicated at *DEVSPEC-1*: e.g. if the byte there has a value of 8, this means that the entire table consists of 8 entries, 4 bytes each, or 32 bytes total.

*Note: in SpartaDOS X 4.2 it was not possible to register new kernel devices – an entity corresponding to the *DEVNAME* table did exist, but was living in ROM.*

A single entry consists of four bytes, the first three ones are occupied by the identifier proper, encoded as an uppercase ASCII string (e.g. „DSK”); the fourth byte is unused and has a value of \$20 (ASCII space). A free entry is filled with spaces.

Usually the first six entries are occupied by the devices already pre-installed by the system.

When a device driver is about to be registered, the first thing to do is to find a free entry in that table. The same code may at the same time check, if a driver with a conflicting ASCII identifier is not already installed. Once a free entry is found, we need to fill it with our identifier. Let us assume that our device identifier is „WRK:”

```

        lda  DEVSPEC-1
        asl
        asl
        sta  _maxix

        ldx  #$000
_srchdev lda  DEVNAME,x
        cmp  #$20
        beq  _found
        cmp  _ourdevn
        bne  _nxn
        lda  DEVNAME+1,x
        cmp  _ourdevn+1
        bne  _nxn
        lda  DEVNAME+2,x
        cmp  _ourdevn+2
        beq  _ups
_nxn    inx
        inx
        inx
        inx
        cpx  _maxix
        bcc  _srchdev

```

```

        ...                ;error: no more devices allowed
        rts

_maxix  .byte 0
_devix  .byte 0
_ourdevn .byte "WRK"

_ups    ...                ;error: already installed
        rts

_found  txa                ;free entry found, remember id
        lsr
        lsr
        sta _devix         ;save actual id

        ldy #$00           ;fill entry with our identifier
_se     lda _ourdevn,y
        sta DEVNAME,x
        inx
        iny
        cpy #$03
        bcc _se

```

The next step is to add the driver's entry point address to the appropriate kernel table. The entry point address should be stored somewhere within the installation segment as a pointer:

```
_ourdevp .word ourdev
```

Now store this pointer in the kernel device table *dev_vector* \$0750-\$075F at the index found as shown above:

```
dev_vector = $0750

        lda _devix         ;get actual id
        asl                ;multiply by 2
        tax
        lda _ourdevp
        sta dev_vector,x
        lda _ourdevp+1
        sta dev_vector+1,x

```

24.8.4 Enabling micro-buffers

Now, if your driver defines a file device, or any other one which is dealing with real file streams, enable the micro-buffers for it:

```

        ldx _devix
        lda _bufbit,x
        ora COMTAB2-1
        sta COMTAB2-1

```

The *_bufbit* table should contain 8 values with consecutive bits set:

```
_bufbit .byte $01,$02,$04,$08,$10,$20,$40,$80
```

24.8.5 Configuring for the OSS CLI

If the device has the three-character identifier only, zero-out the corresponding entry in the DEVSPEC table:

```
ldx _devix
lda #$000
sta DEVSPEC,x
```

This will cause the legacy command line parser (also known as the *OSS Command Line Interface*, called by old programs) to translate the device's name to „Dxxx:” form, where „xxx” is the three-letter identifier („WRK:” in our case, so the result will be „DWRK:”).

Otherwise, if you want the legacy command line parser to translate that three-letter identifier to the CIO format, the corresponding entry in the DEVSPEC table should contain the appropriate letter. Say, you want the device to be represented in CIO as „W:”, so you do this instead:

```
lda #'W'
sta DEVSPEC,x
```

However, for this to work, your driver must, obviously, also install and maintain the appropriate CIO device driver „W:”, otherwise programs will not find it.

24.8.6 Initialization

A good practice is to call now the initialization function of the device, if not already done so:

```
device = $0761
```

```
lda _devix
asl
asl
asl
asl
sta device
ldy #kd_initz
jsr jkernel
```

It is up to the device driver to check for errors and react appropriately. Just remember that your driver is already registered in the system, so it is not a good idea to just exit now and release all the driver's memory without cleaning up the system tables we have edited in this chapter.

24.8.7 Completion

The last step is to let the system know that the relocatable modules belonging to the driver should be kept resident in the memory, and quit:

```
dec INSTALL
rts
```

24.9 Operation

As hinted above, the communication of your driver with the hardware, and its internal organization, as being dependent on the device's nature as well as the programmer's personal style and taste, is basically up to you. There are however some points to consider when it comes to the other side of things, namely the interaction of the driver with the SpartaDOS kernel and the operating system in general.

24.9.1 Assigning file handles

One of the values your device driver must give back to the system upon a successful return from the *kd_open* or *kd_first* function is the handle of the file which has been thus opened. This handle should be returned in *file_id* \$0760. It cannot be an arbitrary number: the system allows up to 16 files to be open simultaneously and the file handles cannot be conflicting (i.e. no two files must be assigned the same handle).

To allocate a new file handle, your code has to scan the table *dev_index* \$0740-\$074F. Its length is 16 bytes. The first step is to find a free entry in that table; to accomplish that simple search for \$FF, as all free entries always have this value:

```
dev_index = $0740

newhandle ldy #$00
          lda #$ff
_srch     cmp dev_index,y
          beq _found
          iny
          cpy #$10
          bcc _srch
          lda #161                ;error: too many channels open
          rts
_found    sty file_id
          lda #$01                ;success
          rts
```

If the *dev_index* \$0740-\$074F has no more free entries, this means that all available file handles are already in use and no new file can be opened. In this case the error 161 should be returned as shown above. Otherwise, the index of the entry we are going to allocate now should be written to *file_id* \$0760, and status \$01 should be returned.

There is no reason for your code to write anything back to *dev_index* \$0740-\$074F at any time, the system will take care of its contents.

Obviously, you may (and surely will) want to make provisions for your device driver to associate the handle with the actual file and identify it internally during any read/write operations being performed later, but how to do that it is completely up to you.

Also, your device driver may of course limit the number of files that can be opened simultaneously.

24.9.2 Global RAM locations your driver may use internally

As long as your driver talks directly to the hardware and does not call any underlying system functions, the same RAM locations are available as listed in chapter 22.4.2, and the same restrictions apply.

Otherwise you may use the same locations temporarily, but keeping in mind, that any system call may destroy their values; and it is safest to assume that a system call destroys all of them, because you never know, what kind of device and device driver would be effectively called: when things comes to the SIO Interface, it can be a serial drive, a parallel drive, a HDD, a ramdisk etc. and every device driver is free to use that area to its needs.

24.9.3 Memory management within the driver

During data transfers the memory must always be setup so that the I/O memory is available to the I/O routines; in other words, the memory banks should be first switched as requested in the *memreix* \$0787, then the I/O should be performed, then the memory should be switched back to restore the previous configuration. Setting up the memory is accomplished using a pair of system calls: *V_setme* \$07F1 to swap the memory in, and *V_popme* \$07F4 to swap it out afterwards.

However, as long as your kernel driver calls the underlying SIO via *jsector_io* \$070C vector (see chapter 22), it should not trouble itself with anything like that: as long as the actual transfers are done by the SpartaDOS SIO Interface, your driver may happily pass the *memreix* \$0787 state to it without bothering itself with setting up memory banks.

This is the way the PCL: driver works, which uses the SIO as its transport layer. But in fact it is a rare case and probably most device drivers will want to talk directly to the hardware, which also means that they become responsible for managing the memory correctly (or otherwise the system will quickly get into severe trouble).

In this case you should generally follow the same directions as for a SIO device driver as described in chapter 22.4.3.

25 Misc_Interface

25.1 Function

The *misc_* interface contains a set of miscellaneous auxiliary functions, dealing mostly with pathname parsing, directory formatting and minor system management. Some of the calls duplicate few functions normally contained within the SpartaDOS X library, to make the functionality available at times when the library itself has been swapped out of the CPU address space. These, too, are mostly used by system components such as X.COM.

The misc_ interface's code is physically located inside the SPARTA.SYS system file and is not available before that one gets loaded into the memory.

25.2 Calling convention

The *misc_* entry point is located at *jmisc* \$0709. This call requires function code to be loaded to the accumulator. For example:

```
jmisc  = $0709

      lda #misc_formdir
      jsr jmisc
```

There is no consistent way of passing parameters to *misc_*: each function defines them according to individual requirements (see below). *Also, misc_ does not preserve registers!*

25.3 List of function codes

Unless noted, most *misc_* functions do not return any specific status codes, nor errors are handled automatically (by calling *U_FAIL* or otherwise). The function codes themselves are defined as follows:

Code	Name	Synopsis
0	<i>misc_initz</i>	Initialization. This function will, among other things, read out the \$DAY-TIME variable and setup the internal time settings accordingly.
1	<i>misc_gefina</i>	An equivalent to <i>U_GEFINA</i> . Unlike that one, <i>misc_gefina</i> gets input from <i>copybuf</i> (COMTAB+\$BF); the output is stored into <i>device</i> (\$0761), <i>path</i> (\$07A0) and <i>name</i> (\$0762). A decoding error is signalled by setting <i>device</i> (\$0761) to \$FF.
2	<i>misc_gepath</i>	An equivalent to <i>U_GEPATH</i> . As above: input from <i>copybuf</i> (COMTAB+\$BF), output into <i>device</i> (\$0761) and <i>path</i> (\$07A0). A decoding error is signalled by setting <i>device</i> (\$0761) to \$FF.
3	<i>misc_proname</i>	An equivalent to <i>PRO_NAME</i> . Input: path name to crunch pointed to by <i>zsba</i> (\$47-\$48), <i>Y + 1</i> is an offset into that. Output: <i>name</i> (\$0762) decoded name, <i>Z=1</i> if no characters in <i>name</i> , <i>A</i> = delimiter character, <i>zsba</i> = new pointer, <i>Y</i> = new offset.
4	<i>misc_formdir</i>	Converts a raw directory entry (stored in <i>dentry</i> =\$0789) into a readable text line stored in <i>lbuf</i> (COMTAB+\$3F). The formatting mode bitfield is expected at <i>attr</i> (\$077A) – the meaning of the bits is the same as <i>mode</i> (\$0778) in <i>misc_dopen</i> . The size of the resulting string is returned in <i>Y</i> .
5	<i>misc_convdc</i>	Convert a 24-bit unsigned integer, passed in <i>divend</i> (3 bytes at COMTAB-6),

		into an ASCII string of decimal digits saved in <i>decout</i> (8 bytes at COMTAB-\$13).
6	<i>misc_dopen</i>	The back-end of FDOPEN. Input: formatting mode bitfield in <i>mode</i> (\$0778) – for the meaning of bits, see FDOPEN; file scan attributes in <i>scan</i> (\$077A); device and pathname specification in <i>device</i> (\$0761), <i>path</i> (\$07A0) and <i>name</i> (\$0762) in the usual internal format (see chapter 6). Output: status in A. The handle for the opened I/O stream is stored into <i>file_id</i> (\$0760), but is also remembered internally, so that <i>misc_dgetc</i> and <i>misc_dclose</i> can select it automatically. One byte only is assigned to that, therefore only one stream of formatted directory data may be opened at a time.
7	<i>misc_dgetc</i>	The back-end of FDGETC. Input: none (the handle for the I/O stream is setup automatically). Output: the next character of the formatted directory in ICAX6Z (\$2F), status in A. When there are no more lines in the directory to list files, but just the „FREE SECTORS” line remains to be read out in the buffer, the first byte of <i>dentry</i> (\$0789) gets zeroed out so that a program may catch the end of the actual listing before the free sectors information.
8	<i>misc_dclose</i>	The back-end of FDCLOSE. The handle for the I/O stream is setup automatically, then this stream gets closed.
9	<i>misc_setpath</i>	Fetch \$PATH and setup the internal system variables accordingly.
10	<i>misc_openpath</i>	Find a file through \$PATH searching, then open it. Input: file name to search for must be in <i>name</i> (\$0762); <i>path</i> (\$07A0) must be empty (must contain a zero on its first byte). The <i>mode</i> (\$0778) should be set to \$24. Output: status in A.
11	<i>misc_convdc32</i>	As of SpartaDOS X 4.40: convert a 32-bit unsigned integer, passed in <i>divend</i> (4 bytes at COMTAB-6), into an ASCII string of decimal digits saved in <i>decout2</i> (10 bytes at COMTAB-\$15).
12	<i>misc_validfn</i>	As of SpartaDOS X 4.50: check if the ASCII code given in the Y register belongs to a valid pathname character. Return values: C=0 – it is a valid pathname character C=1 – it is an invalid pathname character or a valid delimiter Z=1 – the delimiter is a ‘.’ or a ‘*’ A – the input character adjusted as necessary: letters converted to upper case, \ converted to >.

The functions of most interest to application programs seem to be 4-8 and 10-12. The *misc_formdir* (4) is commonly used by file managers (such as MENU or Sparta Commander) to present directories in formatted manner.

More function codes may be defined in the future. *Remark: calling an undefined function may cause the misc_ interface to perform an undefined action.*

26 Hwctrl Interface

26.1 Function

The Hwctrl interface provides low-level cartridge control, that is, the control over the memory area of \$A000-\$BFFF. This subsystem may prove useful to applications which want both to use SpartaDOS X library and to temporarily utilize the RAM area beneath.

26.2 Calling convention

As of SpartaDOS X 4.50 the Hwctrl entry point is located at *jhwctrl* \$070F. This call requires function code to be loaded to the X register. For example:

```
jhwctrl = $070f

ldx #hw_id
jsr jhwctrl
```

The Hwctrl interface provides no status codes: it is assumed that a call always succeeds.

The interface is in fact present in the system as of SpartaDOS X 4.43. If the compatibility with older SpartaDOS X version is of concern, it should be checked before the call, if the byte at \$070F contains the 6502 JMP instruction opcode value (\$4C).

26.3 List of function codes

Code	Label	Function
0	<i>hw_lib</i>	Activate the SpartaDOS X library ROM in area \$A000-\$BFFF (default).
1	<i>hw_ram</i>	Disable the SpartaDOS X library and activate the internal RAM instead.
2	<i>hw_bas</i>	Disable the SpartaDOS X library and activate the internal BASIC instead. The caller should check afterwards, if a valid cartridge ROM has appeared at \$A000-\$BFFF (because e.g. a 1200XL has no BASIC and many other XL/XE systems may have internal BASIC deactivated).
3	<i>hw_ext</i>	Disable the SpartaDOS X library and activate the external cartridge instead.
4	<i>hw_id</i>	Return SpartaDOS X cartridge type in A: bit 7 = 1 – pass-through hardware bits 6-4 – reserved, %000 bits 3-0 – cartridge identification number: \$0 – SDX 128 \$1 – MaxFlash 1Mbit \$2 – MaxFlash IDE \$3 – MaxFlash 8Mbit \$4 – TurboFreezer 2005 \$5 – FooCart/SIDE1 \$6 – SIC! Cart \$7 – IDE+2.0 \$8 – Ultimate 1MB/Incognito \$9 – MyIDE II RAM \$A – SIDE2 \$B – MyIDE II ROM \$C – SuperSDX Cart

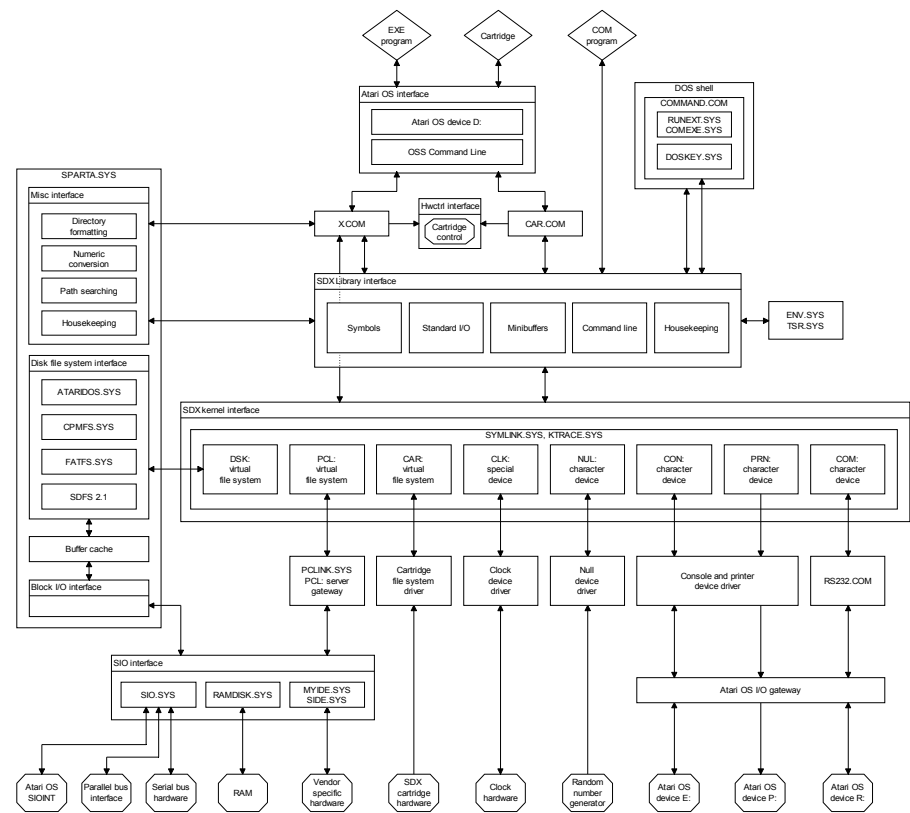
		\$D – RAM-Cart \$E – SIC+ \$F – SIDE3 Additionally, as of SpartaDOS X 4.50, return an ASCII string identifying the specific build into the <i>copybuf</i> (COMTAB+\$BF)
--	--	--

This list, and especially the list of cartridge identification numbers, may be expanded as new hardware appears.

Remark: calling the Hwctrl using an undefined opcode will cause undefined results.

27 SpartaDOS X Block Diagram

The following diagram shows schematically how various system components are internally related inside SpartaDOS X:



Remark: Scale the viewing size for this page to approximately 300 % or more.

Index

System functions and variables

- 65C02, 90
- 65C802, 90
- 65C816, 8s., 63, **81**, 85s., 88ss., 93s., 100, 119, 125, 136, 138s., 141
- addpos, 10, 14, 36, 91s., 101, 113, 118, **136**, 138, 142
- attr, 33, **34**, 52, 91s., **136**, 142, 148
- Axlon, 9, 16s.
- bm_abort, **134**
- bm_flush, **134**
- bm_fread, **134**
- bm_initz, **134**
- bm_read, **134**
- bm_set, **134**
- bm_write, **134**
- BUFOFF, **21**, 69, 75s.
- BUILDDIR, **53**
- bytbuf, 10, 36, 91s., 101, **136**, 142
- CHDIR, **52**, 112, 141
- CHMOD, **52**, 112, 140
- CHVOL, **54**, 112, 141
- CKSPEC, **55**, 112
- COMFNAM, **21**
- COMTAB, 16, **17**, 18, 21s., 28, 55, 69, 75, 78, 124, 148s., 151
- COMTAB2, 17, 90, 101, 125
- copybuf, 28, 148, 151
- CURDEV, 25, **54**, 125
- date, **136**, **141**
- dateset, **136**
- daux1, **125**, 134
- daux2, **125**, 134
- daux3, **125**
- daux4, **125**
- dbfx1, **125**
- dbfx2, **125**
- dbufa, **124**, 129, 134
- dbyt, **125**, 129, 132, 134
- DCB, **124**, 131
- dcmnd, **124**
- ddevic, 53, **124**, 131, 134
- decout, **149**
- decout2, **149**
- DELETE, 135
- dentry, 33, 47, 74, 118, **137**, 148s.
- dev_index, **146**
- dev_vector, **144**
- device, **25**, 26, 29, 39, 42, 54, 71, 73s., 136ss., 148s.
- DEVNAME, **73**, **143**
- DEVSPEC, 143, **145**
- DIV_32, **55**, 105
- divend, 22, 69, 113, **148s.**
- DIVIO, **43**, 78, 112
- dstats, **124**, 129, 131, 134
- dtimlo, **125**, 134
- dunit, 53, 98, **124**, 128, 131, 134
- dunuse, **125**
- ENV_P, **31**
- ERR_DMSG, **20**
- ERR_GMSG, **20**
- EXTENDED, **15**, 49, 59s., 85, 126, 142
- FADD, 110
- FASC, **42**
- FCLEVEL, **35**
- FCLOSE, **34**, 35, 82, 112, 135, 140
- FCLOSEAL, **34**, 43, 112
- FDCLOSE, **47**, 48, 112, 149
- FDGETC, **47**, 112, 149
- FDIV, 110
- FDOPEN, **47**, 48, 112, 149
- FFIRST, **47**, 72, 74, 112, 120, 122, 140
- FGETC, **35**, 112, 135, 139
- FGETS, **35**, 102, 139
- file_id, 34ss., 39, 42ss., 47, 101, 122, **136**, 137, 146, 149
- FILE_P, 22, 24ss., 29, **33**, 43, 47ss., 51ss., 75, 100, 121
- FILELENG, 35, **36**, 113, 140
- FILLEN, **21**, 22, 24, 26
- FLAG, 27, **49**, 50, 75
- FMUL, 110
- FNEXT, 35, **47**, 113, 120, 122, 140
- FOPEN, **33**, 74, 100, 135, 140
- FORMAT, **53**
- FPI, 109
- FPRINTF, **37**, 44, 103, 139
- FPUTC, **35**, 44, 113, 135, 139
- FPUTS, **35**, 44, 102, 139
- FREAD, 14, 35, **36**, 100ss., 117, 135, 139

FSEEK, 35, **36**, 72, 113, 139

FSUB, 110

FTELL, 35, **36**, 113, 139

FWRITE, 14, 35, **36**, 100ss., 135, 139

GETC, **42**, 113, 139

GETCWD, **52**, 113, 141

GETDFREE, **53**, 112, 141

GETENV, **31**, 96, 126

GETS, **43**, 113, 139

H_CHDIR, **112**

H_CHKDSK, **112**

H_CHMOD, **112**

H_CHVOL, **112**

H_CKSPEC, **112**

H_DIVIO, **112**

H_DIVS16, **105**

H_DIVS32, **107**

H_DIVU16, **105**

H_DIVU32, **107**

H_ERROR, **112**

H_EXPAND, **112**

H_FADD, **110**

H_FAIL, **90**

H_FCLOSE, **112**

H_FCLSAL, **112**

H_FDCLS, **112**

H_FDGETC, **112**

H_FDIV, **110**

H_FDOPEN, **112**

H_FENCE, 61

H_FFIRST, **112**

H_FGETC, **112**

H_FGETS, **102**

H_FILE_P, **100**, 103, 112s.

H_FLEN, **113**

H_FMUL, **110**

H_FNEXT, **113**

H_FOPEN, **100**

H_FPI32, **109**

H_FPI32S, 109, **110**

H_FPRNTE, **103**

H_FPUTC, **113**

H_FPUTS, **102**

H_FREAD, **101**

H_FSEEK, **113**

H_FSPEC, **113**

H_FSUB, **110**

H_FTELL, **113**

H_FWRITE, **101**, 102

H_GEFINA, 100, **113**

H_GEPATH, 100, **113**

H_GETADR, **113**

H_GETC, **113**

H_GETCWD, **113**

H_GETENV, **96**

H_GETFLG, **113**

H_GETKEY, **113**

H_GETNUM, **113**

H_GETPAR, **113**

H_GETS, **113**

H_GONOFF, **113**

H_IFP32, **109**

H_IFP32S, **109**

H_LOAD, 88, **99**

H_LOWERC, **111**, 113

H_MALLOC, **92**, 93

H_MEMCPY, **94**

H_MEMSET, **95**

H_MFREE, **92**, 93

H_MKDIR, **113**

H_MULS16, **106**

H_MULS32, **108**

H_MULU16, **106**

H_MULU32, **108**

H_NUMENV, **96s.**

H_PARAM, **113**

H_PATH, **113**

H_PERROR, **113**

H_PRINTF, **103**, 104s.

H_PUT_V, **103**

H_PUTC, **113**

H_PUTENV, **96s.**

H_PUTS, **113**

H_RANDOM, **111**

H_REMOVE, **113**

H_RENAME, **113**

H_RENDIR, **113**

H_RMDIR, **113**

H_SADD, **85**

H_SADDIZ, **95**

H_SBOOT, **113**

H_SFAIL, **89**, 90

H_SFIRST, **85**

H_SIOADD, **98**

H_SLASH, **113**

H_SNEXT, **85**

H_SPRNTE, **104**

H_TIME, **110**

- H_TOKEN, **113**
- H_UPPERC, **111**, 113
- H_VPRNTE, **103**
- H_XDIVIO, **113**
- H_XFAIL, 89, **90**
- hw_bas, **150**
- hw_ext, **150**
- hw_id, **150**
- hw_lib, **150**
- hw_ram, **150**
- Hwctrl, 117, **150**, 151
- I_FMTTD, 59
- IFP, 109
- INSTALL, 12, 91, **126**
- jblock_io, **133**
- jext_off, **13**
- jext_on, **13**
- jfsymbol, 17, **59**
- jhwctrl, **150**
- jkernel, **135**
- jmisc, **148**
- jsector_io, **124**, 147
- JSIOINT, 124
- kd_boot, **141**
- kd_chdir, 26, **141**
- kd_check, 137, **141**
- kd_chvol, **141**
- kd_close, **139s.**
- kd_credir, **141**
- kd_cwd, 26, **141**
- kd_deldir, **141**
- kd_delete, **140**
- kd_first, **140**, 146
- kd_gtlen, **140**
- kd_gtpos, 139
- kd_initz, **139s.**
- kd_next, **140**
- kd_open, **139s.**, 146
- kd_pos, **139**
- kd_read, **138s.**
- kd_rename, **140**
- kd_setatr, **140**
- kd_status, 140
- kd_write, **138s.**
- kd_xdrv, **137**, 139, 141
- kernel, 135
- LBUF, **21**, 69, 75s.
- lsio, 124
- LSIO, **124**
- MALLOC, **10s.**, 12ss., 75, 86, 91ss., 101, 116, 118, 136
- memlo, 5, 9ss., 60, 75, 77
- memreix, 7, 36, 101, 124, 127, 129, 133s., **136**, 138, 142, 147
- memtop, 9s.
- micro-buffers, **35**
- misc_, 47, 73, **148**, 149
- misc_convdc, **148**
- misc_convdc32, **149**
- misc_dclose, **149**
- misc_dgetc, **149**
- misc_dopen, 148, **149**
- misc_formdir, **148**, 149
- misc_gefina, 73, **148**
- misc_gepath, **148**
- misc_initz, **148**
- misc_openpath, **149**
- misc_proname, **148**
- misc_setpath, **149**
- misc_validfn, **149**
- MKDIR, **52**, 113, 141
- mode, **33**, 34, 48s., **136**, 142, 148s.
- MUL_32, **55**, 105
- name, 25s., **29**, 136s., 142, 148s.
- NUMENV, **31**, 96s., 126
- path, **25**, 26, **29**, 50, 52, **53**, 74, 136s., 148s.
- PORTB, 9, 16s.
- PRINTF, **39**, 68, 103, 139
- PRO_NAME, **29**, 148
- PUT_V, **44**, 70, 78, 103
- PUTC, **39**, 113, 139
- PUTENV, **31**, 96s., 126
- PUTS, **39**, 113, 139
- REMOVE, **51**, 113, 140
- RENAME, **51**, 113, 135, 140
- RENDIR, **51**, 113
- RMDIR, **51**, 113, 141
- runad, **88**
- S_ADD, **60**, 85
- S_ADDIZ, **57**, 95
- S_CLEAR, **60**
- S_LOOKUP, 7, **59**, 60, 85
- S_NEXT, **59**, 85
- scan, 26, 33, **34**, 47s., 51s., 91s., 121, **136**, 142
- SD_DPATH, **123**
- SD_EXCB, **123**
- SD_FIRST, 120, **122**, 123

SD_FPATH, **123**
 SD_INCB, **123**
 SD_INIT, **121**, 122s.
 SD_NEXT, 120, **122**
 SD_VARS, 122
 SETBOOT, **52**, 113, 141
 sio_index, **125**, 128, 131
 sio_vector, **125**, 126ss.
 SIOV, **124**, 127
 SpartaDOS X 4.20, 9, 34
 SpartaDOS X 4.22, 34, 54
 SpartaDOS X 4.40, 9s., 40, 44, 51ss., 57, 59, 149
 SpartaDOS X 4.41, 33, 48, 54
 SpartaDOS X 4.42, 20, 33, 42, 51, 73, 76
 SpartaDOS X 4.43, 5s., 10, 44, 54, 150
 SpartaDOS X 4.44, 22, 69
 SpartaDOS X 4.46, 13, 17, 19, 47, 50, 59, 90
 SpartaDOS X 4.47, 5, 9, 11, 13, 20, 22, 24, 31, 36, 40, 49s.
 SpartaDOS X 4.48, 23, 39, 42
 SpartaDOS X 4.50, 7, 29, 43, 74, 77, 81, 149ss.
 STATUS, **77**
 SYMBOL, 60, 85
 SYSLEVEL, **34**, 50, **61**
 T_, **16**, 17
 time, **136**, **141**
 TOUPPER, **55**, 112s.
 U_ERROR, **20**, 112
 U_EXPAND, **29**, 112
 U_FAIL, **19**, 20, 23ss., 29, 34ss., 42s., 47, 49, 51ss., 77, 90, 122, 148
 U_FSPEC, **26**, 113
 U_GEFINA, **25**, 26, 29, 71, 73, 113, 148
 U_GEPATH, **25**, 29, 113, 148
 U_GETADR, **22**, 23, 113
 U_GETATR, **26**
 U_GETFLG, **27**, 113
 U_GETKEY, **44**, 82, 113
 U_GETNUM, **22**, 69, 113
 U_GETPAR, 27, 75, 113, 121
 U_GONOFF, **23**, 27, 113
 U_LOAD, 27, 34s., **49**, 50, 75ss., 85s., 99, 115, 120s.
 U_PARAM, **28**, 113
 U_PATH, **29**, 74, 113
 U_PERROR, **20**, 113
 U_SFAL, **19**, 89s.
 U_SLASH, **23**, 27, 113
 U_TOKEN, **24**, 42, 70, 113
 U_UNLOAD, 34, **50**, 60
 U_XFAIL, **19**, 89s.
 V_popme, **13**, **15**, 129, 131, 147
 V_PRINTF, 104
 V_setme, **13ss.**, 129, 131, 147
 vdos, **50**, 57
 VPRINTF, **44**, 70
 x_name, **136**, 142
 XCOMLI, 75, **76**, 77
 XDCEB, **124**, 125, 127, 131, 133s.
 XDIVIO, **43**, 78, 113
 XRWMEM, **14**, 93
 _65816, **83**
 _816FLG, 101
 _CIO, **61**
 _CRUNCH, **28**
 _DOS, **57**
 _EDIT, **61**
 _INITZ, **57**



SPARTA DOS X

Programming Guide

SpartaDOS X is the most advanced disk operating system ever designed for your ATARI 8-Bit computer. While still supporting the first generation machines ATARI 400 & 800 with at least 48 KB of memory, the all new SpartaDOS X supports all modern devices and gadgets available for the ATARI 8-Bit line of computers. And it is a living project.



The "SpartaDOS X Programming Guide" unveils to coders the necessary insights and therefore enables them to write applications and system programs for SpartaDOS X.

Reasonable knowledge on Atari 8-bit programming and advanced user experiences of SpartaDOS X are mandatory for successful coding. Additionally, some decent system specific hardware information is a good prerequisite for conquering the A8 environment. The information provided in this guide is not covered in the "SpartaDOS X User Guide".



The assembly listings in this guide are written in pseudo-code modeled after the MAE assembler syntax. It is close to the MAC/65 syntax, and differs in details from the syntax used by the cross-assembler MADRS. The programmer would be willing to adjust it to the preferred assembler.

SPARTADOS X Programming Guide